

MODUL **PELATIHAN JAVA** **FUNDAMENTAL**



DAFTAR ISI

DAFTAR ISI	i
BAB I PENGENALAN JAVA, OUTPUT, TIPE DATA & VARIABEL, OPERATOR, DAN INPUT	1
1.1 Pengenalan Java	1
1.2 Yang Harus Disiapkan	1
1.2.1 JDK (Java Development Kit)	1
1.2.2 IDE	2
1.3 Cara Menjalankan Java	2
1.4 Method Output	3
1.5 Tipe Data	3
1.5.1 Primitif.....	4
1.5.2 Komposit	4
1.6 Variabel.....	6
1.7 Method Input	7
1.7.1 BufferedReader	7
1.7.2 Scanner.....	8
1.8 Operator	9
BAB II MEMAHAMI STRUKTUR & ATURAN PENULISAN SINTAKS, PERCABANGAN, PERULANGAN, DAN METHOD	10
2.1 Struktur Dasar Program Java	10
2.1.1 Deklarasi Package.....	10
2.1.2 Impor Library.....	10
2.1.3 Bagian Class.....	11
2.1.4 Method Main	11
2.1.5 Gaya Penulisan Case.....	11
2.2 Percabangan	12

2.2.1 Percabangan IF ELSE.....	12
2.2.2 Percabangan Ternary Operator	12
2.2.3 Percabangan Switch Case.....	12
2.3 Perulangan / Iterasi	13
2.3.1 Counted Loop.....	13
2.3.2 Uncounted Loop.....	14
2.4 Method.....	14
2.4.1 Cara Membuat Method	15
2.4.2 Cara Memanggil Method	15
2.4.3 Menggunakan Parameter.....	16
2.4.4 Method Yang Mengembalikan Nilai	16
2.4.5 Static & Non-static.....	16
BAB III PENGANTAR OOP, CLASS, OBJECT, & ACCES MODIFIER	18
3.1 Pengenalan OOP.....	18
3.2 Class.....	18
3.3 Object	19
3.4 Constructor.....	20
3.5 Access Modifier	21
3.5.1 Private	22
3.5.2 Default.....	22
3.5.3 Protected.....	23
3.5.4 Public.....	23
BAB IV ENKAPSULASI, KEYWOD THIS, COLLECTION TYPE, DAN MEMBUAT CRUD MENGGUNAKAN COLLECTION TYPE	24
4.1 Enkapsulasi	24
4.1.1 Cara Membuat Method Setter & Getter	25
4.1.2 Cara Menggunakan Method Setter & Getter	26
4.2 This	26

4.3 Collection Type	27
4.3.1 ArrayList	27
4.3.2 HashMap	28
BAB V INHERITANCE & METHOD OVERRIDING	31
5.1 Inheritance	31
5.1.1 Cara Membuat Inheritance	32
5.1.2 Keyword extends	34
5.2 Method Overriding	35
BAB VI ABSTRACT CLASS, INTERFACE & OVERLOADING	36
6.1 Abstract Class	36
6.1.1 Cara Membuat Abstract Class	37
6.2 Interface	39
6.3 Perbedaan Abstract Class & Interface	40
6.4 Overloading	41
BAB VII PENGENALAN SPRING DAN SPRING BOOT	43
7.1 Framework	43
7.2 Spring Boot	44
7.3 Maven dan Gradle	45
7.3.1 Gradle	45
7.3.2 Maven	45
7.4 Set Up Spring Boot di Visual Studio Code	47
BAB VIII MODEL VIEW CONTROLLER (MVC) DI JAVA	53
8.1 Konsep MVC (Model-View-Controller)	53
8.2 Alur MVC	53
8.3 Keuntungan Penggunaan MVC	54
8.4 Implementasi MVC dalam Java Spring Boot	54
8.5 Contoh Implementasi	55
BAB IX THYMELEAF	61

9.1 Pengertian Thymeleaf	61
9.2 Thymeleaf Dan Spring Boot.....	63
9.3 Atribut Dalam Thymeleaf	66
9.4 Expression Dalam Thymeleaf	67
9.4.1 Thymeleaf Variable Expression	67
9.4.2 Thymeleaf Link Expression	74
9.4.3 Thymeleaf Fragment Expression.....	75
9.5 Cara menambahkan CSS dan JS Kedalam Thymeleaf	77
9.6 Penggunaan If Else dan Switch Case	79
9.7 Perulangan Dalam Thymeleaf	84
BAB X MENGGUNAKAN JPA DENGAN THYMELEAF DI JAVA.....	87
10.1 Pengenalan JPA.....	87
10.1.1 Definisi JPA (Java Persistence API).....	87
10.1.2 Perbedaan antara JDBC dan JPA:.....	87
10.1.3 Entity dan Entity Class	87
10.1.4 Anotasi JPA untuk Entity Class:	88
10.2 Konfigurasi Spring Boot dengan JPA.....	88
10.2.1 Mengatur sumber data (Data Source):	88
10.2.2 Konfigurasi Entity Manager Factory	88
10.2.3 Repository Interface	89
10.3 Membuat Project CRUD Dengan JPA.....	89
10.3.1 Membuat Project	89
10.3.2 Mengimplementasikan konsep CRUD menggunakan JPA di SpringBoot	92
BAB XI RELASI ANTAR TABEL (ASSOCIATION MAPPING)	108
11.1 Association Mapping	108
11.2 Implementasi Kode.....	108
11.2.1 Membuat Entity Sales	109
11.2.2 Akses Database dengan SalesRepository	110

11.2.3 Membuat End-Point dengan SalesController.....	110
11.2.4 Menampilkan Data dengan Thymelaf	111
11.3 Menambahkan Data.....	112
11.3.1 Membuat End-Point Untuk Menerima Inputan.....	112
11.3.2 Membuat Tampilan Inputan di Halaman Web.....	113
11.3.3 Menyimpan Data ke Database	114
BAB XII SEARCH & SORTING	120
12.1 Search	120
12.2 Sorting	121

BAB I

PENGENALAN JAVA, OUTPUT, TIPE DATA & VARIABEL, OPERATOR, DAN INPUT

1.1 Pengenalan Java



Java adalah bahasa pemrograman berorientasi objek yang dirancang oleh **James Gosling** dan **Sun Microsystems** (sekarang Oracle Corporation) pada tahun 1995. Java adalah bahasa pemrograman yang populer dan banyak digunakan untuk mengembangkan aplikasi web, aplikasi desktop, dan aplikasi seluler. Sintak java hampir sama dengan bahasa pemrograman C/C++ karena java terinspirasi dari kedua bahasa tersebut.

Bahasa ini lahir ketika perusahaan Sun Microsystems merasa bahwa bahasa C/C++ tidak lagi mencukupi kebutuhan proyek yang sedang mereka kerjakan. Java pada awalnya bernama **Oak** yang terinspirasi dari pohon oak yang ada di depan kantor James Gosling. Namun karena nama oak sudah terdaftar sebagai merek dagang di perusahaan lain, maka James Gosling menggantinya dengan nama "**Java**" yang terinspirasi dari "**Java Coffee**" atau kopi Jawa. Karena itulah logo bahasa Java berupa gambar secangkir kopi.

1.2 Yang Harus Disiapkan

Berikut ini adalah beberapa hal yang harus disiapkan (*install*) untuk belajar pemrograman java:

1.2.1 JDK (Java Development Kit)

JDK adalah paket perangkat lunak yang berisi semua yang Anda butuhkan untuk mengembangkan dan menjalankan aplikasi Java. JDK mencakup Java Virtual Machine (JVM), Java Compiler, Java Debugger, dan alat-alat lain yang diperlukan untuk mengembangkan aplikasi Java.

Berikut ini adalah langkah-langkah untuk menginstall JDK:

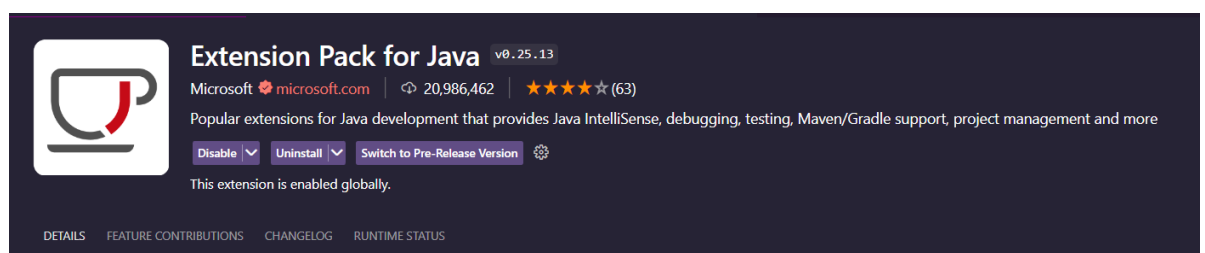
1. Buka website <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>, kemudian pilih versi LTS (Long-Term Support) yang terbaru yaitu Java SE Development Kit 17.0.8, dan pilih Windows x64 Installer.

Linux x64 Debian Package	149.34 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_linux-x64_bin.deb (sha256)
Linux x64 RPM Package	173.66 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_linux-x64_bin.rpm (sha256)
macOS Arm 64 Compressed Archive	168.12 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_macos-aarch64_bin.tar.gz (sha256)
macOS Arm 64 DMG Installer	167.55 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_macos-aarch64_bin.dmg (sha256)
macOS x64 Compressed Archive	170.56 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_macos-x64_bin.tar.gz (sha256)
macOS x64 DMG Installer	169.98 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_macos-x64_bin.dmg (sha256)
Windows x64 Compressed Archive	172.38 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_windows-x64_bin.zip (sha256)
Windows x64 Installer	153.48 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_windows-x64_bin.exe (sha256)
Windows x64 MSI Installer	152.27 MB	https://download.oracle.com/java/17/archive/jdk-17.0.8_windows-x64_bin.msi (sha256)

2. Jika sudah menginstall, maka klik tombol next sampai selesai kemudian close.
3. Untuk mengetahui apakah JDK sudah terinstall atau belum. Maka perlu membuka Command Prompt kemudian ketik perintah **java -version**. Jika JDK sudah terinstall, maka akan muncul versi JDK yang terinstall. Jika JDK belum terinstall, maka akan menunjukkan pesan kesalahan.

1.2.2 IDE

IDE yang akan kita gunakan adalah vscode dan intelliJ. Namun penggunaan intelliJ akan digunakan setelah UTS. Jika sudah menginstall vscode, maka perlu menginstall Extension Pack for Java.



1.3 Cara Menjalankan Java

Untuk memudahkan cara menjalankan java, ikutilah langkah-langkah berikut ini:

1. Buatlah sebuah file dengan nama **Main.java**
2. Kemudian tambahkan kode berikut ini ke dalam file **Main.java**



```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

3. Jika sudah menambahkan kode di atas, maka save file tersebut
4. Jalankan perintah berikut di terminal

```
javac Main.java
```

5. Kode di atas berguna untuk mengkompilasi kode java ke bytecode. Kemudian jalankan perintah di bawah ini untuk menjalankan program java

```
java Main
```

6. Selamat Anda sudah menjalankan program pertama!

1.4 Method Output

Method output di java adalah fungsi yang digunakan untuk mencetak output ke konsol. Method output yang paling umum adalah `System.out.println()`. Method ini memiliki satu argumen yang dapat berupa teks atau objek dan menambahkan baris baru di ujungnya.

Selain `System.out.println()`, ada beberapa method lainnya di java, yaitu:

1. `System.out.print()`, method ini digunakan untuk mencetak teks ke konsol tanpa menambah baris baru. Contoh:

```
System.out.print("Hello, World!");  
System.out.print("Hello Universe!");
```

2. `System.out.printf()`, method ini digunakan untuk mencetak teks ke konsol dengan format tertentu. Contoh:

```
int age = 20;  
System.out.printf("Umur saya %d tahun", age);
```

1.5 Tipe Data

Ada beberapa 2 tipe data yang ada di java, yaitu :

1.5.1 Primitif

Adalah tipe data yang langsung disimpan oleh komputer. Contohnya adalah `byte`, `short`, `int`, `long`, `float`, `double`, `char`, dan `boolean`. Contoh penggunaannya adalah sebagai berikut:

```
char golonganDarah = 'A';
int umur = 20;
double tinggi = 1.74;
boolean menikah = true;
```

1.5.2 Komposit

Adalah tipe data yang merupakan objek, yaitu `String`, `Array`, `Object` dan `Class`.

1. String

Penulisan string pada java harus diawali dengan huruf S besar "`String`", hal ini disebabkan karena `String` merupakan sebuah kelas di java. Berikut adalah contoh penggunaan string.

```
String nama = "Dimas Firmansyah";
```

`String` dapat dimanipulasi dengan berbagai method berikut ini:

- `charAt()`, untuk mengambil karakter tertentu pada string
- `length()`, untuk menghitung panjang string
- `concat()`, untuk menggabungkan 2 string
- `toLowerCase()`, untuk mengubah semua huruf dalam string menjadi huruf kecil
- `toUpperCase()`, untuk mengubah semua huruf dalam string menjadi huruf besar
- `equalsIgnoreCase()`, untuk memeriksa apakah 2 string sama, terlepas dari huruf besar ataupun kecil
- `equals()`, untuk memeriksa apakah 2 string sama

- dll.

2. Array

Adalah struktur data yang menyimpan sekumpulan data dengan tipe yang sama. Array di Java direpresentasikan menggunakan kelas Array. Contoh penggunaannya adalah:

```
int[] arr = { 1, 2, 3, 4, 5 };

int[] number = new int[5];
number[0] = 10;
number[1] = 20;
number[2] = 30;
System.out.println(number[0]);
```

Array dapat dimanipulasi dengan berbagai method seperti:

- **sort()**, untuk mengurutkan elemen array
- **copyOf()**, untuk menyalin sebuah array ke dalam array baru
- **fill()**, untuk mengisi array dengan nilai yang sama
- **binarySearch()**, untuk mencari nilai dalam array menggunakan pencarian binner
- **equals()**, untuk membandingkan dua array

Tipe data array memiliki kelemahan yaitu ukurannya tidak dapat berubah. Untuk mengatasi hal tersebut maka java membuat sebuah struktur data bernama ArrayList yang bisa menampung sekumpulan objek dengan tipe data yang sama, ukurannya lebih fleksibel dan bisa diubah sesuai kebutuhan. Berikut contoh penggunaan ArrayList:

```
// membuat ArrayList
ArrayList<String> listName = new ArrayList<>();

// menambahkan nilai ke elemen ArrayList
listName.add("Dilan");
listName.add("Milea");

// mengambil nilai ArrayList
```

```
System.out.println(listName.get(0));  
System.out.println(listName.get(1));
```

ArrayList juga bisa dimanipulasi dengan berbagai method seperti:

- **add()**, digunakan untuk menambahkan objek
- **remove()**, digunakan untuk menghapus sebuah objek
- **get()**, untuk mengambil sebuah objek berdasarkan nilainya
- **set()**, untuk menetapkan objek baru ke indeks yang sudah ditentukan (update)
- **size()**, untuk mendapatkan ukuran ArrayList
- **clear()**, untuk menghapus semua objek dari ArrayList
- **isEmpty()**, digunakan untuk memeriksa apakah ArrayList kosong atau tidak
- **contains()**, digunakan untuk memeriksa apakah objek tertentu ada di dalam ArrayList atau tidak
- dll.

3. Object & Class

Meteri object dan class akan dibahas dipertemuan OOP.

1.6 Variabel

Di java terdapat beberapa jenis variabel berdasarkan cakupan, karakteristik, dan tujuan penggunaannya. Berikut ini adalah beberapa jenis variabel dan contohnya:

1. Variabel Lokal

Variabel lokal dideklarasikan di dalam blok kode seperti metode, konstruktor, atau blok lainnya. Variabel ini hanya bisa diakses di dalam blok tempat mereka dideklarasikan.

```
public class Main {  
    public static void main(String[] args) {  
        int number = 10; //variabel Lokal  
        System.out.println(number);  
    }  
}
```

```
}  
}
```

2. Variabel Final

Variabel final adalah variabel yang nilainya tidak dapat diubah setelah di inisialisasi. Ini bisa digunakan untuk membuat konstanta.

```
public class Main {  
    final int number = 0; // variabel final  
  
    public static void main(String[] args) {  
        Main example = new Main();  
        System.out.println(example.number);  
    }  
}
```

3. Variabel Kelas & Instance

Materi ini akan dijelaskan di bagian OOP

1.7 Method Input

Di java, terdapat 2 cara untuk menerima inputan dari user yaitu Scanner dan BufferedReader. Scanner adalah class yang lebih baru dan lebih powerful, sedangkan BufferedReader adalah class yang lebih lama dan lebih sederhana.

1.7.1 BufferedReader

BufferedReader hanya bisa digunakan untuk membaca input dari file dan sifatnya yang lebih sederhana dibandingkan dengan Scanner sehingga cocok untuk tugas-tugas sederhana. Berikut contoh penggunaannya.

```
// yang harus di import  
import java.io.BufferedReader;  
import java.io.IOException;  
import java.io.InputStreamReader;  
  
public class Main {  
    public static void main(String[] args) throws IOException  
    {  
        BufferedReader input =  
            new BufferedReader(new InputStreamReader(System.in));  
        String nama;
```

```

        System.out.print("Masukan nama : ");
        nama = input.readLine();
        System.out.println("Selamat datang " + nama);
    }
}

```

Karena hasil dari buffer adalah string, maka wajib melakukan parsing (mengubah tipe data) ke tipe data yang sesuai.

Konversi Ke-	Cara Konvensi
boolean	<code>Boolean.parseBoolean(String)</code>
float	<code>Float.parseFloat(String)</code>
double	<code>Double.parseDouble(String)</code>
byte	<code>Byte.parseByte(String)</code>
short	<code>Short.parseShort(String)</code>
int	<code>Integer.parseInt(String)</code>
long	<code>Long.parseLong(String)</code>

1.7.2 Scanner

Scanner dapat digunakan untuk menerima input dari berbagai sumber termasuk keyboard, file, dan URL. Scanner memiliki banyak method dibanding dengan `BufferedReader` sehingga scanner lebih fleksibel dan dapat digunakan untuk berbagai tugas. Berikut contoh penggunaannya.

```

// yang harus di import
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Nama : ");
        String nama = input.nextLine();
        System.out.println("Selamat Datang " + nama);
    }
}

```

```
}
```

Untuk melakukan inputan ke tipe data selain string, maka kita harus menggunakan beberapa method berikut:

Method	Penjelasan
<code>nextByte()</code>	Membaca tipe data byte
<code>nextShort()</code>	Membaca tipe data short
<code>nextInt()</code>	Membaca tipe data integer
<code>nextLong()</code>	Membaca tipe data long
<code>nextFloat()</code>	Membaca tipe data float
<code>nextDouble()</code>	Membaca tipe data double
<code>next()</code>	Membaca sebaris string
<code>nextLine()</code>	Membaca sebaris string dengan baris baru di akhir

1.8 Operator

Semua operator di java sama dengan operator yang ada di bahasa C, sehingga kalian bisa mencari tahu atau mengingat kembali materi yang telah diajarkan oleh para instruktur.

BAB II

MEMAHAMI STRUKTUR & ATURAN PENULISAN SINTAKS, PERCABANGAN, PERULANGAN, DAN METHOD

2.1 Struktur Dasar Program Java

Struktur program java secara umum dibagi menjadi 4 bagian, yaitu deklarasi package, impor library, bagian class, dan method main.

```
package com.pub_course; // <- 1. Deklarasi Package

import java.io.*; // <- 2. Impor Library

public class Program { // <- 3. Bagian Class
    public static void main(String[] args) { // <- 4. Method Main
        System.out.println("Hello World!");
    }
}
```

2.1.1 Deklarasi Package

Package merupakan sebuah folder yang berisi sekumpulan program Java. Deklarasi **package** biasanya dilakukan saat membuat program atau aplikasi besar. Contoh deklarasi package:

```
package com.pub_course;
```

Biasanya nama **package** mengikuti nama domain dari sebuah vendor yang mengeluarkan program tersebut. Pada contoh di atas, **com.pub_course** adalah nama domain dari pub_course.com.

Aturan penulisan package adalah nama domain nya dibalik. Misalnya youtube.com dibalik menjadi **com.youtube**.

2.1.2 Impor Library

Pada bagian ini, kita melakukan impor library yang dibutuhkan pada program. Library merupakan sekumpulan class dan fungsi yang bisa kita gunakan dalam membuat program. Contoh impor library:

```
import java.util.Scanner;
```

Pada contoh tersebut, kita mengimpor class `Scanner` dari package `java.util`.

2.1.3 Bagian Class

Java merupakan bahasa pemrograman yang menggunakan paradigma OOP (*Object Oriented Programming*). Setiap program harus dibungkus di dalam class agar nanti bisa dibuat menjadi objek. Berikut adalah contohnya:

```
public class Program {  
    Statement.....  
}
```

Di dalam tanda kurung kurawal `{}` kita bisa mengisinya dengan berbagai method dan juga variabel.

2.1.4 Method Main

Method `main()` merupakan blok program yang akan dieksekusi pertama kali. Ini adalah entri point dari program. Method `main()` wajib kita buat. Kalau tidak, maka programnya tidak akan bisa dieksekusi.

```
public static void main(String[] args) {  
    Statement....  
}
```

2.1.5 Gaya Penulisan Case

Gaya penulisan case (case style) yang digunakan oleh Java adalah: camelCase, PascalCase, dan UPPER_CASE.

1. camelCase, digunakan pada nama variabel, nama objek, dan nama method

```
// contoh penulisan method  
void oddEvenMethod() {  
  
    // contoh penulisan variabel  
    int bilanganGanjil;  
    int bilanganGenap;  
  
    // contoh penulisan objek  
    Program programGanjilGenap = new Program();
```

```
}
```

2. PascalCase, digunakan pada penulisan nama class

```
class MahasiwaPub {  
    ...  
}
```

3. UPPER_CASE, digunakan untuk membuat variabel konstanta atau **final**

```
final String FIRST_NAME = "Wahyu";
```

2.2 Percabangan

2.2.1 Percabangan IF ELSE

Penggunaan **if else** sama seperti bahasa C atau bahasa pemrograman lain. Berikut contoh sederhananya:

```
public static void main(String[] args) {  
    int a = 10;  
    if (a % 2 == 0) {  
        System.out.println("Ini bilangan genap");  
    }  
    else {  
        System.out.println("Ini bilangan ganjil");  
    }  
}
```

2.2.2 Percabangan Ternary Operator

Percabangan ini digunakan untuk mempersingkat baris kondisi pada sebuah program. Berikut contohnya:

```
public static void main(String[] args) {  
    boolean suka = true;  
    String jawaban = suka ? "iya" : "tidak";  
    System.out.println(jawaban);  
}
```

2.2.3 Percabangan Switch Case

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);
```

```
String lampu;
System.out.print("Sekarang lampu apa : ");
lampu = input.nextLine();
switch (lampu) {
    case "merah":
        System.out.println("Berhenti!");
        break;
    case "kuning":
        System.out.println("Bersiap!");
        break;
    case "hijau":
        System.out.println("Jalan!");
        break;
    default:
        System.out.println("Inputan salah");
}
}
```

2.3 Perulangan / Iterasi

Iterasi/perulangan dibagi menjadi dua, yaitu counted loop dan uncounted loop

2.3.1 Counted Loop

Arti dari *counted loop* adalah perulangan yang jumlah pengulangannya terhitung atau sudah di definisikan berapa kali program dapat berulang.

1. For

Berikut adalah contoh program perulangan menggunakan **for**

```
public static void main(String[] args) {
    for (int i = 0; i < 5; i++) {
        System.out.println(i);
    }
}
```

2. For each

```
public static void main(String[] args) {
    String[] name = {"Dimas", "Wahyu", "Anggi", "Rey"};
    for (String n : name) {
        System.out.println(n);
    }
}
```

2.3.2 Uncounted Loop

Arti dari *uncounted loop* adalah perulangan yang jumlah pengulangannya tidak terhitung atau belum di definisikan kapan perulangan itu berhenti.

1. While

While bisa kita artikan selama. Cara kerja perulangan ini seperti percabangan, ia akan melakukan perulangan selama kondisinya bernilai **true**. Contohnya adalah sebagai berikut:

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean ulang = true;
    int hitung = 0;
    String jawab;
    while (ulang) {
        System.out.print("Apakah ingin mengulang (y/t) : ");
        jawab = input.nextLine();
        if (jawab.equalsIgnoreCase("t")) {
            ulang = false;
        }
        hitung++;
    }
    System.out.println("Anda telah melakukan perulangan sebanyak "
        + hitung + " kali");
}
```

2. Do while

Cara kerja perulangan Do/While sebenarnya sama seperti perulangan While. Bedanya, Do/While melakukan satu kali perulangan dulu. Kemudian mengecek kondisinya. Berikut contoh programnya:

```
public static void main(String[] args) {
    int i = 0;
    do {
        i++;
        System.out.println("Perulangan ke-" + i);
    } while (i < 10);
}
```

2.4 Method

Apa perbedaan procedure, function, dan method di bahasa pemrograman java?

- Procedure adalah sebutan untuk fungsi yang tidak mengembalikan nilai. Fungsi ini biasanya ditandai dengan kata kunci **void**.
- Function adalah sebutan untuk fungsi yang mengembalikan nilai.
- Method adalah fungsi yang berada dalam suatu class. Sebutan ini biasanya digunakan pada OOP.

2.4.1 Cara Membuat Method

Untuk membuat suatu method, kita harus menuliskannya di dalam sebuah class. Struktur dasar sebuah method adalah sebagai berikut:

```
static TypeDataKembalian namaFungsi(){
    // statemen atau kode fungsi
}
```

Penjelasan:

- Kata kunci **static**, artinya kita membuat fungsi yang dapat dipanggil tanpa harus membuat instansiasi objek. *Penjelasan di akhir modul*
- **TypeDataKembalian** adalah tipe data dari nilai yang dikembalikan setelah fungsi dieksekusi.
- **namaFungsi()** adalah nama fungsinya. Biasanya ditulis dengan huruf kecil di awalnya. Lalu, kalau terdapat lebih dari satu suku kata, huruf awal di kata kedua ditulis kapital (camelCase).

Berikut adalah contoh programnya:

```
static void sapaan(){
    System.out.println("Hello World");
}
```

2.4.2 Cara Memanggil Method

Setelah kita membuat fungsi, selanjutnya kita akan mengeksekusi fungsinya. Fungsi dapat dipanggil dari fungsi **main** atau dari fungsi yang lainnya. Contoh pemanggilan fungsi dalam fungsi **main**:

```
static void sapaan() {
    System.out.println("Hello World");
}

public static void main(String[] args) {
    sapaan();
}
```

```
}
```

2.4.3 Menggunakan Parameter

Parameter adalah variabel yang menampung nilai untuk diproses di dalam fungsi. Parameter berperan sebagai input untuk fungsi. Berikut adalah contoh programnya:

```
static void jumlah(int x, int y) {  
    System.out.println(x + x);  
}  
  
public static void main(String[] args) {  
    jumlah(5, 5);  
}
```

2.4.4 Method Yang Mengembalikan Nilai

Setelah fungsi memproses data yang diinputkan melalui parameter, selanjutnya fungsi harus mengembalikan nilai agar dapat diolah pada proses berikutnya. Pengembalian nilai pada fungsi menggunakan kata kunci **return**. Berikut adalah contoh programnya:

```
static int luasPersegi(int x) {  
    return x * x;  
}  
  
public static void main(String[] args) {  
    System.out.println("Luas persegi adalah : " + luasPersegi(5));  
}
```

2.4.5 Static & Non-static

Pada contoh-contoh diatas, kita menggunakan kata kunci **static** sebelum membuat fungsi. Kata kunci **static** akan membuat fungsi dapat dieksekusi langsung, tanpa harus membuat instansiasi objek dari class.

```
public class Program {  
  
    // static function  
    static void minum() {  
        System.out.println("Saya sedang minum kopi");  
    }  
  
    // non-static function  
    void makan(){  
        System.out.println("Hi!");  
    }  
}
```

```
        System.out.println("Saya sedang makan bubur");
    }

    public static void main(String[] args) {
        // menggunakan static
        minum();

        // tidak menggunakan static
        Program saya = new Program();
        saya.makan();
    }
}
```

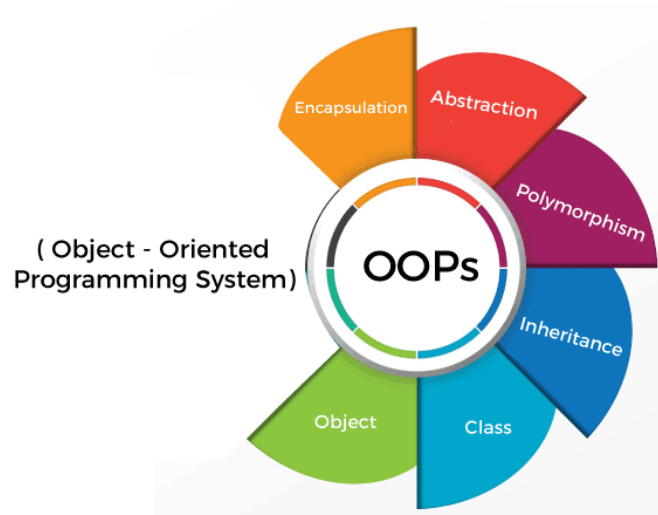
BAB III

PENGANTAR OOP, CLASS, OBJECT, & ACCES MODIFIER

3.1 Pengenalan OOP

OOP (Object Oriented Programming) atau dalam bahasa Indonesia dikenal dengan Pemrograman Berorientasikan Objek (PBO) merupakan sebuah paradigma atau teknik pemrograman yang berorientasi Objek.

Pada OOP, Fungsi dan variabel **dibungkus** dalam sebuah **objek** atau class yang dapat saling berinteraksi, sehingga membentuk sebuah program. Dengan demikian, tidak akan ada lagi kode yang berantakan.



3.2 Class

Dalam bahasa pemrograman Java, class adalah cetakan atau blueprint dari sebuah objek. Class berisi definisi variabel dan fungsi yang menggambarkan sebuah objek. Variabel di dalam class disebut sebagai **attribute** atau **properti**, sedangkan fungsi di dalam class disebut sebagai **method**.

```
class Car {  
  
    // attribute  
    String nameCar;  
    String brand;  
    int price;  
  
    // method  
    void drive() {  
        System.out.println("Mobil melaju...");  
    }  
}
```

```

    }

    void stop() {
        System.out.println("Mobil berhenti.");
    }
}

```

Di dalam bahasa pemrograman java, terdapat beberapa jenis class yang bisa Anda pelajari sendiri. Diantaranya adalah sebagai berikut::

1. Regular class
2. Abstract class
3. Interface
4. Enum class
5. Inner class
6. Lambda class
7. Anonymous class
8. Static class

3.3 Object

Dalam paradigma OOP, objek merupakan instance atau wujud nyata dari suatu kelas. Salah satu kegunaan objek adalah untuk mengakses berbagai fungsi dan metode pada kelas.

Karena objek merupakan instansiasi dari kelas, untuk menciptakan suatu objek kita perlu mendeklarasikan tiap objek yang dibuat dengan mengikutsertakan kelasnya. Pada Java untuk membuat sebuah objek terdapat 3 tahap :

- Deklarasi : mendeklarasikan nama sebuah objek
- Instansiasi : memerlukan sebuah perintah atau keyword **new** untuk menciptakan objek
- Inisialisasi : inisialisasi dari sebuah objek setelah perintah **new**

Agar lebih memudahkan lagi kita pakai contoh class bernama Car. Sekarang kita buat sebuah objek dari class Car tersebut, caranya adalah sebagai berikut:

```

public static void main(String[] args) {
    // deklarasi objek
    Car mobil;
}

```

```
// instansiasi objek
new Car();

// inisialisasi objek
mobil = new Car();

}
```

Penjelasan dari kode di atas adalah :

- Kita membuat sebuah objek di method main (pembuatan objek bisa di method apa saja)
- Keyword **Car** artinya kita ingin mengakses sebuah class yang bernama **Car**
- Variabel **mobil** adalah nama dari objek yang sedang kita buat
- Keyword **new** berguna untuk menginstansiasi sebuah objek atau membuat objek baru

Dengan kode tersebut kita bisa dengan leluasa menggunakan properti maupun method yang berada di class Car. Misalnya kita ingin mengisi properti **nameCar** dengan **"Lamborghini"**. Maka kodenya adalah berikut ini.

```
public static void main(String[] args) {
    // instansiasi objek
    Car mobil = new Car();

    // mengisi value sebuah properti
    mobil.nameCar = "Lamborghini";

    // menampilkan isi sebuah properti
    System.out.println(mobil.nameCar);
}
```

3.4 Constructor

Constructor adalah method yang secara default sudah terbentuk ketika kelas dibuat. Ketika suatu kelas dibuat (instansiasi) maka konstruktor akan terpanggil juga. Constructor harus memiliki nama yang sama dengan nama kelasnya.

Intinya, kegunaan dari constructor ini adalah method khusus yang akan dipanggil pertama kali saat objek di buat. Misalnya kita gunakan class **Car**, kemudian kita buat sebuah constructor dengan kode berikut ini:

```
class Car {
```

```
// membuat constructor
Car(){
    System.out.println("Ini adalah constructor");
}

// statement lainnya...
}
```

Kemudian jika kita jalankan lagi programnya maka output **"Ini adalah constructor"** akan muncul di paling atas.

Karena constructor ini adalah sebuah method, maka kita bisa menambahkan parameter pada saat membuatnya. Perhatikan contoh berikut ini:

```
Car(String text){
    System.out.println(text + " buatan Indonesia");
}
```

Kemudian pada saat membuat objek (instansiasi) kita harus menambahkan sebuah argumen, berikut contoh pada saat pembuatan objek.

```
// instansiasi objek
Car mobil = new Car("Lamborghini");
```

Hal-hal yang harus diperhatikan dalam membuat constructor adalah :

- Nama constructor harus sama dengan nama class
- Tidak perlu memakai tipe data karena merupakan method khusus
- Constructor tidak memiliki return value

3.5 Access Modifier

Class dalam program Java dapat saling berhubungan dengan cara memberikan akses terhadap member mereka. Semua yang ada di dalam sebuah class (atribut/properti dan method) disebut **member**. Biasanya akan ada tingkatan akses yang disebut **modifier**.

Access modifier di dalam Object Oriented Programming (OOP) akan menentukan apakah kelas lain dapat menggunakan properti atau meng-invoke methods dari suatu kelas. Ada beberapa macam access modifier yang dapat digunakan yaitu **private**, **default**, **protected**, dan **public**.

Modifier	Class yang sama	Package yang sama	Subclass	Global
----------	-----------------	-------------------	----------	--------

private	✓			
default	✓	✓		
protected	✓	✓	✓	
public	✓	✓	✓	✓

3.5.1 Private

Access modifier **private** akan membatasi akses hanya di dalam class. **Private** biasanya digunakan sebagai modifier dari member dan metode suatu class. Berikut adalah contoh penggunaan modifier **private** adalah sebagai berikut.

```
package com.pasim.application;

public class Car {
    private String nameCar;
    String brand;
    int price;
    private void drive() {
        System.out.println("Mobil melaju...");
    }
    void stop() {
        System.out.println("Mobil berhenti.");
    }
}

public class CarApplication {
    public static void main(String[] args) {
        Car mobil = new Car();
        System.out.println(mobil.drive()); // kode berikut akan error
    }
}
```

Dalam kode di atas, terdapat variabel **nameCar** yang memiliki modifier **private** dan method **drive()**. Jika kita mengakses kedua member tersebut di class lain seperti class **CarApplication** pada contoh di atas, maka hasilnya akan error.

3.5.2 Default

Default modifier berarti penulisan kodenya tanpa atribut modifier. Ini berlaku untuk semua kelas, member, atau fungsi yang kita tuliskan tanpa access modifier. Modifier **default** bisa diakses selama masih dalam satu package.

Dari contoh di atas member yang memiliki modifier adalah:

- Atribut **brand**
- Atribut **price**
- Method **stop()**

3.5.3 Protected

Access modifier **protected** bisa diakses selama masih dalam satu package. **Protected** memiliki sedikit perbedaan dengan **default** modifier. Perbedaannya adalah **protected** bisa diakses dari luar package. Akan tetapi, satu-satunya cara untuk akses dari luar package adalah kelas yang hendak mengakses, merupakan kelas turunannya.

Misalnya kita ubah modifier atribut brand dengan **protected**

```
protected String brand;
```

dengan mengubah modifier menjadi **protected**, maka atribut tersebut bisa di akses oleh class **CarApplication**.

3.5.4 Public

Access modifier **public** bisa kita sebut sebagai modifier global. Artinya bisa diakses dari manapun bahkan package yang berbeda. Seperti pada contoh kode sebelumnya, class **Car** ditambahkan modifier public. Karena modifiernya **public** maka bisa diakses dari package lainnya.

BAB IV

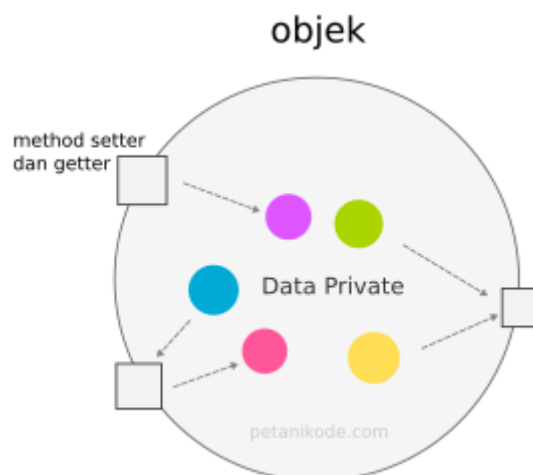
ENKAPSULASI, KEYWOD THIS, COLLECTION TYPE, DAN MEMBUAT CRUD MENGGUNAKAN COLLECTION TYPE

4.1 Enkapsulasi

Dalam OOP, enkapsulasi adalah cara menyembunyikan detail implementasi suatu kelas dari akses luar dan hanya mengekspos interface **public** yang dapat digunakan untuk berinteraksi dengan kelas tersebut.

Sederhananya, enkapsulasi (pembungkusan) adalah proses dimana data dibungkus dengan modifier **private** agar tidak bisa diakses secara langsung dari luar class.

Lalu bagaimana caranya agar data tersebut diakses dan bisa berguna? Caranya adalah dengan menggunakan method setter dan getter.



Mengapa harus dibuat seperti ini? Ada beberapa alasan kita harus menggunakan proses enkapsulasi:

- Untuk meningkatkan keamanan data
- Agar lebih mudah dalam mengontrol atribut atau method
- Class bisa dibuat menjadi read-only dan write-only
- Fleksibel, dalam artian developer dapat mengganti sebagian kode tanpa harus takut berdampak pada kode lain

4.1.1 Cara Membuat Method Setter & Getter

Untuk membuat method setter & getter sama saja seperti kita membuat method biasa. Misalnya kita memiliki sebuah data login user yang ingin kita rahasiakan.

Dalam contoh berikut terdapat atribut `username` dan `password` yang ingin kita rahasiakan datanya. Kemudian kita buat class dengan nama `User`, serta menambahkan kode berikut:

```
class User {  
    // Data yang ingin dirahasiakan  
    private String username;  
    private String password;  
  
    // method getter username  
    public String getUsername() {  
        return username;  
    }  
  
    // method setter username  
    public void setUsername(String username) {  
        this.username = username;  
    }  
  
    // method getter password  
    public String getPassword() {  
        return password;  
    }  
  
    // method setter password  
    public void setPassword(String password) {  
        this.password = password;  
    }  
}
```

Method setter dan getter harus diberikan modifier `public`, karena method ini akan diakses dari luar class. Perbedaan method setter dengan getter terletak pada nilai kembalian, parameter, dan isi method-nya.

Method setter tidak memiliki nilai kembalian `void` (kosong). Karena tugasnya hanya untuk mengisi data ke dalam atribut. Sedangkan method getter memiliki nilai kembalian sesuai dengan tipe data yang akan diambil.

4.1.2 Cara Menggunakan Method Setter & Getter

Cara menggunakannya pun cukup mudah, misalnya kita akses di method main. Kemudian kita ingin membuat program login seperti biasa. Maka berikut contoh programnya:

```
public static void main(String[] args) {
    User user = new User();
    Scanner input = new Scanner(System.in);

    System.out.print("Masukan username : ");
    String username = input.nextLine();
    user.setUsername(username);

    System.out.print("Masukan password : ");
    String pass = input.nextLine();
    user.setPassword(pass);

    if (user.getUsername().equalsIgnoreCase("admin") &&
        user.getPassword().equalsIgnoreCase("123")) {
        System.out.println("Login berhasil");
    } else {
        System.out.println("Login gagal");
    }
    input.close();
}
```

Program di atas akan menampilkan output “**Login berhasil**” jika kondisi **true**, dan “**Login gagal**” jika kondisi **false**.

4.2 This

Pada contoh sebelumnya, kita telah menerapkan proses enkapsulasi dengan method setter dan getter. Di dalam method tersebut kita menggunakan keyword **this** untuk mengisi variabel.

```
public void setUsername(String username) {
    this.username = username;
}
```

Jika keyword **this** di hapus, maka komputer akan bingung memilih variabel **username** yang dimaksud. Apakah variabel yang di class (**private String username**) ataupun variabel **username** yang ada di parameter **setUsername**.

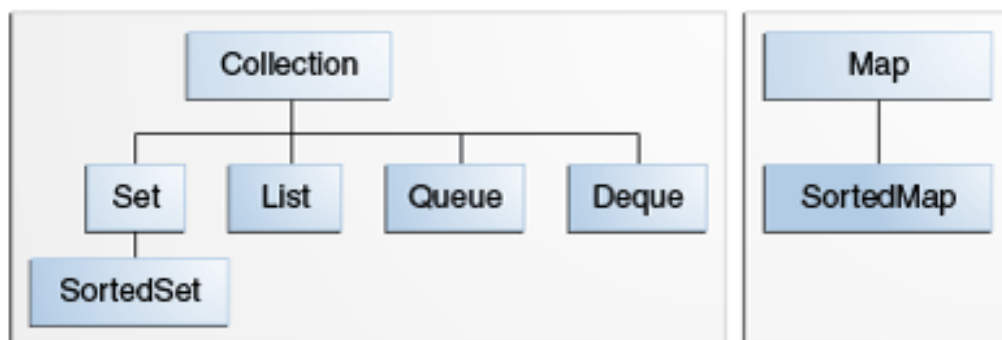
Jadi, keyword **this** digunakan sebagai referensi dari class itu sendiri.

4.3 Collection Type

Collection adalah kumpulan interface yang digunakan sebagai wadah untuk mengumpulkan beberapa elemen menjadi satu kesatuan. Apa itu interface? Interface sangat mirip dengan class pada umumnya, namun tidak memiliki atribut dan memiliki method yang di deklarasikan tanpa isi.

Jika definisi di atas terlalu sulit dimengerti, maka pengertian collection type secara sederhana adalah struktur data yang lebih kompleks dan lebih canggih dibandingkan dengan array.

Core collection interface dapat dirangkum pada gambar di bawah ini.



Dalam pelatihan ini, yang akan kita bahas adalah **ArrayList** yang salah satu dari interface **List**, dan **HashMap** sebagai salah satu implementasi dari interface **Map**.

4.3.1 ArrayList

List adalah suatu Collection di mana data yang masuk akan disimpan secara teratur. List mempunyai indeks berdasarkan urutan objek yang dimasukkan ke dalam list.

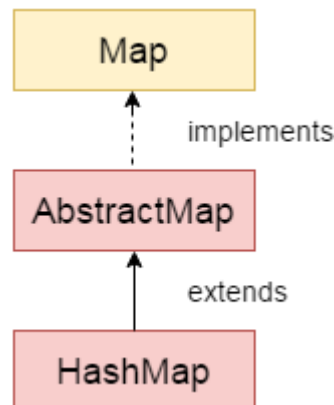
ArrayList merupakan salah satu contoh interface dari list yang mirip dengan Array. Perbedaannya hanya di ukurannya saja, ArrayList memiliki ukuran yang tidak tetap atau fleksibel sedangkan Array biasa ukurannya tidak bisa diubah.

Berikut adalah contoh penerapannya:

```
public static void main(String[] args) {
    ArrayList<String> list = new ArrayList<>();
    list.add("Freya");
    list.add("Zee");
    list.add("Wahyu");
    list.remove(2);
    list.set(1, "Shani");
    System.out.println(list);
}
```

4.3.2 HashMap

Class HashMap merupakan class turunan dari class AbstractMap dan implementasi dari interface Map.



HashMap adalah sebuah class yang berisi sekumpulan pasangan nilai (value) dan kunci (key). Nilai bisa dalam bentuk string, integer, boolean, float, double, dan objek. Sedangkan untuk key biasanya dalam bentuk string dan integer.

Coba perhatikan gambar berikut ini.

Key	Value
"name"	"Petani Kode"
"url"	"https://www.petanikode.com/"
"email"	"info@petanikode.com"
"isActive"	true

Gambar di atas terdiri dari pasangan key dan value, seperti inilah isi dari class atau objek HashMap.

Berikut adalah contoh program menggunakan HashMap.

```
public static void main(String[] args) {
    HashMap<String, String> days = new HashMap<String, String>();
}
```

```
// mengisi hashmap
days.put("senin", "Menghapal vocab");
days.put("selasa", "Hapalan vocab");
days.put("rabu", "Ada tes vocab");
days.put("kamis", "Pengumuman nilai vocab");
days.put("jumat", "Kecewa dengan pengumuman vocab");
days.put("sabtu", "Dikasih vocab baru");
days.put("minggu", "Aku suka vocab :)");

// cetak hashmap
for (Map.Entry x : days.entrySet()) {
    System.out.println(x.getKey() + " : " + x.getValue());
}
}
```

Kode di atas, menjelaskan jika kita ingin membuat sebuah HashMap yang berisi quotes of the days in PUB. Kemudian kita mencetaknya dengan menggunakan perulangan for each.

Bagaimana jika kita ingin mengganti value di HashMap, caranya adalah menggunakan method **replace()**. Berikut contohnya:

```
// mengganti value jumat
days.replace("jumat", "Aku cinta PUB");
```

Bagaimana jika kita ingin menghapus value dari HashMap, caranya adalah menggunakan method **remove()**. Berikut cara penggunaanya:

```
// menghapus value minggu
days.remove("minggu");
```

Ada beberapa method lagi yang bisa kalian pakai dan pelajari, diantaranya adalah :

- **clear()**, untuk membersihkan isi HashMap
- **isEmpty()**, untuk mengecek apakah HashMap dalam keadaan kosong atau tidak
- **size()**, untuk mengambil ukuran HashMap
- **values()**, untuk mengambil semua value yang ada di HashMap
- **keySet()**, untuk mengambil semua key yang ada di HashMap

- **clone()**, untuk menggandakan objek HashMap

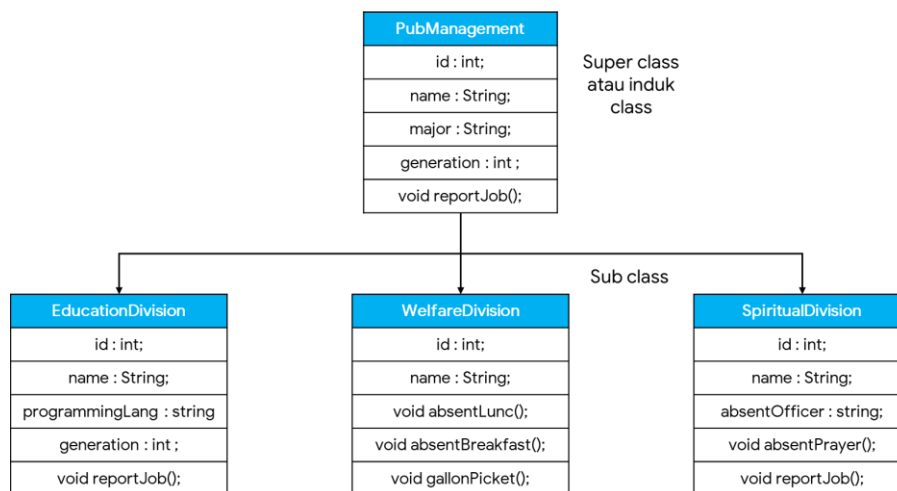
BAB V

INHERITANCE & METHOD OVERRIDING

5.1 Inheritance

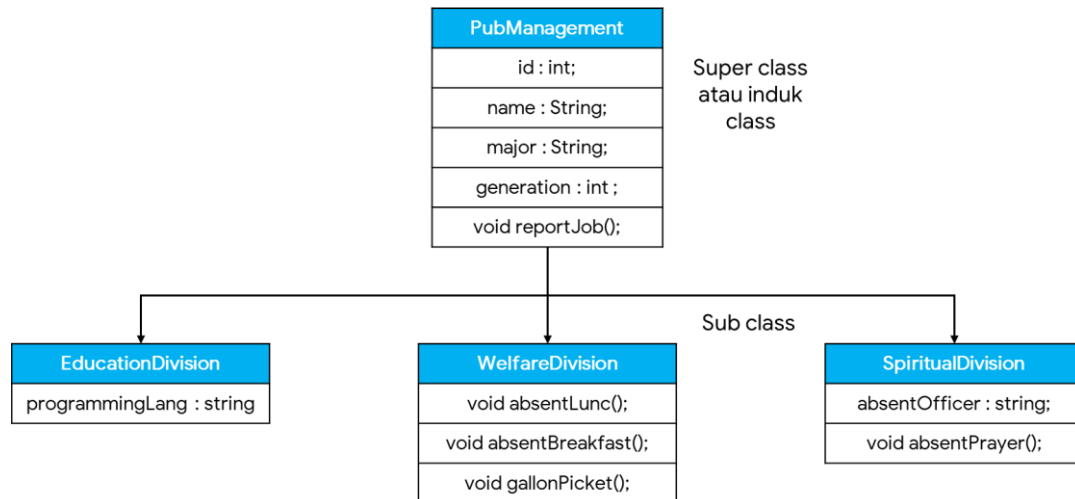
Pada pelatihan sebelumnya, kita sudah mempelajari bahwa sebuah class atau objek bisa saling berhubungan dengan class yang lain. Salah satu bentuk hubungannya adalah inheritance. Hubungan ini seperti hubungan keluarga antara orang tua dan anak.

Jadi sederhananya, inheritance (Pewarisan) adalah sebuah konsep pewarisan sifat berupa variabel (property) dan fungsi (method) yang dimiliki oleh class untuk diwariskan kepada class-class yang lain.



Pada gambar entitas di atas, terdapat beberapa properti dan method yang juga dimiliki oleh entitas/class lain. Ini merupakan sebuah pemborosan kode, karena dengan konsep inheritance, kita bisa menyederhanakan entitas-entitas tersebut menjadi lebih sederhana.

Berikut hasilnya jika kita menggunakan konsep inheritance.



Sebelum kita mempelajari inheritance, kita harus paham dulu beberapa kata berikut ini:

- Superclass, adalah class induk atau bisa disebut juga parent
- Subclass, adalah class turunan atau class anak (child)

5.1.1 Cara Membuat Inheritance

Untuk lebih memahami konsep inheritance, mari kita buat program sederhana. Yaitu membuat program Kepengurusan PUB.

Pertama, yang harus kita buat adalah membuat file dengan nama **PubManagement.java**, **EducationalDivision.java**, **WelfareDivision.java**, dan **SpiritualDivision.java** di package **model**.

Kemudian kita akan menjadikan class **PubManagement.java** sebagai class parent/ super class dengan menambahkan kode berikut.

```

public class PubManagement {
    public int id;
    public String name;
    public String major;
    public int generation;

    public void reportJob() {
        System.out.println("Ini adalah laporan kepengurusan");
    }
}
  
```

Kemudian sisa class yang kita buat tadi, akan dijadikan sebagai child/subclass. Sekarang isi class **EducationalDivision.java** dengan kode berikut.

```
public class EducationDivision extends PubManagement{
    public String programmingLang;
}
```

Selanjutnya, isi class **WalfareDivision.java** dengan kode berikut.

```
public class WalfareDivision extends PubManagement{
    public void absentLunch() {
        System.out.println("Ini absen makan siang");
    }

    public void absentBreackfast() {
        System.out.println("Ini absen sarapan");
    }

    public void gallonPicket() {
        System.out.println("Ini absen piket galon");
    }
}
```

Setelah itu, isi class **SpiritualDivision.java** dengan kode berikut.

```
public class SpiritualDivision extends PubManagement{
    public String absentOfficer;

    public void absentPrayer() {
        System.out.println("Ini absen sholat");
    }
}
```

Sekarang kita sudah membuat semua entitas, menjadi sebuah class. Sekarang kita tambahkan kode berikut di class utama atau **Main.java**

```
import model.EducationDivision;
import model.PubManagement;
import model.SpiritualDivision;
import model.WalfareDivision;

public class App {
    public static void main(String[] args) throws Exception {
        // instansiasi objek seperti biasa
        PubManagement pengurus = new PubManagement();
        EducationDivision divdik = new EducationDivision();
        SpiritualDivision rohani = new SpiritualDivision();
        WalfareDivision sejahtera = new WalfareDivision();
    }
}
```

```

// memanggil properti atau method di class super
pengurus.reportJob();

// mengolah properti atau method di class subclass
divdik.programmingLang = "Java";
rohani.absentOfficer = "Wabred";
sejahtera.absentBreakfast();

// mengakses properti atau method yang ada
// di class super, tapi menggunakan subclass
rohani.generation = 20;
System.out.println(rohani.generation);
}
}

```

Jika kita jalankan kode berikut, maka akan menghasilkan output:

```

Ini adalah laporan kepengurusan
Ini absen sarapan
20

```

Kenapa properti **generation** bisa diakses menggunakan objek **rohani**? Bukannya harus menggunakan objek **pengurus**? Marilah kita pahami beberapa materi berikut.

5.1.2 Keyword extends

Dalam kode sebelumnya, kita menyertakan sebuah keyword yang bernama extends. Extends adalah keyword yang digunakan untuk mendefinisikan hubungan inheritance antara dua kelas.

```

public class EducationDivision extends PubManagement{
    public String programmingLang;
}

```

Dalam contoh di atas, class **EducationDivision** merupakan turunan/subclass dari **PubManagement**, dan **PubManagement** merupakan induk/superclass dari **EducationDivision**.

Artinya class **EducationDivision** dapat mengakses properti atau pun method yang berada di class **PubManagement**.

```

divdik.major = "Akuntansi";

```

Contoh di atas berarti bahwa objek **divdik** mampu mengakses properti **major** yang ada di class **PubManagement**.

Apakah superclass bisa mengakses properti atau method yang ada di subclass? Jawabannya adalah tidak. Karena yang bisa mengakses properti atau method ke class lain hanyalah subclass atau turunan.

```
pengurus.absentBreackfast();
```

Kode di atas akan menghasilkan output error. Karena objek **pengurus** tidak bisa mengakses properti atau method class turunannya.

5.2 Method Overriding

Overriding adalah sebuah fitur OOP yang memungkinkan class turunan mendefinisikan kembali method yang diwarisi oleh class induk.

Untuk melakukan overriding, class turunan harus memiliki method dengan nama, parameter, dan tipe pengembalian yang sama dengan method yang ada di super class.

Misalnya kita ingin membuat ulang method **reportJob()** yang ada di **PubManagement** ke class **SpiritualDivision**. Caranya adalah sebagai berikut:

Buat kode berikut ini di dalam class SpiritualDivision

```
@Override
public void reportJob(){
    System.out.println("Alhamdulillah tidak ada yang botak");
}
```

Dalam kode di atas, kita menambahkan anotasi **@Override** untuk manandai bahwa method **reportJob()** merupakan method yang di deklarasi ulang dari class induk.

BAB VI

ABSTRACT CLASS, INTERFACE & OVERLOADING

6.1 Abstract Class

Abstract Class merupakan class yang masih dalam bentuk abstrak. Seringkali juga disebut sebagai jenis kelas yang tidak dapat di instansiasi (tidak dapat membuat objek langsung dari kelas tersebut) dan seringkali berfungsi sebagai kerangka atau cetakan (blueprint) untuk kelas-kelas anak (subclass) yang akan mengimplementasikan metode-metode abstrak yang di deklarasikan dalam class abstract tersebut. Adapun beberapa karakteristik dari class abstract adalah sebagai berikut:

1. Tidak dapat di instansiasi

Class abstract tidak dapat secara langsung dibuat sebuah objek dengan menggunakan keyword new. Contohnya `KelasAbstrak objek = new KelasAbstrak();`

2. Harus memiliki setidaknya satu method abstrak

Class abstract dapat memiliki metode-metode yang di deklarasikan sebagai "abstract." Metode abstrak adalah metode yang tidak memiliki implementasi (tubuh) di dalam class abstract. Sebaliknya, metode abstrak ini harus diimplementasikan (disediakan tubuhnya) oleh kelas anak yang mewarisi class abstract tersebut

3. Dapat memiliki method biasa

Selain metode abstrak, class abstract juga dapat memiliki metode biasa (konkrit) dengan implementasi yang lengkap

4. Dapat memiliki atribut

Class abstract juga dapat memiliki atribut seperti kelas biasa.

5. Keyword abstract

Untuk mendeklarasikan kelas sebagai abstrak, harus terdapat kata kunci `abstract` sebelum kata kunci `class` saat mendefinisikan kelas tersebut

Berikut adalah contoh sederhana dari abstract class

```
public abstract class Character {
    // Metode abstrak yang akan diimplementasikan oleh kelas anak
    public abstract void attack();
    // Metode biasa
    public void survive() {
        System.out.println("Method bertahan.");
    }
}
```

Dalam contoh di atas, **Character** adalah class abstract dengan satu metode abstrak **attack()**, yang harus di implementasikan oleh kelas anak yang mewarisi kelas **Character**.

6.1.1 Cara Membuat Abstract Class

Untuk lebih memahami konsep dari class abstract, mari kita buat sebuah program sederhana. Yaitu membuat sebuah program dengan konsep permainan menyerang antar karakter.

Pertama, buat sebuah program java dengan nama **CharacterGame**. Setelah itu buat beberapa file dengan nama **Character.java**, **Hero.java** dan **Enemy.java** dalam package **src**.

Selanjutnya, jadikan class **Character.java** sebagai class abstract dengan menambahkan kode sebagai berikut.

```
public abstract class Character {
    // Atribut atau properti
    private String name;
    // Constructor
    public Character(String name) {
        this.name = name;
    }
    // Method abstrak
    public abstract void attack();
    // Method biasa
    public void survive(){
        System.out.println("Karakter bertahan");
    }
    // Method Getter
    public String getName() {
        return name;
    }
    // Method Setter
    public void setName(String name) {
        this.name = name;
    }
}
```

Kemudian untuk class **Hero.java** dan **Enemy.java** buat jadi subclass dengan meng-extends class abstract **Character.java**. Untuk kode program nya seperti dibawah ini:

Hero.java :

```
public class Hero extends Character{
    public Hero(String name) {
        super(name);
    }

    @Override
    public void attack() {
        System.out.println("Pahlawan menyerang musuh !!!");
    }
}
```

Enemy.java :

```
public class Enemy extends Character{
    public Enemy(String name) {
        super(name);
    }
    @Override
    public void attack() {
        System.out.println("Musuh menyerang pahlawan !!!");
    }
}
```

Sekarang kita sudah membuat semua entitas, menjadi sebuah class. Sekarang ki

Selanjutnya untuk class utama atau **App.java** , kode program nya sebagai berikut :

```
public class App {
    public static void main(String[] args) throws Exception {
        // Instansiasi objek
        Hero hero = new Hero("Anas");
        Enemy enemy = new Enemy("Jokowow");
        // Memanggil method attack() dari kedua kubu
        hero.attack();
        enemy.attack();
    }
}
```

6.2 Interface

Untuk lebih memahami konsep dari interface, mari kita buat sebuah program sederhana. Yaitu membuat sebuah program hewan.

Pertama, buat sebuah program java dengan nama Animal. Setelah itu buat beberapa file dengan nama **Animal.java**, **Cat.java** dan **Crocodile.java** dalam package **src**.

Selanjutnya, jadikan class **Animal.java** sebagai interface dengan menambahkan kode sebagai berikut.

```
public interface Animal {  
    // Method abstract  
    String eat();  
    String sleep();  
}
```

Kemudian untuk class **Cat.java** dan **Crocodile.java** buat jadi subclass dengan meng-implements interface **Character.java**. Untuk kode program nya seperti dibawah ini:

Cat.java

```
public class Cat implements Animal{  
    @Override  
    public String eat() {  
        return "Kucing makan ikan bakar";  
    }  
    @Override  
    public String sleep() {  
        return "Kucing tidur di kamar tidur";  
    }  
}
```

Crocodile.java

```
public class Crocodile implements Animal{  
    @Override  
    public String eat() {  
        return "Buaya darat makan ayam bakar";  
    }  
    @Override  
    public String sleep() {  
        return "Buaya darat tidur di asrama";  
    }  
}
```

```
}
```

Selanjutnya untuk class utama atau **App.java**, kode program nya sebagai berikut:

```
public class App {  
    public static void main(String[] args) throws Exception {  
        // Instansiasi objek dari class Cat dan Crocodile  
        Cat kucing = new Cat();  
        Crocodile buaya = new Crocodile();  
        // Memanggil method yang ada di setiap class  
        System.out.println(kucing.eat());  
        System.out.println(kucing.sleep());  
        System.out.println(buaya.eat());  
        System.out.println(buaya.sleep());  
    }  
}
```

Nah sekarang kan tadi class bisa multiple interface, adapun buat file baru lagi yaaitu **Attack.java** dengan tipe interface. Untuk kode nya sebagai berikut:

```
public interface Attack {  
    String attack();  
}
```

Kemudian di class utama atau **App.java** ada juga tambahan kode, yaitu:

```
System.out.println(buaya.attack());
```

6.3 Perbedaan Abstract Class & Interface

Interface	Abstract Class
Interface tidak dapat memiliki access specifier untuk fungsinya karena secara default bersifat public.	Abstract dapat memiliki access specifier.
Hanya dapat berisi signature, dan tidak dapat berisi implementasinya.	Dapat berisi implementasi secara lengkap.

Hanya dapat berisi signature, dan tidak dapat berisi implementasinya.	Kecepatan proses relative lebih cepat.
Tidak dapat berisi field.	Dapat berisi field dan konstanta.
Interface hanya bisa menampung method abstract.	Dapat menampung method non-abstract.

6.4 Overloading

Overloading adalah sebuah konsep dalam pemrograman Java yang memungkinkan untuk mendefinisikan beberapa metode dengan nama yang sama dalam satu kelas, asalkan parameter-parameter yang diterima oleh metode-metode tersebut berbeda. Overloading memungkinkan membuat beberapa versi dari metode dengan nama yang sama, tetapi dengan tipe, jumlah, atau urutan parameter yang berbeda.

Dalam overloading, metode-metode yang memiliki nama yang sama harus memiliki parameter yang berbeda, yaitu kombinasi yang unik dari jenis, jumlah, dan urutan parameter. Berikut adalah contoh sederhana overloading dalam Java:

```
public class Result {
    // Method pertama dengan 1 parameter bertipe int
    void printData(int nim){
        System.out.println("NIM : " + nim);
    }
    // Method kedua dengan 2 parameter bertipe int dan string
    void printData( int nim, String name){
        System.out.println("NIM : " + nim);
        System.out.println("Nama : " + name);
    }
    // Method ketiga dengan 3 parameter bertipe int, string dan string
    void printData(int nim, String name, String major){
        System.out.println("NIM : " + nim);
        System.out.println("Nama : " + name);
        System.out.println("Prodi : " + major);
    }
    public static void main(String[] args) {
        Result result = new Result();
        result.printData(123456789);
        result.printData(123456789, "Aisyah Romaito Siregar");
        result.printData(123456789, "Aisyah Romaito Siregar",
            "D3 Manajemen Informatika");
    }
}
```

Overloading memungkinkan membuat metode yang lebih fleksibel dan mudah digunakan dalam berbagai konteks. Ini mempermudah pemrograman karena dapat menggunakan nama metode yang sama untuk tugas yang berbeda, selama parameter berbeda.

BAB VII

PENGENALAN SPRING DAN SPRING BOOT

7.1 Framework

Framework adalah kerangka kerja yang digunakan untuk mengembangkan aplikasi berbasis desktop, mobile, maupun website. Framework ini diciptakan untuk membantu developer dalam menulis baris kode. Dengan menggunakan framework penulisan kode akan jauh lebih mudah, cepat, dan terstruktur rapi.



Java memiliki berbagai macam framework yang dapat digunakan untuk pengembangan aplikasi web, seluler, dan data science. Berikut adalah beberapa framework Java yang populer:

- Spring Boot : <https://spring.io/>
- Hibernate : <https://hibernate.org/>
- Struts : <https://struts.apache.org/>
- JSF : <https://www.oracle.com/java/technologies/javaserverfaces.html>
- Grails : <https://grails.org/>
- Dll.

Framework java yang akan kita pelajari adalah Spring, karena framework ini adalah framework paling populer di bahasa pemrograman java. Spring adalah kerangka kerja (framework) aplikasi Java yang populer untuk membangun aplikasi yang skalabel, mudah dipelihara, dan berbasis pada komponen (component-based). Spring menyediakan solusi untuk berbagai tantangan pengembangan perangkat lunak, termasuk manajemen objek, transaksi, keamanan, dan banyak lagi.

Spring memiliki berbagai fitur yang bisa kita coba, yaitu:

a) Spring Core Container

Ini adalah inti dari Spring Framework. Container ini mengelola siklus hidup objek, melakukan injeksi dependensi (dependency injection), dan menyediakan konfigurasi.

b) Spring AOP (Aspect-Oriented Programming)

Spring mendukung pemrograman berorientasi aspek, yang memungkinkan Anda untuk memisahkan perhatian lintas lapisan (cross-cutting concerns) dari kode bisnis.

c) Spring Data Access/Integration

Spring menyediakan abstraksi untuk berbagai teknologi akses data, seperti JDBC, JPA, Hibernate, dan lain-lain.

d) Spring MVC (Model-View-Controller)

Spring MVC adalah framework yang digunakan untuk mengembangkan aplikasi web dengan pola desain Model-View-Controller (Ini yang akan kita pelajari lebih lanjut).

Spring framework sebenarnya adalah kumpulan project-project dengan fungsionalitasnya masing-masing. Misalnya ada Spring Boot, Spring Security, Spring Rest, Spring Data, Spring Cloud, dan masih banyak lagi. Di pelatihan fundamental ini, kita akan membahas mengenai Spring Boot.

7.2 Spring Boot

Spring Boot adalah subproyek dalam ekosistem Spring yang dirancang untuk membuat pengembangan aplikasi Spring lebih cepat, lebih mudah, dan lebih ringkas. Sederhananya, spring boot adalah framework untuk membangun aplikasi java berbasis web tanpa perlu melakukan konfigurasi yang rumit.



Apa saja sih fitur-fitur yang dimiliki Spring Boot? Berikut ini adalah beberapa alasan para developer menggunakan framework ini:

- Konfigurasi Otomatis (Auto-Configuration): Spring Boot menyediakan konfigurasi otomatis untuk komponen-komponen Spring yang umum digunakan, sehingga kita dapat fokus pada pengembangan aplikasi.

- **Starter Packs:** Spring Boot menyediakan starter packs yang siap pakai untuk teknologi-teknologi tertentu, seperti web, data, keamanan, dan lain-lain.
- **Embedded Servers:** Spring Boot mendukung server web embedded seperti Tomcat, Jetty, dan Undertow, sehingga kita tidak perlu menginstal server web eksternal.
- **Production Ready:** Spring Boot menyediakan berbagai alat untuk membuat aplikasi siap untuk produksi, termasuk manajemen properti eksternal dan logika pengemasan.

7.3 Maven dan Gradle

7.3.1 Gradle

Gradle adalah alat otomatisasi pembangunan sumber terbuka yang berfokus pada fleksibilitas dan kinerja. Berikut keuntungan menggunakan gradle:

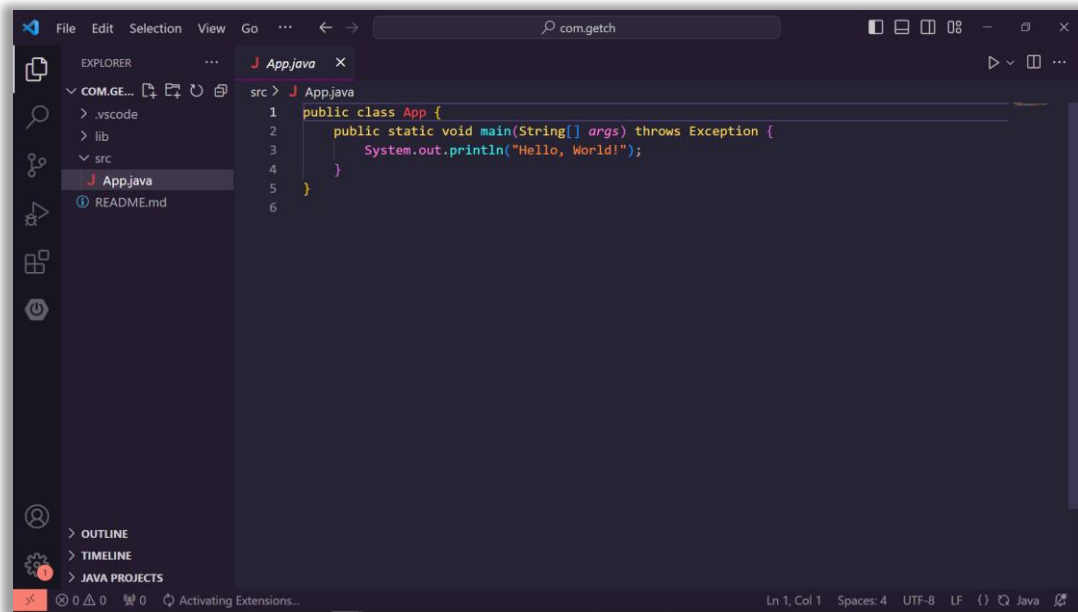
- Dengan Gradle, Anda lebih bebas untuk menentukan bagaimana Anda ingin menyusun proyek Anda.
- Anda dapat membuat struktur folder dan file sesuai keinginan Anda, sehingga lebih sesuai dengan kebutuhan proyek yang lebih kreatif dan tidak terbatas oleh konvensi ketat.
- Anda mendefinisikan susunan folder dan file proyek Anda dalam berkas skrip yang biasanya ditulis dalam bahasa Groovy atau Kotlin.
- Ini memberi Anda fleksibilitas lebih besar dalam pengaturan proyek dan tidak terlalu terikat oleh struktur konvensional.

7.3.2 Maven

Maven adalah salah satu build tool yang sering digunakan dalam proyek aplikasi Java. Dengan Maven kita bisa mengompilasi source code dengan mudah, melakukan testing, dan menginstal library yang dibutuhkan. Berikut adalah keuntungan menggunakan maven:

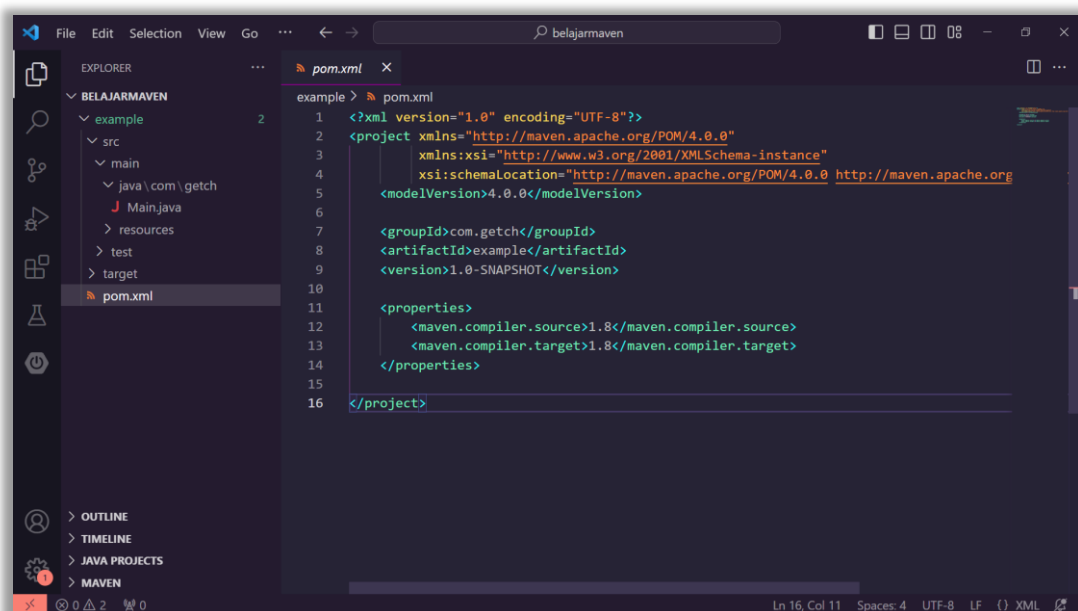
- Ketika kita menggunakan Maven, proyek Anda akan memiliki struktur folder yang sudah ditentukan dengan baik.
- Ada folder khusus untuk menyimpan kode sumber (seperti Java), folder untuk menyimpan sumber daya (seperti file konfigurasi), dan lain-lain.
- Maven juga memiliki berkas konfigurasi berformat XML yang disebut POM (Project Object Model) yang mengatur proyek kita dan dependensinya.
- Maven akan mengikuti konvensi ketat dalam menempatkan dan mengatur berkas dan folder di dalam proyek kita.

Lantas apa perbedaan ketika menggunakan maven dengan tidak? Jika kita tidak menggunakan maven maka struktur projectnya akan seperti gambar di bawah ini.



Di dalam struktur projectnya, terdapat package **lib** yang berfungsi sebagai penyimpanan library yang kita pasang. Jadi, untuk menambahkan library yang tidak disediakan oleh java, kita harus meng-install manual dan melakukan konfigurasi ulang, itu cukup merepotkan.

Namun, jika kita menggunakan maven sebagai build tools project, maka kita sudah mendapatkan file **pom.xml** dimana kita bisa mengatur dependensi yang dibutuhkan tanpa harus melakukan konfigurasi ulang.



Jadi, jika Anda ingin struktur folder dan file yang telah ditentukan dan konsisten, maka Maven adalah pilihan yang baik. Tetapi jika Anda ingin lebih fleksibel dan kreatif dalam mengatur struktur proyek Anda, Gradle adalah opsi yang lebih baik, dan di pelatihan kali ini kita akan menggunakan Maven untuk membuat proyek Spring Boot.

7.4 Set Up Spring Boot di Visual Studio Code

Visual Studio Code adalah lingkungan pengembangan yang sangat cocok untuk para pengembang aplikasi Spring Boot. Di sana terdapat beberapa ekstensi yang sangat berguna, seperti:

- Spring Boot Tools
- Spring Initializr
- Spring Boot Dashboard

Untuk mulai mengembangkan aplikasi Spring Boot di Visual Studio Code, kita perlu memasang hal-hal berikut:

- Java Development Kit (JDK)
- Paket Ekstensi untuk Java
- Paket Ekstensi Spring Boot

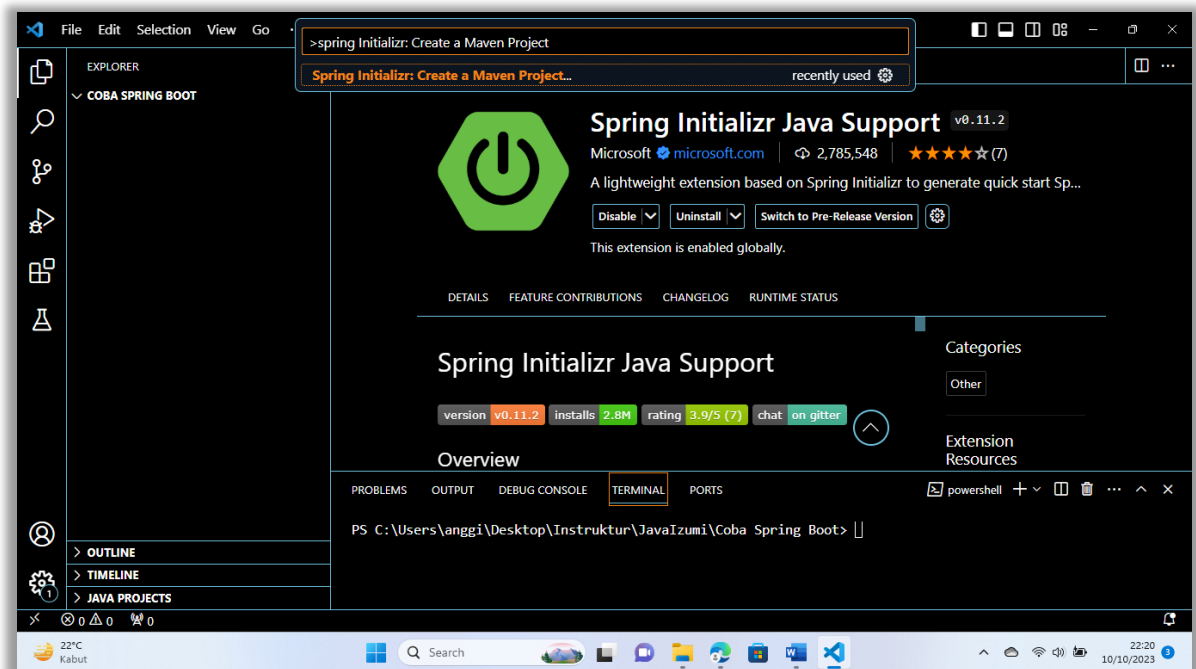
Untuk memudahkan pengembangan Spring Boot dengan Java, kita dapat menggunakan template profil Java Spring yang sudah termasuk ekstensi yang bermanfaat, pengaturan, dan potongan kode Spring Boot.

Ekstensi Spring Initializr memungkinkan Anda untuk mencari dependensi dan menghasilkan proyek Spring Boot yang baru.

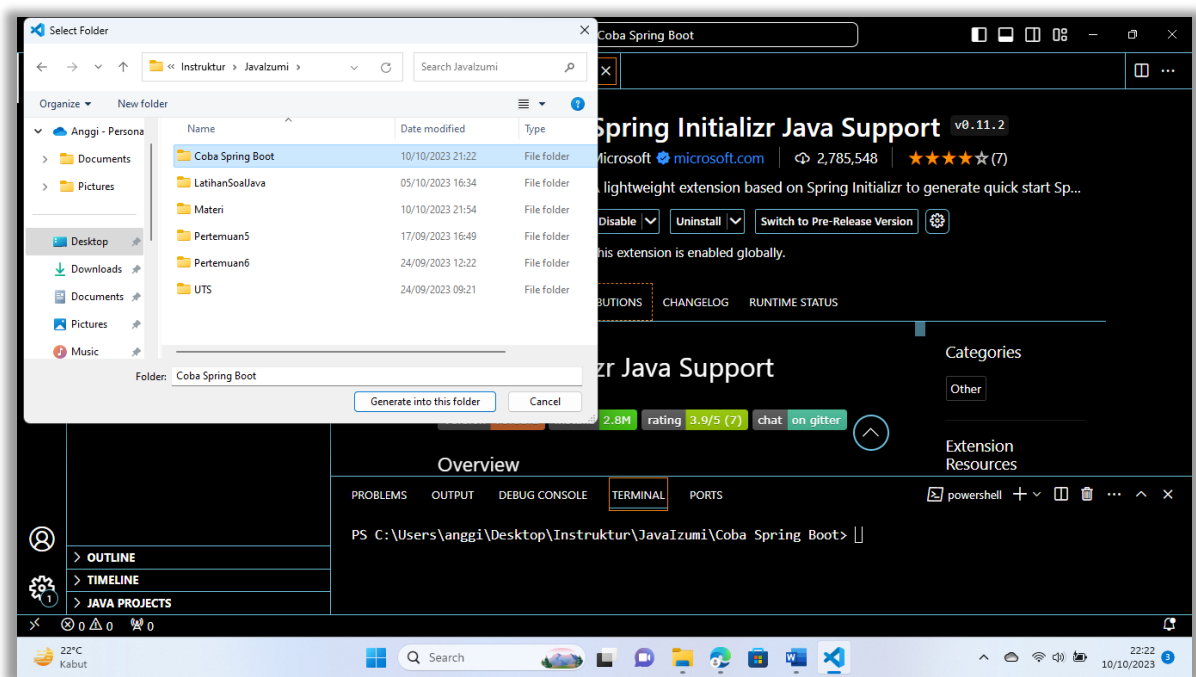
Untuk memasangnya, buka Visual Studio Code dan dari tampilan "**Extensions**" (Ctrl+Shift+X), cari "**Spring Initializr Java Support**".



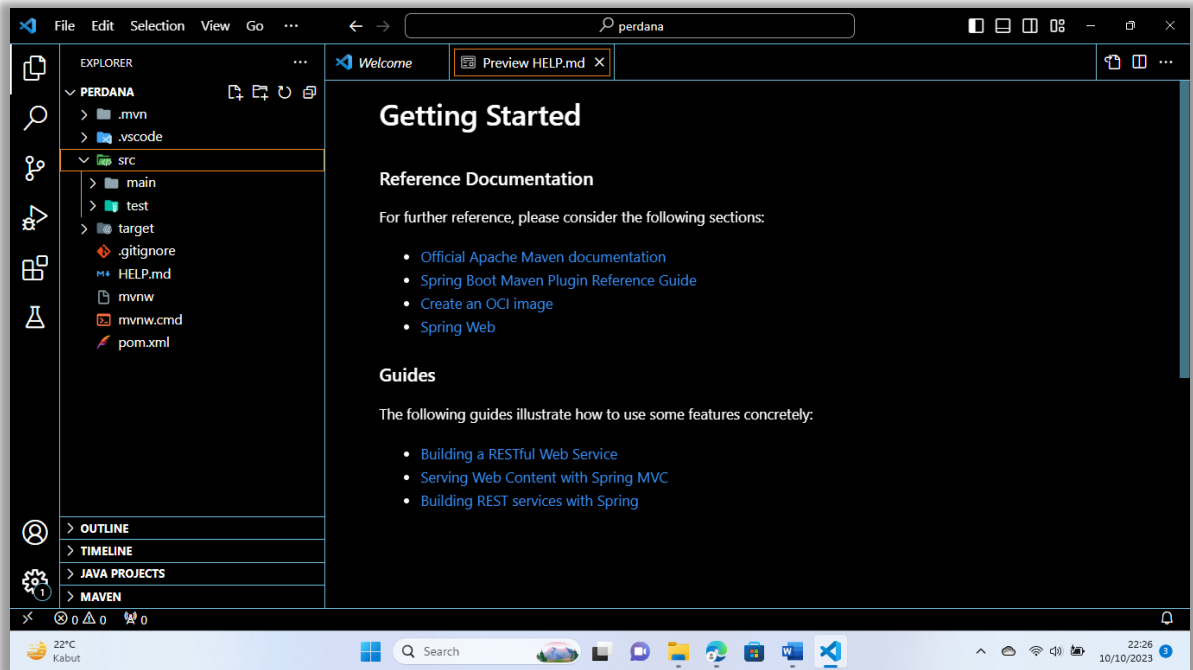
Setelah Anda memasang ekstensi ini, buka **"Command Palette"** (Ctrl+Shift+P) dan tulis perintah untuk mulai menghasilkan proyek Maven atau Gradle, lalu ikuti panduan yang muncul.



Setelah anda membuat project maka anda harus menentukan lokasi penyimpanan untuk project anda.

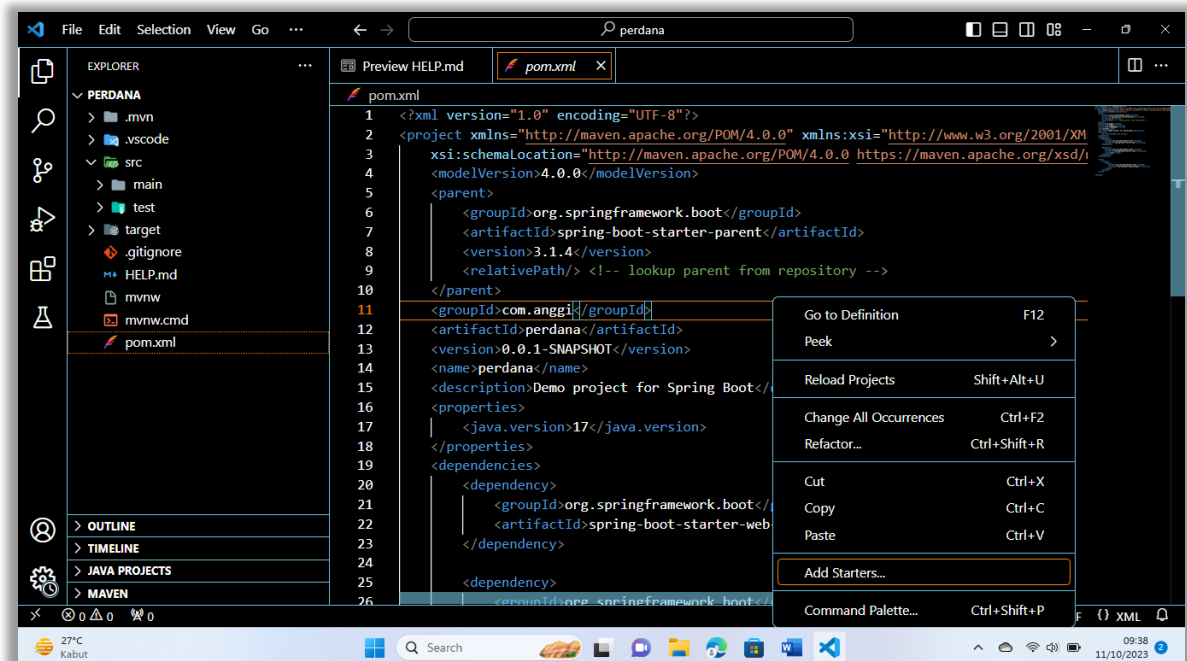


Berikut ini adalah tampilan awal ketika kita membuka project yang tadi telah anda buat.

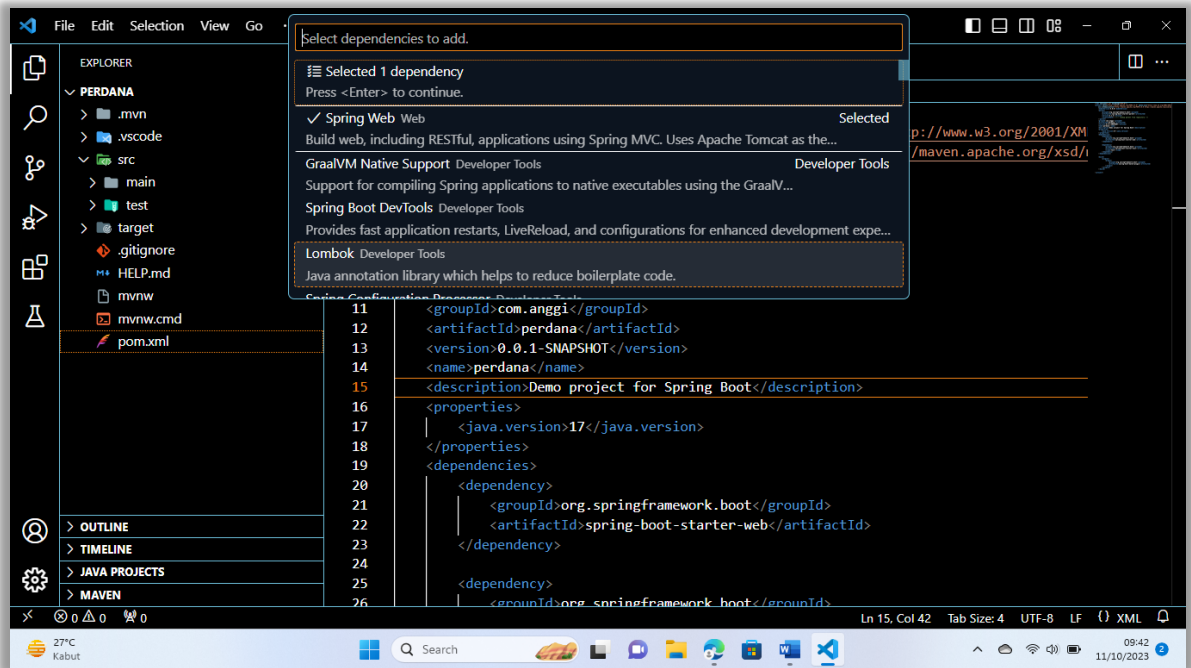


Ekstensi Spring Initializr juga memungkinkan kita untuk menambahkan dependensi setelah menghasilkan proyek Spring Boot yang baru.

Kita cukup buka berkas proyek dan klik kanan di dalamnya, lalu pilih **"Add starters..."**. Anda akan melihat daftar dependensi yang sudah ada, dimulai dengan tanda centang. Kita bisa mencari dependensi lain yang ingin kita tambahkan ke proyek, atau mengklik dependensi yang sudah ada untuk menghapusnya.



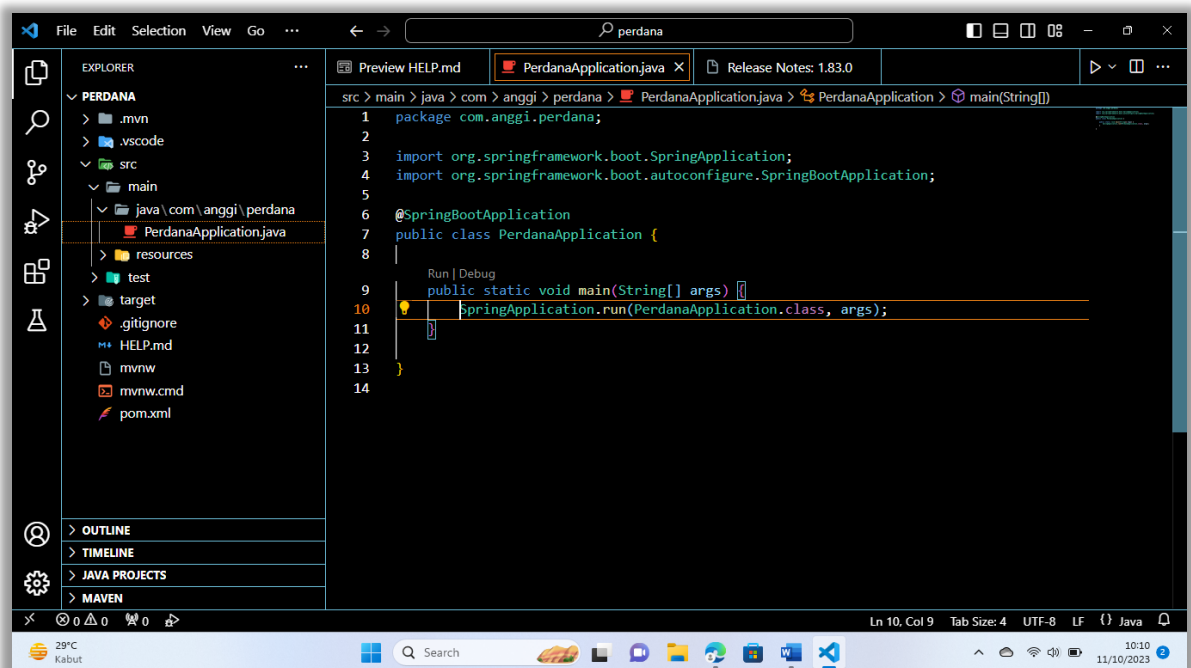
Maka selanjutnya adalah anda hanya cukup memilih dependensi yang akan ditambahkan ke dalam proyek Spring Boot ini.



Selanjutnya setelah menambahkan dependensi maka dependensi yang ditambahkan akan muncul di **pom.xml**.

Setelah selesai membuat proyek dengan Spring Boot, kita akan mencoba kode paling sederhana yang bisa kita akses melalui web browser, kita akan membuat program untuk mencetak **"Hello World"** ke web browser.

Buka file utama Java yang telah dibuat, biasanya terdapat di dalam package **src/main/java** dan seterusnya, lalu cari file yang berakhiran **.java**, file itulah yang akan kita ubah.



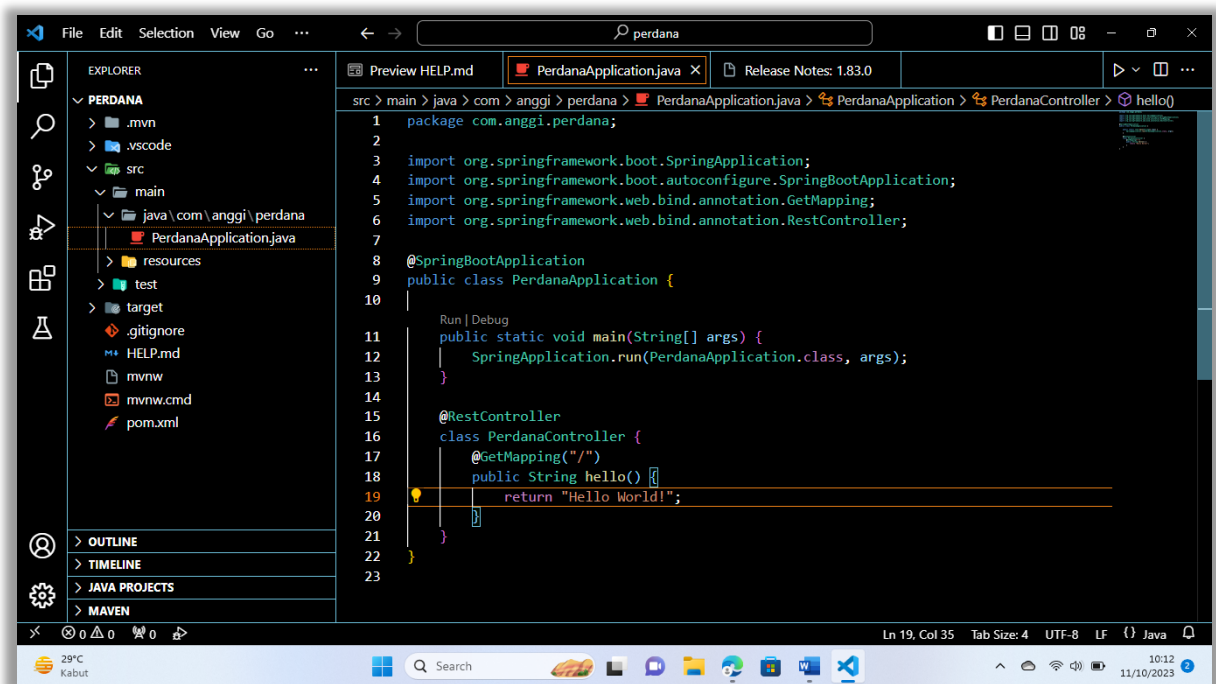
Selanjutnya tambahkan kode berikut pada file tersebut :

```
@Controller
class PerdanaController {
    @GetMapping("/")
    public String hello() {
        return "Hello World!";
    }
}
```

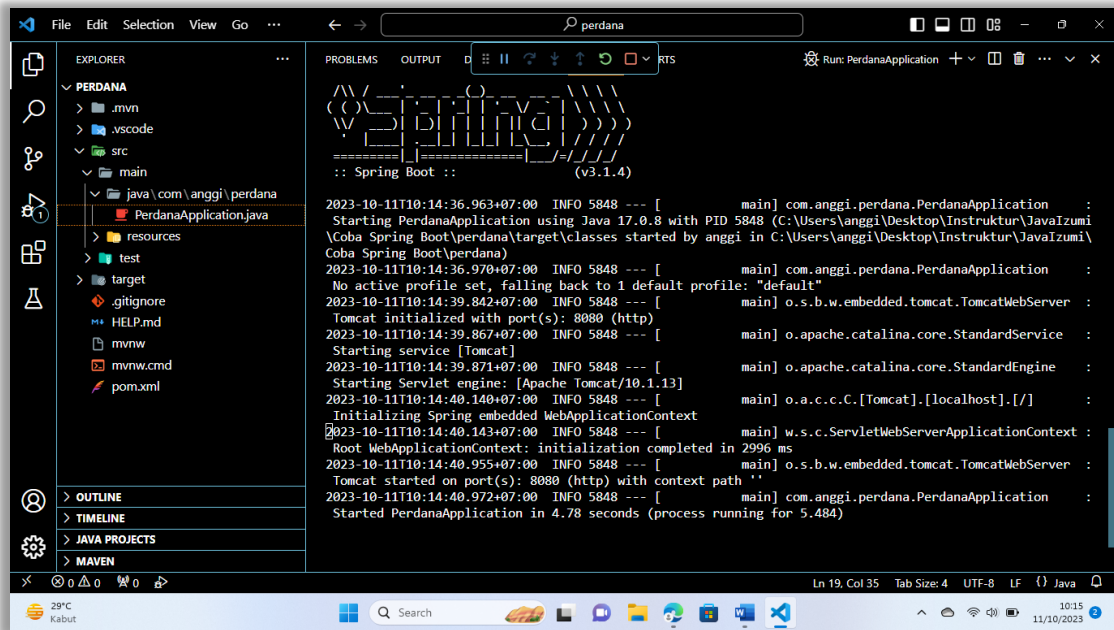
Penjelasan dari kode di atas adalah sebagai berikut:

- **@Controller** berfungsi untuk menangani request HTTP dan mengembalikan response HTTP.
- **@GetMapping** berfungsi untuk membuat endpoint HTTP GET.

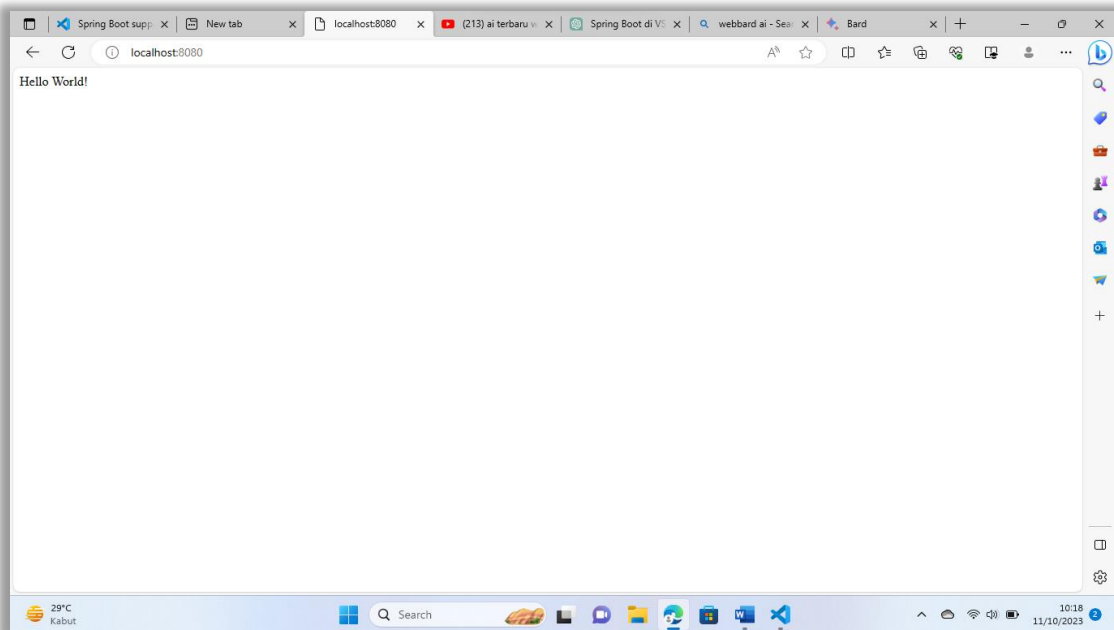
Hasilnya akan menjadi seperti ini :



Lalu run aplikasi dengan cara klik tombol segitiga di pojok kanan atas, atau klik kanan lalu pilih run java, maka akan otomatis memunculkan terminal dan tampilan seperti berikut :



Setelah itu buka web browser yang anda inginkan lalu ketik `http://localhost:8080/`, karena Spring Boot akan otomatis berjalan di port **8080**.



Jika tampilan web browser sudah menampilkan Hello World, maka selamat proyek Spring Boot pertama anda berjalan lancar.

BAB VIII

MODEL VIEW CONTROLLER (MVC) DI JAVA

8.1 Konsep MVC (Model-View-Controller)

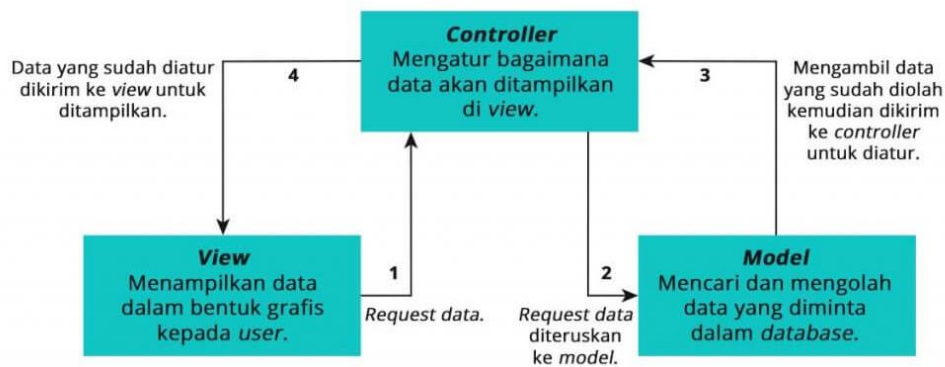
MVC adalah sebuah pola desain (design pattern) yang digunakan dalam pengembangan perangkat lunak untuk memisahkan aplikasi menjadi tiga komponen utama:

- **Model:** Bagian yang bertugas untuk menyiapkan, mengatur, memanipulasi, dan mengorganisasikan data yang ada di database.
- **View:** View bertanggung jawab untuk menampilkan data dari Model kepada pengguna dan menerima input dari pengguna. Ini adalah komponen yang mengatur tampilan grafis atau antarmuka pengguna. Dalam Java, view sering kali berupa kelas-kelas Swing atau JavaFX yang menangani GUI.
- **Controller:** Controller berfungsi sebagai penghubung antara Model dan View

8.2 Alur MVC

Setelah mengetahui penjelasan dan komponen dari MVC, sekarang kita akan membahas alur proses dari MVC. Berikut ini adalah alur prosesnya.

- a) Proses pertama adalah view akan meminta data untuk ditampilkan dalam bentuk grafis kepada pengguna.
- b) Permintaan tersebut diterima oleh controller dan diteruskan ke model untuk diproses.
- c) Model akan mencari dan mengolah data yang diminta di dalam database
- d) Setelah data ditemukan dan diolah, model akan mengirimkan data tersebut kepada controller untuk ditampilkan di view.
- e) Controller akan mengambil data hasil pengolahan model dan mengaturnya di bagian view untuk ditampilkan kepada pengguna.



8.3 Keuntungan Penggunaan MVC

- **Pemisahan Keperluan**, MVC memungkinkan pemisahan tugas yang jelas antara Model, View, dan Controller. Ini membuat kode lebih mudah dipahami, dikelola, dan diperbaiki.
- **Reusabilitas**, Setiap komponen (Model, View, dan Controller) dapat digunakan kembali dalam berbagai bagian aplikasi atau dalam proyek yang berbeda.
- **Kolaborasi Tim**, Dengan pemisahan yang jelas antara komponen-komponen MVC, tim pengembangan dapat bekerja secara paralel pada berbagai bagian aplikasi.
- **Mudah Diuji**, Setiap komponen dapat diuji secara terpisah, memudahkan pengujian dan pemecahan masalah.

8.4 Implementasi MVC dalam Java Spring Boot

Dalam Java Spring Boot, Kita dapat menerapkan konsep MVC menggunakan Spring MVC. Spring MVC adalah bagian dari framework Spring yang dirancang untuk memfasilitasi pengembangan aplikasi web berbasis Model-View-Controller. Berikut adalah langkah-langkah dasar untuk mengimplementasikan MVC dalam Spring Boot:

a) Model

Model dalam Spring Boot biasanya berupa kelas Java yang merepresentasikan data dan logika bisnis aplikasi. Kita dapat menggunakan anotasi seperti `@Entity` (untuk Hibernate) atau hanya kelas POJO (Plain Old Java Object) biasa untuk mendefinisikan Model Kita.

b) View

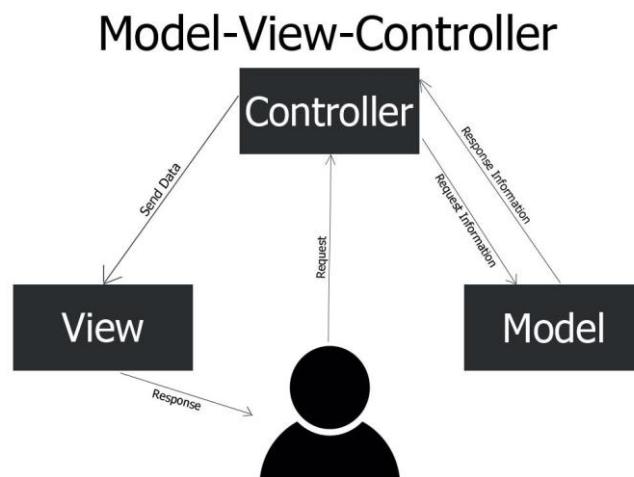
View dalam Spring Boot dapat berupa berbagai hal, mulai dari halaman HTML, Thymeleaf templates, hingga file JSON atau XML. View digunakan untuk menampilkan data dari Model kepada pengguna.

c) Controller

Controller adalah kelas yang menangani permintaan/request HTTP dari pengguna. Kita dapat menggunakan anotasi `@Controller` atau `@RestController` untuk mendefinisikan Controller. Controller berisi metode-metode yang menangani permintaan/request dan memberikan respon/response.

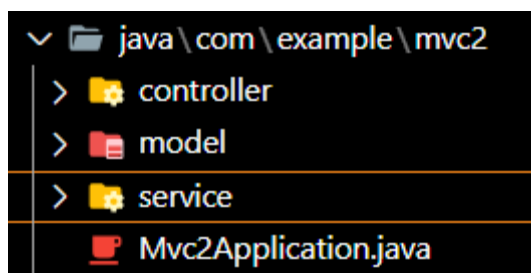
d) Routing

Kita harus menentukan rute (route) atau URL yang akan dikendalikan oleh masing-masing Controller. Kita dapat menggunakan anotasi `@RequestMapping` atau `@GetMapping`, `@PostMapping`, dll.

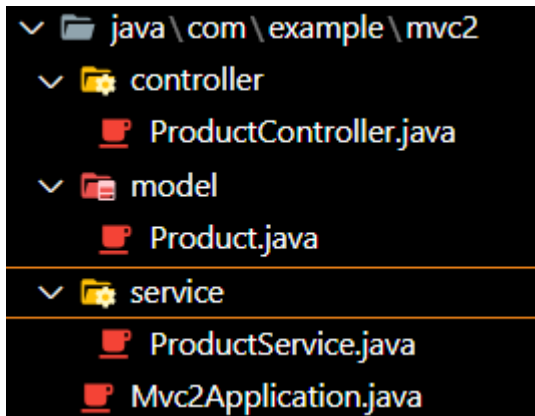


8.5 Contoh Implementasi

- Buat Project Maven Spring Boot seperti yang sudah dilakukan di materi ke 7
- Tambahkan folder/package controllers, models, dan services, Berikut adalah contohnya :



- Buat file `ProductController.java` di dalam package `controller`, `Product.java` di package `model`, dan `ProductService.java` di package `service`. Berikut adalah contohnya :



- d) Selanjutnya, kita akan fokus ke bagian model terlebih dahulu. Buka file **Product.java** lalu isi dengan kode berikut :

```
package com.example.mvc2.model;

public class Product {
    private Long id;
    private String name;
    private double price;

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }
}
```

```

    public void setPrice(double price) {
        this.price = price;
    }
}

```

Class **Product** ini adalah representasi dasar dari produk dalam aplikasi Kita. Kita dapat membuat objek **Product**, mengatur atau mendapatkan nilai atributnya, dan menggunakannya dalam aplikasi Kita, seperti menampilkan daftar produk atau melakukan operasi CRUD pada produk.

- e) Selanjutnya adalah bagian service, di dalam ProductService.java tambahkan kode berikut :

```

package com.example.mvc2.service;

import java.util.ArrayList;
import java.util.List;
import org.springframework.stereotype.Service;
import com.example.mvc2.model.Product;

@Service
public class ProductService {
    public List<Product> getAllProducts() {
        List<Product> products = new ArrayList<>();
        products.add(new Product(1L, "Produk A", 10.0));
        products.add(new Product(2L, "Produk B", 20.0));
        products.add(new Product(3L, "Produk C", 30.0));
        return products;
    }
}

```

Kode ProductService.java yang Kita berikan adalah kelas service yang menyediakan layanan terkait dengan entitas produk dalam aplikasi Kita. Mari kita bahas dengan lebih detail:

- **@Service** Dengan anotasi ini, Spring akan mendeteksi dan mengelola kelas **ProductService** sebagai komponen **service**, dan Kita dapat menginjeksinya ke dalam komponen lain seperti kelas controller.
- **return products;** Metode **getAllProducts()** mengembalikan List yang berisi semua produk yang telah ditambahkan. Ini adalah daftar produk yang akan digunakan oleh controller untuk menampilkan daftar produk dalam tampilan.

ProductService berfungsi untuk menyediakan daftar produk secara statis. Dalam pengembangan yang lebih lanjut, Kita bisa menggantikan implementasi ini dengan memuat produk dari database atau sumber data lain sesuai kebutuhan Kita.

- f) Selanjutnya adalah bagian controller, di dalam ProductController.java tambahkan kode berikut :

```
package com.example.mvc2.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import com.example.mvc2.model.Product;
import com.example.mvc2.service.ProductService;

@Controller
public class ProductController {
    @Autowired
    private ProductService productService;

    @GetMapping("/products")
    public String listProducts(Model model) {
        List<Product> products = productService.getAllProducts();
        model.addAttribute("products", products);
        return "productList";
    }
}
```

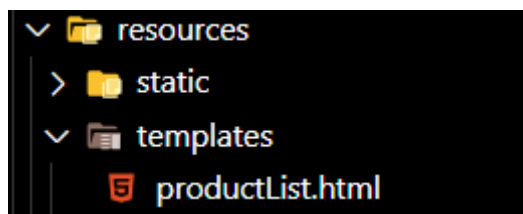
Kode **ProductController.java** adalah kelas controller yang menangani permintaan/request HTTP terkait produk dan mengkoordinasikan antara **model** (entitas produk) dan tampilan (**view**). Mari kita bahas dengan lebih detail:

- **@Controller** adalah anotasi Spring yang digunakan untuk menandai kelas **ProductController** sebagai komponen controller.
- **@Autowired** adalah anotasi Spring yang digunakan untuk melakukan injeksi dependensi. Dalam kasus ini, kita menggunakannya untuk menginjeksikan kelas **ProductService** ke dalam kelas **ProductController**, sehingga Anda dapat mengakses layanan yang disediakan oleh **ProductService**.

- `@GetMapping("/products")` adalah anotasi `@GetMapping` yang menentukan bahwa method `listProducts` akan menangani permintaan/request HTTP `GET` pada URL `/products`. Ketika URL `/products` diakses melalui peramban web, metode `listProducts` akan dipanggil.
- `return "productList";` Metode `listProducts` mengembalikan string `"productList"`. Ini adalah nama tampilan (view) yang akan digunakan untuk menampilkan produk. Nama ini harus sesuai dengan nama file template Thymeleaf atau JSP yang akan digunakan sebagai tampilan produk.

Kode ini menggambarkan bagaimana kelas `controller` bekerja dalam aplikasi Spring MVC, mengelola permintaan HTTP, berinteraksi dengan layanan (`ProductService`), dan mengirimkan data ke tampilan untuk ditampilkan kepada pengguna.

- g) Sekarang tambahkan tampilan sederhana html dengan menggunakan bantuan dependensi thymeleaf. Buat file `productList.html` di dalam `resources/templates`



Lalu tambahkan kode html berikut :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Product List</title>
</head>

<body>
  <h1>Daftar Produk</h1>
  <table>
    <tr>
      <th>ID</th>
      <th>Nama</th>
      <th>Harga</th>
    </tr>
    <tr th:each="product : ${products}">
      <td th:text="${product.id}"></td>
      <td th:text="${product.name}"></td>
      <td th:text="${product.price}"></td>
    </tr>
  </table>
</body>
```

```
</html>
```

Jangan lupa menambahkan dependensi thymeleaf di `pom.xml` :

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-thymeleaf</artifactId>  
</dependency>
```

Setelah itu coba build ulang semua dependensi sampai selesai, kemudian project tersebut di run. Terakhir, buka hasil project melalui browser dengan URL `http://localhost:8080/products` maka akan menampilkan output berikut :



The screenshot shows a web browser window with a white background. At the top, the title 'Daftar Produk' is displayed in a large, bold, black serif font. Below the title, there is a table with three columns: 'ID', 'Nama', and 'Harga'. The table contains three rows of data: '1 Produk A 10.0', '2 Produk B 20.0', and '3 Produk C 30.0'. The text is in a black serif font.

ID	Nama	Harga
1	Produk A	10.0
2	Produk B	20.0
3	Produk C	30.0

Note : Untuk Thymeleaf akan dipelajari di pertemuan mendatang setelah materi JPA.

BAB IX

THYMELEAF

9.1 Pengertian Thymeleaf

Thymeleaf adalah mesin template Java XML/HTML/Javascript/CSS dan bahkan teks biasa yang dapat bekerja baik di lingkungan web maupun non-web. Namun lebih cocok untuk melayani XHTML/HTML5 pada lapisan tampilan aplikasi web berbasis MVC, tetapi dapat memproses file XML apa pun bahkan di lingkungan offline.

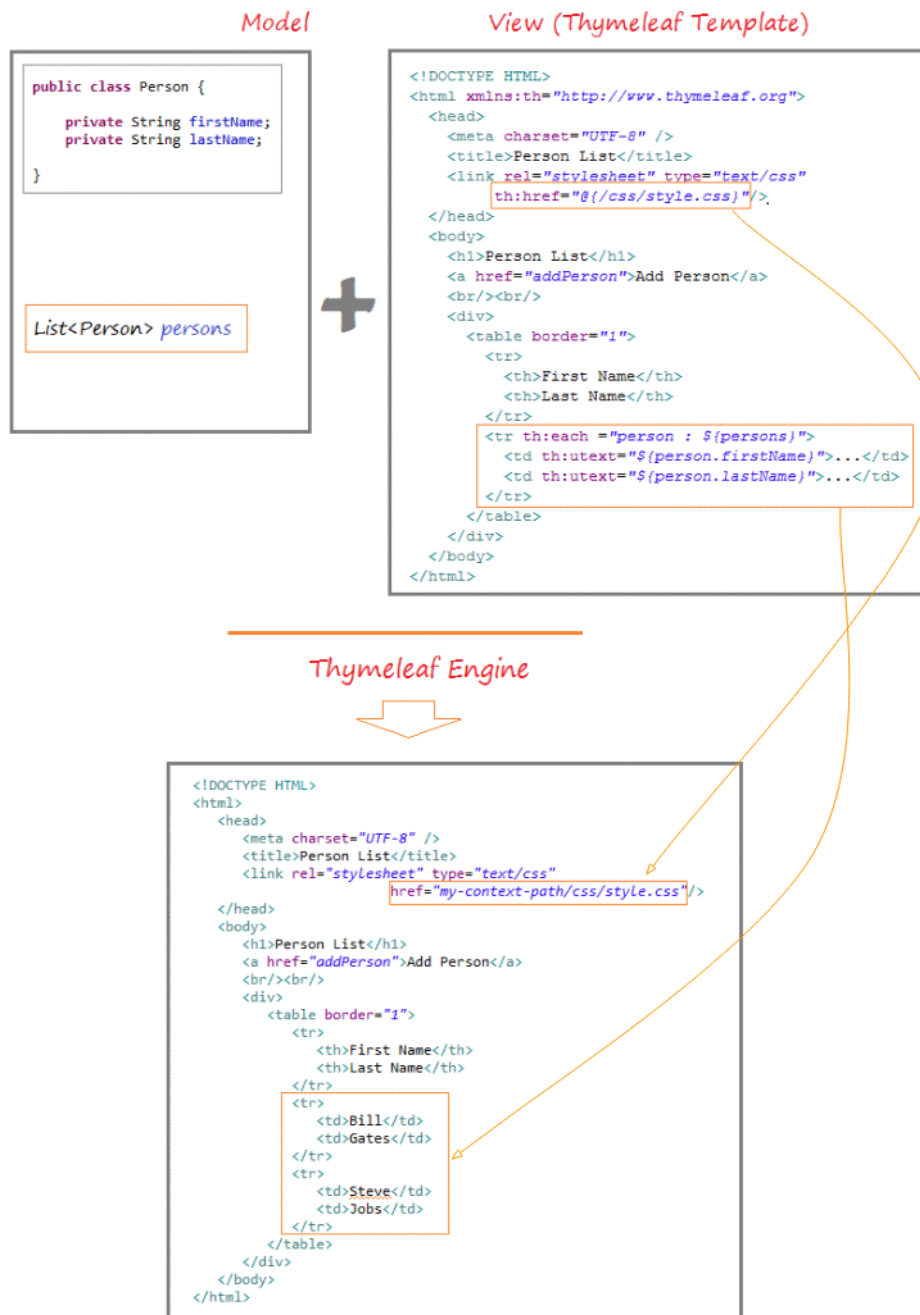
Berikut adalah halaman HTML dengan atribut dan ekspresi Thymeleaf:

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Home Page</title>
7     <link
8       href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css"
9       rel="stylesheet"
10      integrity="sha384-4bw+/aepP/YC94hEpVNVgiZdgIC5+VKNBQNGCHeKRN+PtmoHDEsupvnDJzQIU9"
11      crossorigin="anonymous"
12    />
13   </head>
14
15   <body>
16     <div class="container my-2">
17       <h2>Cars List</h2>
18       <a th:href="@{/add}" class="btn btn-primary btn-sm mb-3">Add Car</a>
19       <table
20         class="table table-dark table-hover table-striped table-responsive-md rounded-3"
21       >
22         <thead class="text-center">
23           <tr>
24             <!-- <th>ID</th> -->
25             <th>Brand</th>
26             <th>Model</th>
27             <th>Description</th>
28             <th>Action</th>
29           </tr>
30         </thead>
31         <tbody>
32           <tr th:each="car : ${allCarList}" class="text-center">
33             <!-- <td th:text="${car.id}"></td> -->
34             <td th:text="${car.brand}"></td>
35             <td th:text="${car.model}"></td>
36             <td th:text="${car.description}"></td>
37             <td class="d-flex justify-content-evenly">
38               <a
39                 th:href="@{/updatecar/{id}(id=${car.id})}"
40                 class="btn btn-secondary"
41               >Update</a>
42             >
43             <a th:href="@{/delete/{id}(id=${car.id})}" class="btn btn-danger"
44             >Delete</a>
45           >
46         </td>
47       </tr>
48     </tbody>
49   </table>
50 </div>
51 </body>
52 </html>

```

Dibawah ini merupakan proses Thymeleaf Engine memproses Thymeleaf Template :



Dari gambar diatas dapat disimpulkan bahwa, dalam aplikasi web Spring MVC, Controller Spring MVC mengembalikan tampilan (Thymeleaf Template) sebagai hasilnya dan akan dirender lagi di browser.

Selama kita belajar Thymeleaf, untuk semua program simpan dalam satu proyek sama agar kalian tidak perlu membuat proyek setiap pertemuannya. Jadi selama belajar Thymeleaf ini buatlah 1 proyek dengan nama domain com.thymeleaf dan nama proyek getchstore.

9.2 Thymeleaf Dan Spring Boot

Thymeleaf dan Spring Boot adalah dua teknologi yang dapat digunakan bersama- sama untuk

membangun aplikasi web yang modern dan scalable. Thymeleaf adalah template engine yang dapat digunakan untuk membuat halaman web statis atau dinamis. Thymeleaf mendukung HTML, XML, dan teks sebagai template. Thymeleaf juga mendukung berbagai macam fitur, seperti perulangan, perbandingan, dan ekspresi.

Spring Boot adalah framework Java yang dapat digunakan untuk membangun aplikasi web dan layanan web. Spring Boot mengabstraksi kerumitan konfigurasi Spring Framework, sehingga memudahkan pengembang untuk membangun aplikasi web yang kompleks dengan cepat dan mudah.



Ini adalah gambaran suatu proses dalam pengembangan aplikasi berbasis web dengan menggunakan Spring Boot dan Thymeleaf. Untuk menggunakan Thymeleaf dan Spring Boot bersama-sama, kita perlu menambahkan dependensi Thymeleaf ke `pom.xml` project kita.

Berikut adalah contoh `pom.xml` yang menambahkan dependensi Thymeleaf:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Untuk mencoba dalam pengaplikasian antara Spring Boot dan Thymeleaf, pertama terlebih dahulu buat package atau folder `model` dalam dalam projek yang sudah dibuat sebelumnya. Kemudian di dalam package `controller` ini buat file `HelloWorldController.java` dan untuk isi file nya seperti dibawah ini :

```

package com.thymeleaf.getchstore.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HelloWorldController { @GetMapping("hello")

    public String hello(Model model) {
        model.addAttribute("message", "Hello World !!!");

        return "helloworld";
    }
}

```

Penjelasan dari program diatas :

HelloWorldController.java :

Annotation `@Controller` adalah annotation yang digunakan untuk menandai kelas sebagai controller Spring Boot. `@Controller` adalah kelas yang bertanggung jawab untuk menangani permintaan HTTP. `Model` digunakan untuk menyimpan data yang akan ditampilkan di halaman web. `@GetMapping` digunakan untuk mendefinisikan metode yang akan dijalankan ketika klien HTTP melakukan permintaan GET ke URL `/hello`.

Sedangkan untuk kode `model.addAttribute("message", "Hello World !!!");` artinya untuk menambahkan sebuah data kedalam model. Data yang ditambahkan adalah `message` dengan isi atau value `"Hello World !!!"`. Dan untuk `return "helloworld";` digunakan untuk mengembalikan nama file template Thymeleaf yang akan ditampilkan. File template Thymeleaf tersebut memiliki nama `helloworld.html`. File template Thymeleaf `helloworld.html` dapat berisi kode HTML biasa dan kode Thymeleaf. Kode Thymeleaf digunakan untuk menampilkan data yang telah ditambahkan ke `model`.

Kemudian buat file html di bagian package `resource.templates`, beri nama file html ini dengan string yang di return pada method `String hello` yang sudah di buat sebelumnya yaitu `helloworld.html`. Untuk isi file `helloworld.html` sebagai berikut :

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Spring Boot With Thymeleaf</title>
  </head>
  <body>
    <h1 th:text="${message}"></h1>
  </body>
</html>

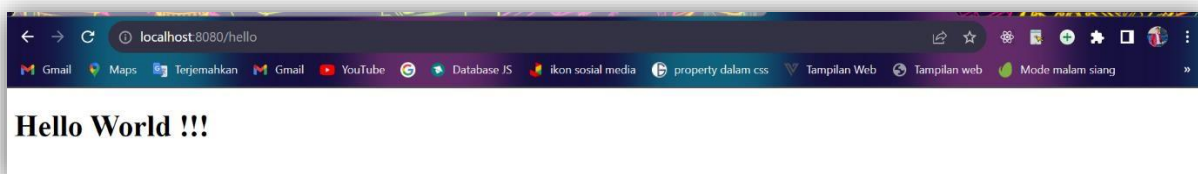
```

Penjelasan dari program diatas :

Atribut `th:text` digunakan untuk menampilkan nilai dari variabel `message`. Variabel `message` ini telah ditambahkan ke `model` oleh controller. Jadi, ketika HTTP melakukan permintaan GET ke URL `/hello`, Spring Boot akan menjalankan metode `hello()` dari controller `HelloWorldController`.

Metode `hello()` akan menambahkan data ke `model` dan mengembalikan nama file template Thymeleaf. Spring Boot akan merender file template Thymeleaf tersebut dan menampilkannya di browser. Pada halaman web yang ditampilkan, akan muncul pesan `"Hello World !!!"`.

Hasilnya:



9.3 Atribut Dalam Thymeleaf

Atribut dalam Thymeleaf adalah bagian dari tag HTML yang digunakan untuk mengontrol tampilan halaman web. Atribut ini dapat digunakan untuk menampilkan nilai dari variabel, menjalankan JavaScript, atau menentukan kondisi untuk menampilkan elemen HTML.

Thymeleaf memiliki berbagai macam atribut yang dapat digunakan. Berikut adalah beberapa atribut yang paling umum digunakan:

- Atribut ekspresi : Atribut ini digunakan untuk menampilkan nilai dari variabel atau ekspresi.
- Atribut kondisional : Atribut ini digunakan untuk menampilkan elemen HTML jika kondisi terpenuhi.
- Atribut iterasi : Atribut ini digunakan untuk mengulang elemen HTML berdasarkan data yang ada di variabel.
- Atribut aksi : Atribut ini digunakan untuk menjalankan JavaScript atau tindakan lainnya.

Adapun berikut ini atribut yang ada di dalam thymeleaf diantaranya yaitu `th:action`, `th:attr`, `th:block`, `th:case`, `th:classappend`, `th:each`, `th:field`, `th:fragment`, `th:href`, `th:if`, `th:include`, `th:insert`, `th:object`, `th:remove`, `th:replace`, `th:src`, `th:style`, `th:switch`, `th:text`, `th:unless`, `th:utext`, `th:value` dan `th:with`.

9.4 Expression Dalam Thymeleaf

Dalam thymeleaf terdapat beberapa atribut yang dapat digunakan diantaranya :

9.4.1 Thymeleaf Variable Expression

Atribut `th:text` berguna untuk menampilkan teks yang dihasilkan atau didapatkan dari ekspresi yang ada di dalamnya.

- Contoh 1 : Atribut Thymeleaf dengan Variable Expressions :

Fitur ini memungkinkan untuk membuat tampilan HTML yang dinamis dan sesuai dengan data yang berasal dari model Java atau dari lingkup tampilan Thymeleaf. Kita dapat menampilkan data, menghitung nilai, atau melakukan operasi lainnya dalam tampilan menggunakan variable expressions.

Untuk membuat contoh penggunaan variable expressions ini, pertama buat terlebih dahulu package model kemudian di dalam package model buat file dengan nama `Laptop.java` dengan isi file sebagai berikut :

```
package com.thymeleaf.getchstore.model;

public class Laptop {

    private String id;

    private String merk;

    private String tipe;

    private String harga;

    public Laptop() {

    }

    public Laptop(String id, String merk, String tipe, String harga) {

        this.id = id;
        this.merk = merk;
        this.tipe = tipe;
        this.harga = harga;

    }

    public String getId() {

        return id;

    }

}
```

```
public void setId(String id) {  
    this.id = id;  
}  
  
public String getMerk() {  
    return merk;  
}  
  
public void setMerk(String merk) {  
    this.merk = merk;  
}  
  
public String getTipe() {  
    return tipe;  
}  
  
public void setTipe(String tipe) {  
    this.tipe = tipe;  
}  
  
public String getHarga() {  
    return harga;  
}  
  
public void setHarga(String harga) {  
    this.harga = harga;  
}  
}
```

Kemudian buat file `controller` dari `model` yang sudah dibuat diatas nama file `LaptopController.java` dan untuk isi file adalah sebagai berikut :

```
package com.thymeleaf.getchstore.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping; import
com.thymeleaf.getchstore.model.Laptop;

@Controller
public class LaptopController {
    @GetMapping("data-laptop")

    public String dataLaptop(Model model) {

        Laptop laptop = new Laptop("1", "Asus", "ROG Zephyrus G14",
20000000);

        model.addAttribute("dataLaptop", laptop);
        return "laptop";

    }
}
```

Yang terakhir buatlah file html untuk menampilkan data yang sudah dibuat diatas. Nama file html harus sama dengan string yang di return di `LaptopController` yaitu `laptop.html`. Untuk isi file adalah sebagai berikut :

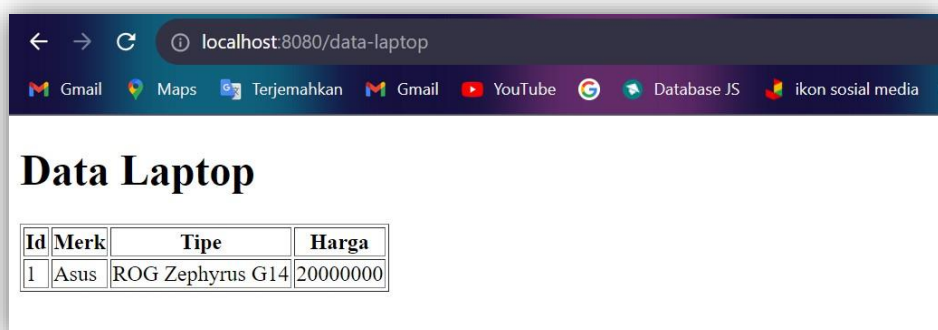
```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Data Laptop</title>
  </head>
  <body>
    <h1>Data Laptop</h1>
    <table border="1">
      <tr>
        <th>Id</th>
        <th>Merk</th>
        <th>Tipe</th>
        <th>Harga</th>
      </tr>
      <tr>
        <td th:text="${dataLaptop.id}"></td>
        <td th:text="${dataLaptop.merk}"></td>
        <td th:text="${dataLaptop.tipe}"></td>
        <td th:text="${dataLaptop.harga}"></td>
      </tr>
    </table>
  </body>
</html>

```

Atribut `th:text` akan mengisi content dari suatu tag atau mengganti content tag apabila sebelumnya sudah terdapat konten. Dalam contoh diatas, diartikan bahwa atribut `th:text` akan mengisi setiap tag `<td>` dengan data dari masing-masing data dari variable `dataLaptop`. Untuk mengetahui hasilnya bisa langsung mengakses link <http://localhost:8080/data-laptop>.

Hasilnya :



- Contoh 2 : Atribut Thymeleaf `th:text` dengan Selection Expressions :

Selection expressions adalah jenis expressions yang digunakan dalam Thymeleaf untuk menampilkan nilai dari atribut atau properti dari objek. Sintaks selection expressions adalah `*{attribute}`, di mana

attribute adalah nama atribut atau properti. Selection expressions dapat digunakan untuk menampilkan nilai dari objek yang telah ditambahkan ke model oleh controller. Objek tersebut dapat berupa objek sederhana, seperti objek string, objek integer, atau objek boolean.

Untuk mengetahui bagaimana penggunaan selection expressions, tambahkan kode program dibawah ini kedalam file `LaptopController.java` dan `laptop.html`.

`LaptopController.java` :

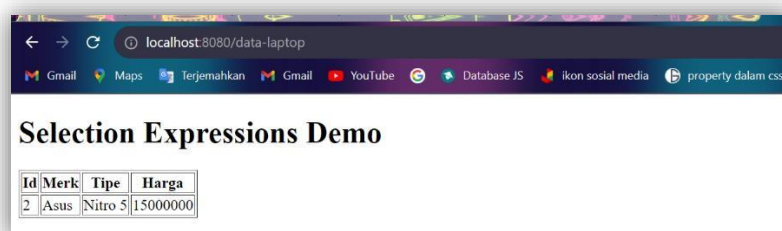
```
Laptop laptop2 = new Laptop("2", "Asus", "Nitro 5", 15000000);
model.addAttribute("dataLaptop2", laptop2);
```

`laptop.html` :

```
<h1>Selection Expressions Demo</h1>
<table border="1">
  <tr>
    <th>Id</th>
    <th>Merk</th>
    <th>Tipe</th>
    <th>Harga</th>
  </tr>
  <tr th:object="${dataLaptop2}">
    <td th:text="*{id}"></td>
    <td th:text="*{merk}"></td>
    <td th:text="*{tipe}"></td>
    <td th:text="*{harga}"></td>
  </tr>
</table>
```

Pada file `LaptopController.java` cukup menambahkan data laptop. Untuk di file `laptop.html`, pada tag `tr` terdapat `th:object="${dataLaptop2}"` digunakan untuk pemanggilan sebuah objek dari controller dengan nama variable `dataLaptop2`, dan `th:text="*{id}"` untuk menampilkan nilai properti `id` dari objek laptop, dan seterusnya.

Hasilnya :



- Contoh 3 : Atribut Thymeleaf `th:text` dengan Message Expressions `Message Expressions` adalah jenis `expressions` yang digunakan dalam Thymeleaf untuk menampilkan pesan dari file properties. Sintaks Message Expressions adalah `#{key}`, di mana `key` adalah nama kunci dari pesan yang akan ditampilkan.

Pesan yang akan ditampilkan disimpan dalam file properties. File properties adalah file teks yang berisi pasangan `key-value`. `Key` adalah nama kunci dari pesan, sedangkan `value` adalah nilai dari pesan.

Untuk mengetahui bagaimana penggunaan message expressions, buat terlebih dahulu file `messages.properties`, di dalam folder `/resources` dan tambahkan kode dibawah ini :

```
app.name=Getch Store
```

Dalam file `LaptopController.java` tambahkan kode program seperti dibawah ini :

```
@GetMapping("message-expression")

public String messageExpression() {
```

```
return "message-expression";
```

Yang terakhir buatlah file `message-expression.html` dan masukan kode program dibawah ini :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8" />
    <title>Message Expressions</title>
  </head>
  <body>
    <h2 th:text="#{app.name}"></h2>
    <h2 th:text="#{welcome.message}"></h2>
  </body>
</html>
```

Hasilnya :



Dalam contoh di atas, `th:text` digunakan dengan message expressions `#{app.name}` dan `#{welcome.message}` untuk menampilkan teks “Getch Store” dan “Selamat Datang di Aplikasi Getch Store” dengan pesan lokal yang sesuai dengan kunci `“app.name”` dan `“welcome.message”` yang telah didefinisikan dalam `messages.properties`.

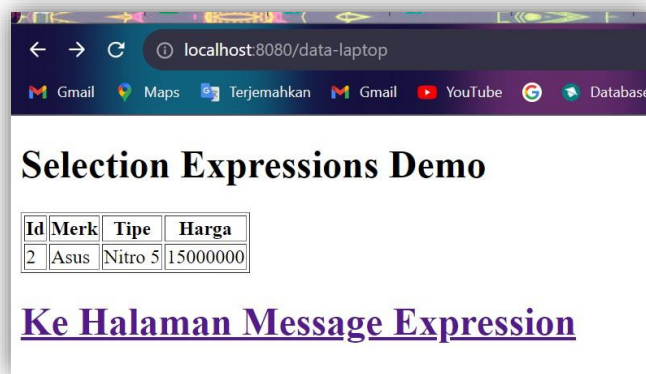
9.4.2 Thymeleaf Link Expression

Thymeleaf Link Expression adalah jenis expressions yang digunakan dalam Thymeleaf untuk membuat tautan ke view atau resource lainnya. Sintaks Thymeleaf Link Expression adalah `@{url}`, di mana `url` adalah URL dari view atau resource yang akan ditautkan. Thymeleaf Link Expression dapat digunakan untuk membuat tautan ke view atau resource lainnya.

Agar lebih jelas, maka langsung saja kepada contoh penerapan dari Thymeleaf Link Expression ini. Pertama, cukup tambahkan kode berikut di dalam file `laptop.html` :

```
<h1>
  <a th:href="@{message-expression}">Ke Halaman Message
  Expression</a>
</h1>
```

Nantinya tampilan file `laptop.html` akan seperti dibawah ini :



Jadi apabila kita klik `Ke Halaman Message Expression` maka akan otomatis ke link ke file `message-expression.html` yang isinya ada 2 pesan tadi yang sudah dibuat sebelumnya.

9.4.3 Thymeleaf Fragment Expression

Thymeleaf Fragment Expression adalah fitur dalam Thymeleaf yang memungkinkan kita untuk membagi tampilan HTML menjadi fragmen-fragmen kecil yang dapat digunakan ulang di berbagai bagian halaman web. Dengan kata lain, kita dapat membuat dan menggunakan potongan kode HTML yang sering digunakan di beberapa tempat tanpa harus mengulanginya.

Sintaks fragment expression adalah `th:fragment="fragment_name"`, di mana `fragment_name` adalah nama dari fragment yang akan dirujuk.

Contoh penggunaan Thymeleaf Fragment Expression adalah sebagai berikut :

Pertama buatlah sebuah folder `fragments` dalam folder `templates`, setelah itu dalam folder `fragments` buat 1 file html ialah `component.html` dengan isi kode program sebagai berikut :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Header Fragment</title>
  </head>
  <body>
    <header th:fragment="header">
      <h1>Ini adalah Header</h1>
    </header>
    <footer th:fragment="footer">
      <h1>Ini adalah Footer</h1>
    </footer>
  </body>
</html>
```

Kedua, buatlah file `fragment-expression.html` dalam folder `templates` dan masukan kode program dibawah ini :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <meta charset="UTF-8" />
    <title>Fragment Expressions</title>
  </head>
  <body>
    <header th:replace="~{fragments/component :: header}"></header>
    <hr />
    <div th:fragment="section">
      <h1>Ini adalah fragment section</h1>
    </div>
    <hr />
    <footer th:insert="~{fragments/component :: footer}"></footer>
  </body>
</html>
```

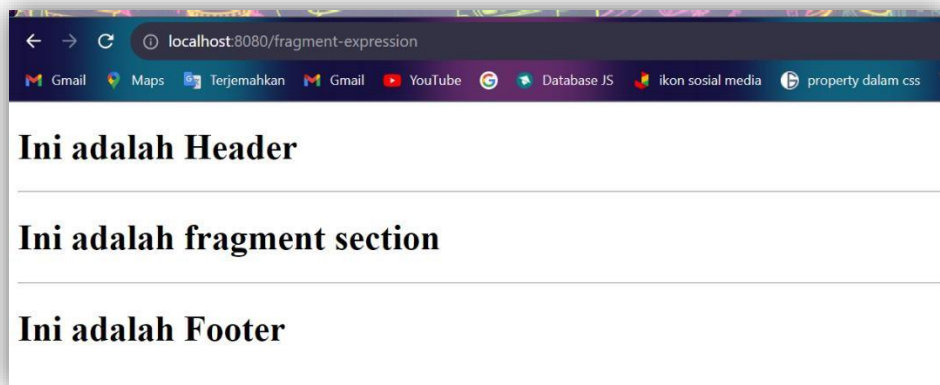
Yang terakhir sebelum menjalankan program fragment expression ini buatlah method yang nantinya merujuk pada file `fragment-expression.html` di dalam file `LaptopController.java`, cukup masukan kode berikut:

```
@GetMapping("fragment-expression")
public String fragmentExpression() {
    return "fragment-expression";
}
```

Untuk lebih jelas dalam penggunaan fragment expression ini, kalian boleh akses link dibawah ini :

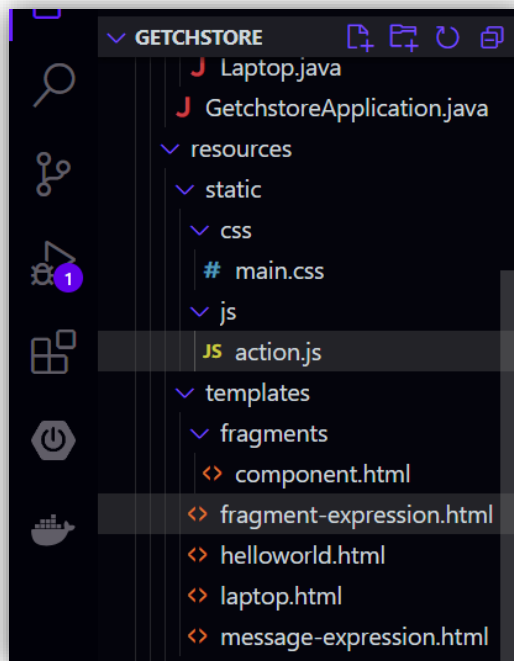
<https://attacomsian-com.translate.qooq/blog/thymeleaf-fragments? x tr sl=en& x tr tl=id& x tr hl=id& x tr pto=tc#thinsert-- threplace-attributes>

Hasilnya :



9.5 Cara menambahkan CSS dan JS Kedalam Thymeleaf

Pertama buatlah terlebih dahulu sebuah folder `css` dan `js` dalam folder `static`, kemudian dalam folder `css` buatlah file `main.css` dan di folder `js` buatlah file `actions.js`. Jadi untuk gambaran struktur folder nya seperti dibawah ini :



Setelah membuat file `main.css` dan `actions.js`, selanjutnya ialah mengisi `main.css` dengan css yang diinginkan, sedangkan untuk `actions.js` bisa diisi javascript sesuai selera kalian masing-masing. Kali ini file `laptop.html` yang akan diberi css dan js. Untuk isi css dan js nya sebagai berikut :

main.css :

```
h1 a {
  color: red;
}
```

```
table tr td {
  padding: 16px;
  color: blue;
}
```

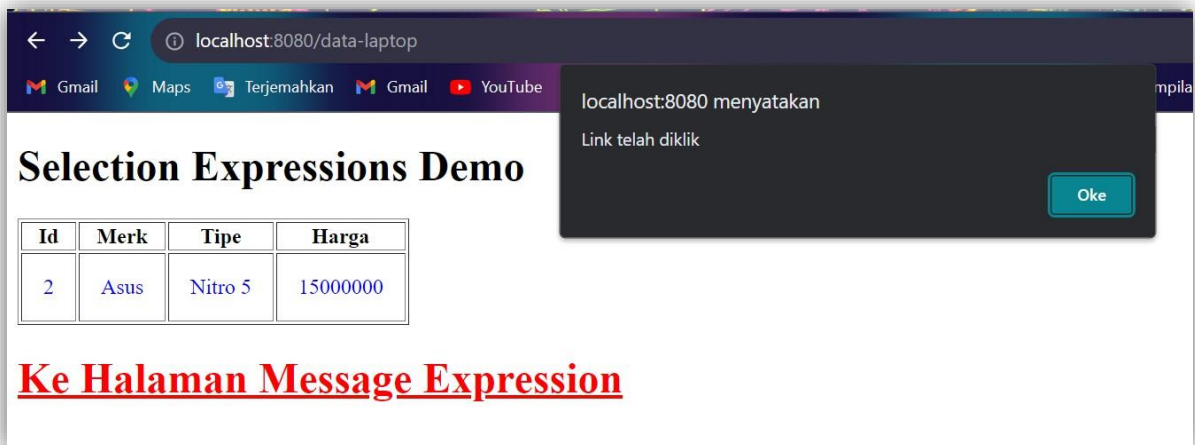
action.js :

```
function showAlert() {
  alert("Link telah diklik");
}
```

Kemudian dibagian head tambahkan kode berikut agar css dan js bisa dipakai di file laptop.html :

```
<link rel="stylesheet" th:href="@{/css/main.css}" />
<script th:src="@{/js/action.js}"></script>
```

Dan di file laptop.html di tag a yang isi konten nya **Ke Halaman Message Expression** tambahkan atribut `th:onclick="showAlert()"` dan hasilnya apabila dijalankan akan seperti dibawah ini :



9.6 Penggunaan If Else dan Switch Case

Thymeleaf mendukung penggunaan if else dan switch case untuk membuat logika kondisional dalam template. Untuk menggunakan if else dalam Thymeleaf, kita dapat menggunakan tag `th:if` dan `th:unless`. Tag `th:if` akan menampilkan kontennya jika kondisinya benar, sedangkan tag `th:unless` akan menampilkan kontennya jika kondisinya salah.

Contoh dasar sintak penggunaan if else :

```
<div th:if="${condition}">
  <p>TRUE</p>
</div>
<div th:unless="${condition}">
  <p>FALSE</p>
</div>
```

Di mana `condition` adalah expression yang akan diperiksa. Jika kondisi tersebut benar, maka blok kode di dalam tag `th:if` akan dijalankan. Jika kondisi tersebut salah, maka blok kode di dalam tag `th:unless` akan dijalankan.

Untuk penerapannya dalam suatu program, pertama buat file `User.java` dalam folder `models` dan masukan kode program dibawah ini :

```
package com.thymeleaf.getchstore.models;

public class User {

    private int id;
    private String name;
    private int age;
    private String role;

    public User() {
    }
}
```

```
public User(int id, String name, int age, String role) {  
  
    this.id    =    id;  
    this.name  = name;  
    this.age   =  age;  
    this.role  = role;  
  
}  
  
public int getId() {  
  
    return id;  
  
}  
  
public void setId(int id) {  
  
    this.id = id;  
  
}  
  
public String getName() {  
  
    return name;  
  
}  
  
public void setName(String name) {  
  
    this.name = name;  
  
}  
  
public int getAge() {  
  
    return age;  
  
}
```

```

    public void setAge(int age) {

        this.age = age;
    }

    public String getRole() {

        return role;
    }

    public void setRole(String role) {

        this.role = role;
    }

}

```

Selanjutnya buatlah file `UserController.java` dalam folder `controllers` dan masukan kode program berikut ini :

```

package com.thymeleaf.getchstore.controllers;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.thymeleaf.getchstore.models.User;

@Controller
public class UserController {

    @GetMapping("users")

```

```

    public String listUsers(Model model) {

        List<User> users = new ArrayList<>();

        users.add(new User(1, "WAHYU", 20, "ADMIN"));

        users.add(new User(2, "JEBREDZ", 22, "USER"));
        users.add(new User(3, "WABREDZ", 21, "ADMIN"));
        model.addAttribute("listUsers", users);
        return "user";

    }

}

```

Dan yang terakhir ialah membuat file html untuk menampilkan data yang sudah dibuat sebelumnya. Jadi buatlah file `user.html` dan masukan kode program berikut ini:

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
    <head>
        <title>User</title>
        <link
            href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.c
            ss "
            rel="stylesheet"
            integrity="sha3-
            84-
            T3c6CoIi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEVDwWykc2MPK8M2
            H N"
            crossorigin="anonymous"
        />
    </head>
    <body>
        <table class="table table-striped table-hover w-50 m-4 caption-top">
            <caption>
                List User
            </caption>
            <thead class="table-dark">
                <tr>
                    <th scope="col">ID</th>
                    <th scope="col">Name</th>
                    <th scope="col">Age</th>
                    <th scope="col">Role</th>

```

```

        <th scope="col">Action</th>
    </tr>
</thead>
<tbody class="table-group-divider">
<tr th:each="user : ${listUsers}">
    <td th:text="${user.id}"></td>
    <td th:text="${user.name}"></td>
    <td th:text="${user.age}"></td>
    <td th:text="${user.role}"></td>
    <td>
        <a class="btn btn-primary" th:if="${user.role} == 'ADMIN'"
            >Update</a>
        <br>
        <a class="btn btn-danger" th:if="${user.role} ==
'ADMIN'">Delete</a>
        <a class="btn btn-primary" th:unless="${user.role} == 'ADMIN'"
            >View</a>
    </td>
</tr>
</tbody>
</table>
</body>
</html>

```

Pada program diatas terdapat atribut `th:if="${user.role} == 'ADMIN'"` yang mana artinya ialah jika kondisi role dari suatu user "ADMIN" maka akan menampilkan button update dan button delete. Dan selain atribut `th:if="${user.role} == 'ADMIN'"`, adapun atribut `th:unless="${user.role} == 'ADMIN'"` yang artinya jika role dari user bukan "ADMIN" maka hanya akan menampilkan button view saja.

Nah sekarang coba masih menggunakan program diatas cuman penggunaannya kita ganti yang tadinya If Else menjadi Switch Case. Cukup ganti kode program yang ada di dalam tag table menjadi seperti dibawah ini :

```

<table class="table table-striped table-hover w-50 m-4 caption-top">
    <caption>
        List User
    </caption>
    <thead class="table-dark">
        <tr>
            <th scope="col">ID</th>
            <th scope="col">Name</th>
            <th scope="col">Age</th>
            <th scope="col">Role</th>
            <th scope="col">Greeting</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>John</td>
            <td>25</td>
            <td>ADMIN</td>
            <td>Hello</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Jane</td>
            <td>30</td>
            <td>USER</td>
            <td>Hi</td>
        </tr>
    </tbody>
</table>

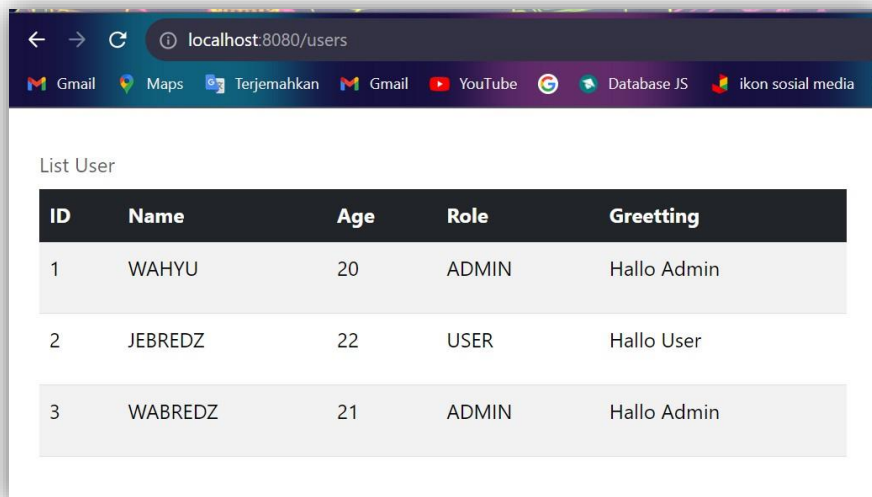
```

```

</thead>
<tbody class="table-group-divider">
<tr th:each="user : ${listUsers}">
  <td th:text="${user.id}"></td>
  <td th:text="${user.name}"></td>
  <td th:text="${user.age}"></td>
  <td th:text="${user.role}"></td>
  <td th:switch="${user.role}">
    <p th:case="'ADMIN'">Hallo Admin</p>
    <p th:case="'USER'">Hallo User</p>
    <p th:case="*">Hallo Guest</p>
  </td>
</tr>
</tbody>
</table>

```

Hasilnya :



List User

ID	Name	Age	Role	Greeting
1	WAHYU	20	ADMIN	Hallo Admin
2	JEBREDZ	22	USER	Hallo User
3	WABREDZ	21	ADMIN	Hallo Admin

9.7 Perulangan Dalam Thymeleaf

Di Thymeleaf, perulangan menggunakan atribut `th:each`. Thymeleaf `th:each` memungkinkan untuk mengulang sebuah variabel. Perulangan ini memiliki dua parameter, yaitu:

- `item` : Variabel yang akan digunakan untuk menyimpan data dari setiap perulangan.
- `collection` : Variabel yang berisi data yang akan diulang.

Sintaks dasar perulangan `th:each` :

```

<ul th:each="item : ${collection}">
  <li th:text="${item}"></li>
</ul>

```

Contoh lengkap penggunaan `th:each` untuk melakukan perulangan sebuah variabel, dalam contoh ini kita menggunakan sebuah array biasa. Pertama buat terlebih dahulu sebuah method dalam file `LaptopController.java`, nama method `dataMerk` serta untuk endpoint nya gunakan `"data-merk"`. Untuk kode lengkap dari method `dataMerk` adalah sebagai berikut :

```
@GetMapping("data-merk")

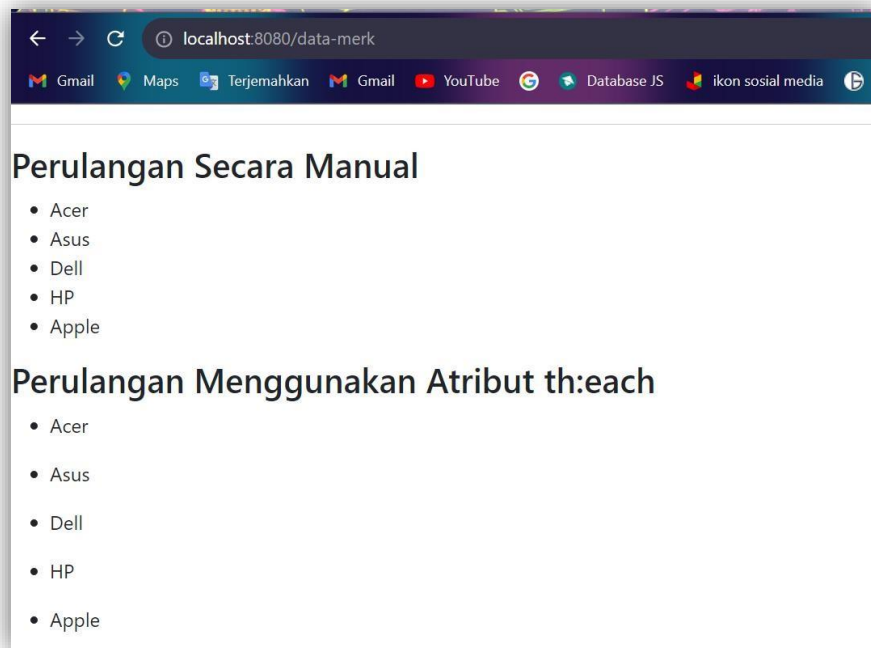
public String dataMerk(Model model) {

    String[] merk = { "Acer", "Asus", "Dell", "HP", "Lenovo", "Apple", "MSI",
    "Razer", "Microsoft", "Toshiba" };
    model.addAttribute("merk", merk);
}
```

Kemudian dalam file `laptop.html` tambahkan kode dibawah ini :

```
<!-- Perulangan Manual -->
<h3>Perulangan Secara Manual</h3>
<ul>
    <li th:text="${merk[0]}"></li>
    <li th:text="${merk[1]}"></li>
    <li th:text="${merk[2]}"></li>
    <li th:text="${merk[3]}"></li>
    <li th:text="${merk[4]}"></li>
</ul>
<!-- Perulangan Menggunakan Atribut th:each -->
<h3>Perulangan Menggunakan Atribut th:each</h3>
<ul th:each="m : ${merk}">
    <li th:text="${m}"></li>
</ul>
```

Hasilnya :



Sebelumnya sudah dijelaskan untuk mengulangi sebuah kumpulan objek menggunakan Thymeleaf melalui atribut `th:each`, dan untuk melakukan perulangan ini bukan hanya sebuah objek saja namun seperti list atau array tetap bisa.

BAB X

MENGGUNAKAN JPA DENGAN THYMELEAF DI JAVA

10.1 Pengenalan JPA

10.1.1 Definisi JPA (Java Persistence API)

Java Persistence API (JPA) adalah spesifikasi Java untuk manajemen persistensi data dalam aplikasi Java Enterprise. JPA adalah bagian dari Jakarta EE (sebelumnya dikenal sebagai Java EE) dan merupakan cara standar untuk berinteraksi dengan database dalam aplikasi Java. JPA menyediakan abstraksi tingkat tinggi yang memungkinkan pengembang untuk bekerja dengan database menggunakan objek Java daripada SQL murni.

Dengan kata lain, JPA adalah spesifikasi Java yang memungkinkan aplikasi Java untuk berinteraksi dengan database menggunakan objek Java, menggantikan penggunaan SQL murni, dan digunakan dalam aplikasi Java Enterprise.

Note: Spesifikasi adalah dokumen atau set aturan dan definisi yang merinci cara sesuatu seharusnya berfungsi atau diimplementasikan

10.1.2 Perbedaan antara JDBC dan JPA:

- a. JDBC (Java Database Connectivity) adalah API Java yang digunakan untuk berinteraksi dengan database secara langsung dengan mengirimkan perintah SQL. JDBC **membutuhkan penulisan kode SQL** dan dapat memakan waktu.
- b. JPA, di sisi lain, adalah spesifikasi yang lebih tinggi dan abstrak yang memungkinkan pengembang untuk **bekerja dengan objek Java yang mewakili data dalam database**. Ini mengurangi penulisan SQL dan menyediakan cara yang lebih mudah dan efisien untuk mengakses dan memanipulasi data.

10.1.3 Entity dan Entity Class

- a. Entity dalam JPA

Entity adalah objek Java yang merepresentasikan data dalam database. Setiap entitas mewakili entitas dalam database dan memiliki properti yang sesuai dengan kolom dalam tabel.

- b. Entity Class adalah kelas Java yang dianotasi dengan `@Entity` untuk menandakan bahwa itu adalah entitas dalam JPA.
- c. Properti dalam kelas ini sesuai dengan kolom dalam tabel yang diwakilinya.
- d. Primary key dari tabel dapat ditandai dengan anotasi `@Id`.

10.1.4 Anotasi JPA untuk Entity Class:

- `@Entity`: Menandakan bahwa kelas adalah entitas dalam JPA.
- `@Id`: Menandai properti sebagai primary key.
- `@GeneratedValue`: Mengatur cara nilai primary key yang dihasilkan (`AUTO`, `IDENTITY`, dll.).
- `@Table`: Menghubungkan entitas dengan tabel dalam database.
- `@Column`: Mengonfigurasi properti entitas ke kolom dalam tabel.

10.2 Konfigurasi Spring Boot dengan JPA

10.2.1 Mengatur sumber data (Data Source):

Data Source adalah komponen yang menghubungkan aplikasi Spring Boot dengan database. Anda dapat mengkonfigurasi Data Source dalam file `application.properties` atau `application.yml`. Berikut adalah contoh konfigurasi database properties dalam `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=username
spring.datasource.password=password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Anda perlu mengganti URL, username, password, dan driver-class-name sesuai dengan pengaturan database Anda.

10.2.2 Konfigurasi Entity Manager Factory

`EntityManagerFactory` adalah komponen yang mengelola entitas dan koneksi database. Anda dapat mengkonfigurasi `EntityManagerFactory` dalam file `application.properties` atau dengan menggunakan Java Configuration. Berikut adalah Contoh konfigurasinya

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

Keterangan :

- `spring.jpa.hibernate.ddl-auto=update` akan mengizinkan Hibernate membuat tabel jika belum ada atau memperbarui skema jika diperlukan.
- `spring.jpa.properties.hibernate.dialect` menentukan dialek Hibernate yang sesuai dengan database yang digunakan.

10.2.3 Repository Interface

Repository Interface adalah komponen yang digunakan untuk berinteraksi dengan entitas dalam database. Repository Interface dapat didefinisikan dengan meng-extend `JpaRepository` atau `CrudRepository` dari Spring Data JPA. Berikut adalah contoh pembuatan Repository Interface:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

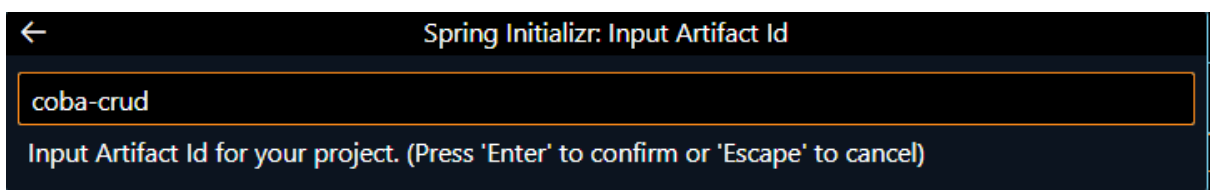
}
```

Ini adalah langkah-langkah dasar untuk mengkonfigurasi Spring Boot dengan JPA. Dengan mengatur Data Source, EntityManagerFactory, dan Repository Interface, Anda dapat mulai mengakses dan memanipulasi data dalam database menggunakan JPA dalam aplikasi Spring Boot.

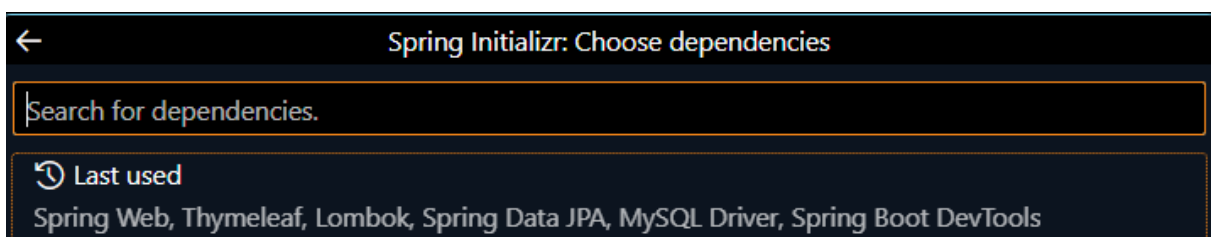
10.3 Membuat Project CRUD Dengan JPA

10.3.1 Membuat Project

Buatlah project SpringBoot Maven dengan nama *coba-crud* :



Tambahkan dependensi yang diperlukan :

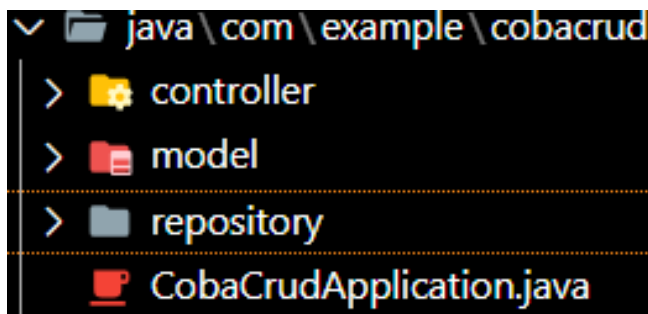


Berikut adalah penjelasan singkat mengenai kegunaan dari masing-masing dependensi yang umumnya digunakan dalam proyek Spring Boot:

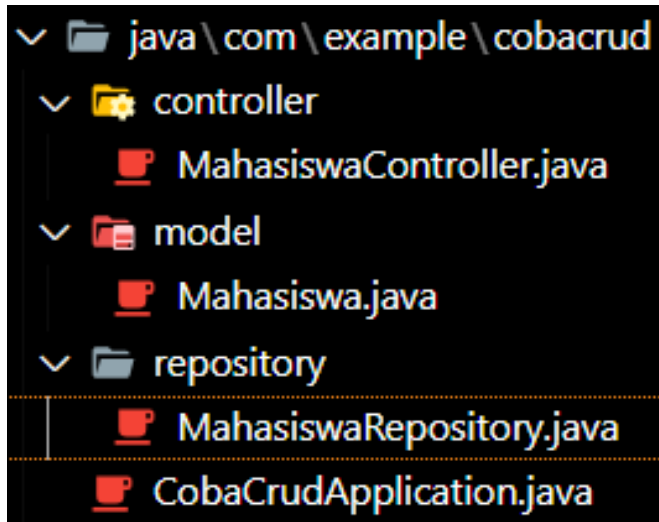
- Spring Web:
 - a. Dependensi Spring Web menyediakan dukungan untuk pengembangan aplikasi web menggunakan Spring.
 - b. Ini mencakup fitur seperti manajemen permintaan HTTP, penanganan RESTful API, dan kemampuan untuk membuat controller dan view web.
 - c. Spring Web memungkinkan Anda untuk dengan mudah membangun aplikasi web dengan Spring Boot.
- Thymeleaf:
 - a. Thymeleaf adalah mesin template Java yang digunakan untuk membuat tampilan web dalam aplikasi Spring Boot.
 - b. Ini memungkinkan pengembang untuk memasukkan data dari model ke dalam tampilan HTML dengan cara yang dekat dengan HTML asli.
 - c. Thymeleaf sangat cocok untuk pengembangan aplikasi web berbasis Java dan sangat terintegrasi dengan Spring Boot.
- Lombok:
 - a. Lombok adalah sebuah framework yang memungkinkan pengembang untuk mengurangi penulisan kode boilerplate dalam Java.
 - b. Dengan Lombok, Anda dapat menggunakan anotasi seperti `@Data`, `@Getter`, `@Setter`, dan lainnya untuk secara otomatis menghasilkan metode `getter`, `setter`, `toString`, `hashCode`, dan `equals`, yang mengurangi penulisan kode berulang.
 - c. Ini mempermudah pengembangan aplikasi dengan mengurangi jumlah kode yang perlu ditulis.
- Spring Data JPA:
 - a. Dependensi Spring Data JPA adalah bagian dari proyek Spring Data yang menyediakan dukungan untuk JPA (Java Persistence API) dalam aplikasi Spring.
 - b. Ini memungkinkan pengembang untuk dengan mudah mengakses dan memanipulasi data dalam database menggunakan entitas Java dan Repository Interface.

- c. Spring Data JPA mengurangi penulisan kode boilerplate yang terkait dengan operasi database.
- MySQL Driver:
 - a. MySQL Driver adalah ketergantungan yang diperlukan untuk menghubungkan aplikasi Spring Boot dengan database MySQL.
 - b. Ini adalah implementasi driver JDBC yang digunakan oleh aplikasi Spring Boot untuk berkomunikasi dengan server database MySQL.
- Spring Boot DevTools:
 - a. Spring Boot DevTools adalah dependensi yang digunakan selama pengembangan dan pengujian aplikasi Spring Boot.
 - b. Ini menyediakan alat yang membantu dalam pengembangan seperti restart otomatis aplikasi saat kode berubah, pemantauan kode sumber, dan pemantauan perubahan properti aplikasi.
 - c. Ini dapat mempercepat siklus pengembangan dan memudahkan debugging aplikasi.

Setelah menambahkan dependensi diatas dan membuat project, buka project tersebut lalu buat package/folder **controller**, **model**, dan **repository** :



Kemudian buat file untuk masing-masing package sebagai berikut :



Project untuk persiapan membuat aplikasi CRUD sederhana telah dibuat.

10.3.2 Mengimplementasikan konsep CRUD menggunakan JPA di SpringBoot

1. Bagian Membuat data (CREATE) :

Tambahkan kode berikut ke file **Mahasiswa.java** :

```
package com.example.cobacrud.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Entity
@Data
public class Mahasiswa {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Integer id;
    String nim;
    String nama;
}
```

Keterangan :

- **@Entity**, digunakan untuk menandai bahwa kelas **Mahasiswa** adalah sebuah entitas JPA. Ini mengindikasikan bahwa kelas ini akan dipetakan ke sebuah tabel dalam database.

- **@Data**, adalah anotasi khusus yang menghasilkan boilerplate code seperti getter, x, **equals**, **hashCode**, dan **toString** secara otomatis. Dengan anotasi **@Data**, Anda tidak perlu menuliskan metode-metode tersebut secara manual.
- **@Id** digunakan untuk menandai bahwa properti **id** adalah primary key dari entitas **Mahasiswa**.
- **@GeneratedValue** digunakan bersamaan dengan **@Id** untuk mengonfigurasi bagaimana nilai primary key akan di-generate.
- **strategy = GenerationType.IDENTITY** menunjukkan bahwa nilai primary key akan di-generate secara otomatis oleh database dengan menggunakan tipe data yang mendukung auto-increment, seperti **AUTO_INCREMENT** pada MySQL.

Kemudian tambahkan kode berikut di file **MahasiswaRepository.java** jangan lupa bahwa repository tersebut berupa interface dan bukan *class* :

```
package com.example.cobacrud.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import com.example.cobacrud.model.Mahasiswa;

public interface MahasiswaRepository
    extends JpaRepository <Mahasiswa, Integer> {

}
```

Dengan adanya **MahasiswaRepository**, kita dapat menggunakan metode yang disediakan oleh **JpaRepository** (seperti **save**, **findById**, **findAll**, **delete**, dll.) untuk melakukan operasi CRUD pada entitas **Mahasiswa** dalam database tanpa perlu menulis implementasi metode-metode tersebut secara manual. Repository ini memungkinkan kita untuk secara mudah berinteraksi dengan database dan mengelola data entitas `Mahasiswa` Anda dalam aplikasi Spring Boot.

Kemudian tambahkan kode berikut pada *MahasiswaController.java* :

```
package com.example.cobacrud.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
```

```

import org.springframework.web.bind.annotation.PostMapping;
import com.example.cobacrud.model.Mahasiswa;
import com.example.cobacrud.repository.MahasiswaRepository;

@Controller
public class MahasiswaController {

    @Autowired
    private MahasiswaRepository mahasiswaRepository;

    @GetMapping("/add-mahasiswa")
    public String addMahasiswa(Model model) {
        Mahasiswa mahasiswa = new Mahasiswa();
        model.addAttribute("mahasiswa", mahasiswa);
        return "addMahasiswa";
    }

    @PostMapping("/save-mahasiswa")
    public String saveMahasiswa (@ModelAttribute("mahasiswa")
                                Mahasiswa mahasiswa) {
        mahasiswaRepository.save(mahasiswa);
        return "redirect:/add-mahasiswa";
    }
}

```

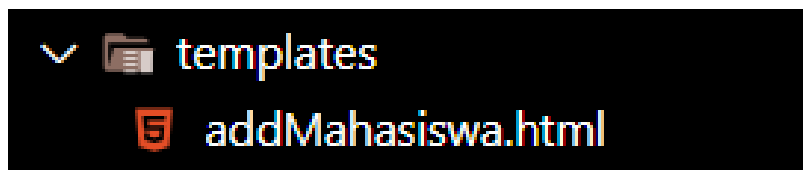
Keterangan :

- @Controller adalah anotasi yang menandakan bahwa kelas `MahasiswaController` adalah komponen Spring dan berperan sebagai controller dalam aplikasi. Controller ini akan menangani permintaan HTTP terkait operasi tambah data mahasiswa.
- @Autowired digunakan untuk melakukan injeksi ketergantungan. Dalam hal ini, `MahasiswaRepository` akan di-injeksi ke dalam controller untuk digunakan.
- @GetMapping("/add-mahasiswa") anotasi ini mengindikasikan bahwa metode `addMahasiswa` akan menangani permintaan HTTP GET ke URL `/add-mahasiswa`. Metode ini digunakan untuk menampilkan halaman tambah mahasiswa.
- public String addMahasiswa(Model model) Metode ini menerima objek `Model` sebagai parameter, yang digunakan untuk mengirim data ke tampilan. Di dalam metode, kita membuat objek `Mahasiswa` kosong yang akan digunakan untuk mengisi formulir tambah mahasiswa. Kemudian, objek `mahasiswa` tersebut ditambahkan ke model dengan nama "mahasiswa". Metode ini mengembalikan nama tampilan "addMahasiswa".

- `@PostMapping("/save-mahasiswa")` Anotasi ini mengindikasikan bahwa metode `saveMahasiswa` akan menangani permintaan HTTP POST ke URL `/save-mahasiswa`. Metode ini digunakan untuk menyimpan data mahasiswa yang telah diisi pada formulir.
- `public String saveMahasiswa(@ModelAttribute("mahasiswa") Mahasiswa mahasiswa)`
Metode ini menerima objek `Mahasiswa` yang diisi dari formulir (dengan menggunakan anotasi `@ModelAttribute`) sebagai parameter. Kemudian, objek `mahasiswa` ini disimpan ke dalam database menggunakan `mahasiswaRepository.save(mahasiswa)`. Setelah penyimpanan data berhasil, metode ini mengembalikan redirect ke URL `/add-mahasiswa`, sehingga pengguna dapat kembali ke halaman tambah data mahasiswa.

Seluruh logika ini memungkinkan pengguna untuk mengisi formulir dengan data mahasiswa baru, dan data tersebut akan disimpan ke dalam database MySQL melalui Spring Data JPA ketika pengguna mengirimkan formulir. Setelah penyimpanan berhasil, pengguna akan diarahkan kembali ke halaman tambah mahasiswa.

Kemudian Buat `addMahasiswa.html` untuk tampilan HTML nya:

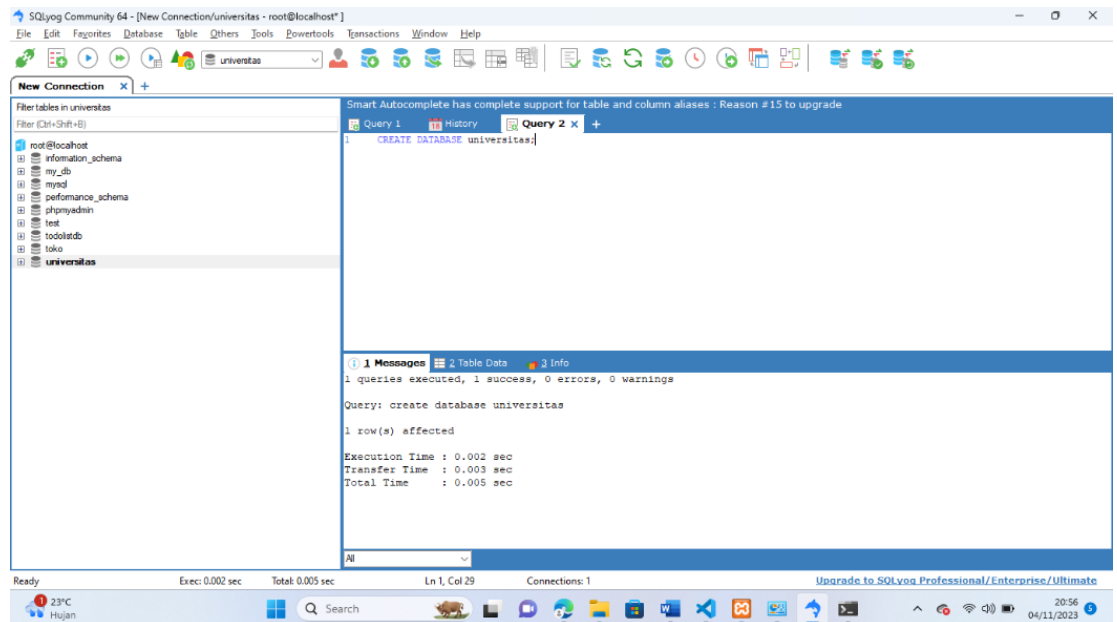


Isi `addMahasiswa.html` dengan kode berikut :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Tambah Mahasiswa</title>
  </head>

  <body>
    <h1>Add Mahasiswa</h1>
    <form th:action="@{/save-mahasiswa}" method="post">
      <input type="text" th:field="${mahasiswa.nim}">
      <input type="text" th:field="${mahasiswa.nama}">
      <button type="submit">Save Mahasiswa</button>
    </form>
  </body>
</html>
```

Kemudian buat database dengan nama *universitas* di MySql :



Lalu tambahkan kode berikut di **application.properties** :

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost:3306/universitas
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect
.MySQLDialect
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type=TRACE
```

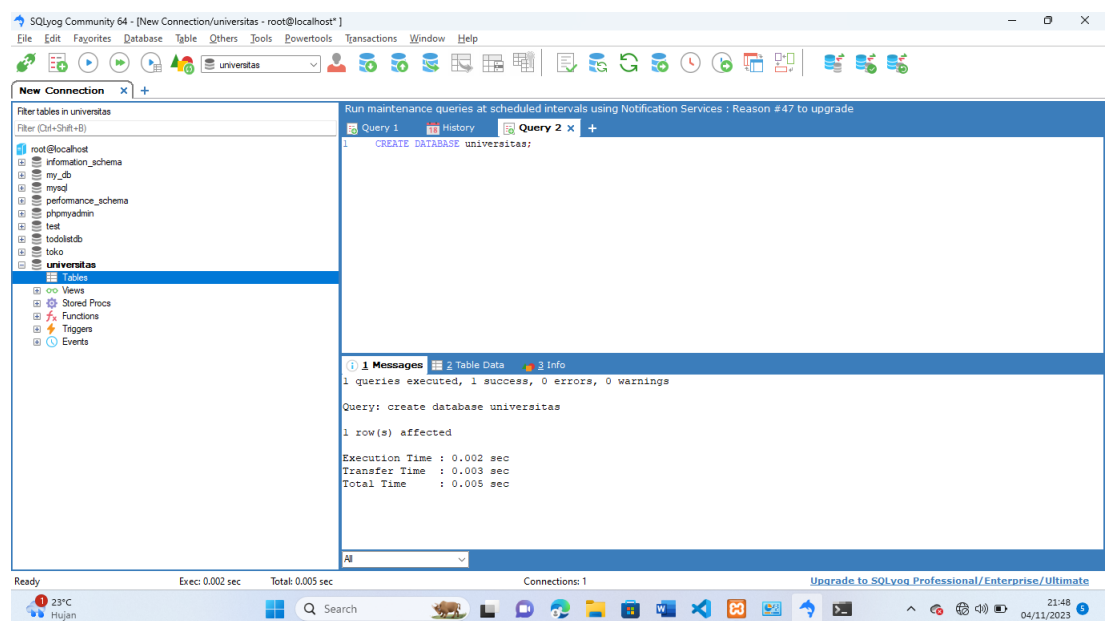
Keterangan :

- **spring.jpa.hibernate.ddl-auto=update** Properti ini mengontrol perilaku Hibernate DDL (Data Definition Language) auto-generate. Nilai **update** akan membuat Hibernate menghasilkan tabel yang sesuai dengan definisi entitas Java Anda dan memperbarui skema database jika diperlukan tanpa menghapus data yang ada.
- **spring.datasource.url=jdbc:mysql://localhost:3306/universitas** Ini adalah URL koneksi JDBC ke database MySQL.
- **spring.datasource.username=root** Ini adalah nama pengguna MySQL yang digunakan untuk mengakses database. Dalam contoh ini, nama pengguna adalah "root."
- **spring.datasource.password=** Ini adalah kata sandi pengguna MySQL yang digunakan untuk mengakses database. Dalam contoh ini, kata sandi dikosongkan (kosong).

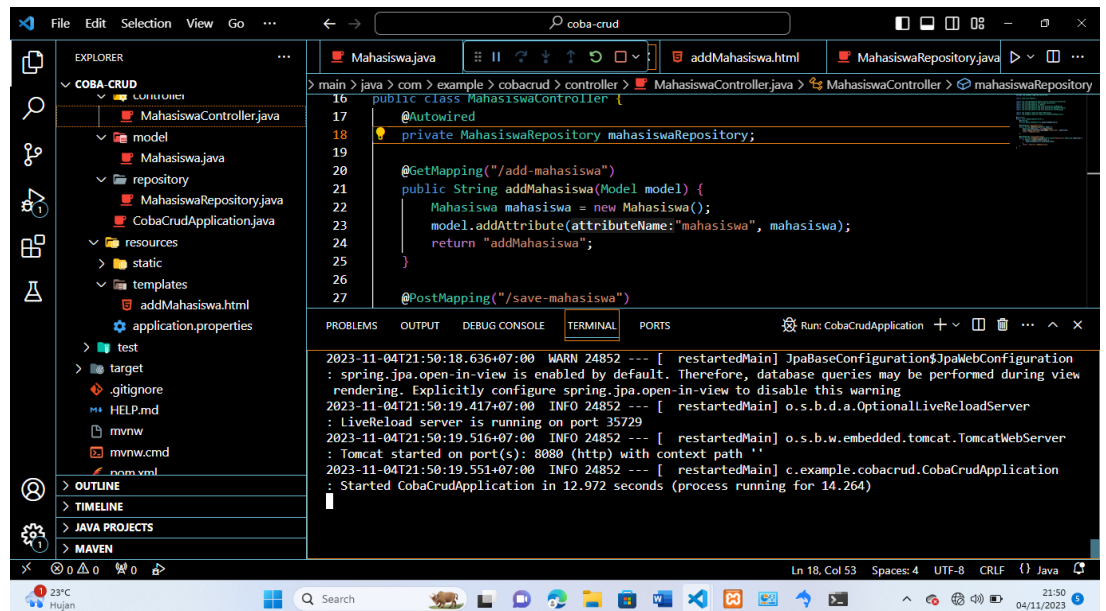
- **spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect** Properti ini mengonfigurasi dialek Hibernate yang sesuai dengan database yang digunakan.
- **logging.level.org.hibernate.SQL=DEBUG** Properti ini mengonfigurasi tingkat log untuk SQL statements yang dihasilkan oleh Hibernate. Di sini, tingkat log diatur ke "DEBUG," sehingga pernyataan SQL akan dicetak ke log dalam tingkat debug.
- **logging.level.org.hibernate.type=TRACE**

Properti ini mengonfigurasi tingkat log untuk jenis Hibernate. Di sini, tingkat log diatur ke **TRACE** sehingga informasi detail tentang jenis Hibernate akan dicetak ke log. Konfigurasi ini digunakan untuk mengatur koneksi ke database MySQL, mengatur perilaku Hibernate, dan mengontrol tingkat log yang dicetak selama proses interaksi dengan database dalam proyek Spring Boot yang menggunakan Spring Data JPA.

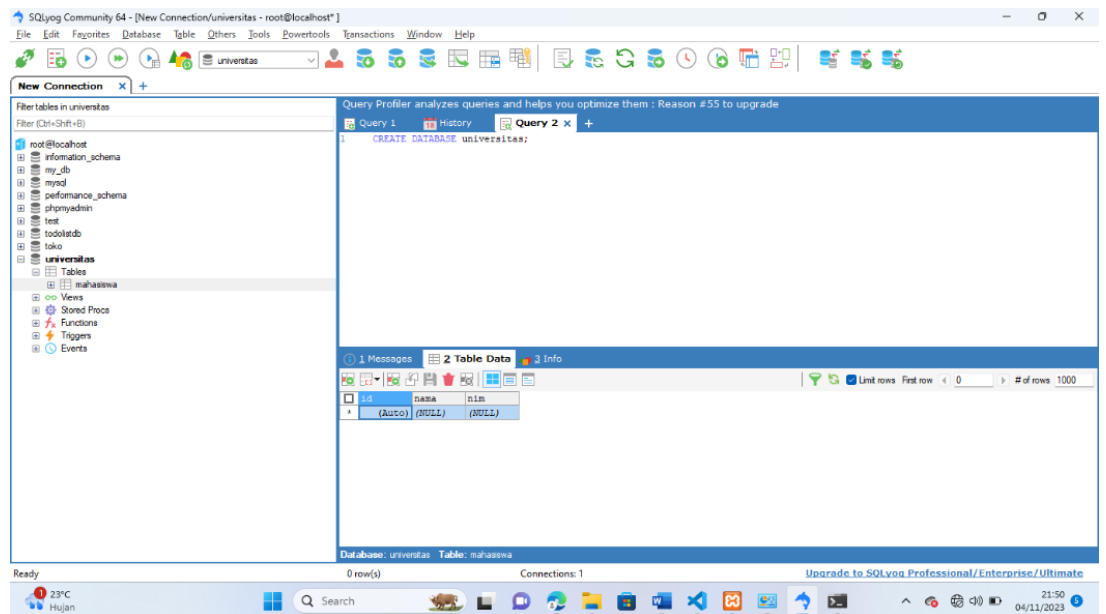
Pada mulanya database tersebut tidak akan memiliki table apapun



Namun setelah kita melakukan run java maka akan muncul semua model yang telah diberi **@Entity** menjadi sebuah table di database



Hasilnya adalah sebagai berikut :



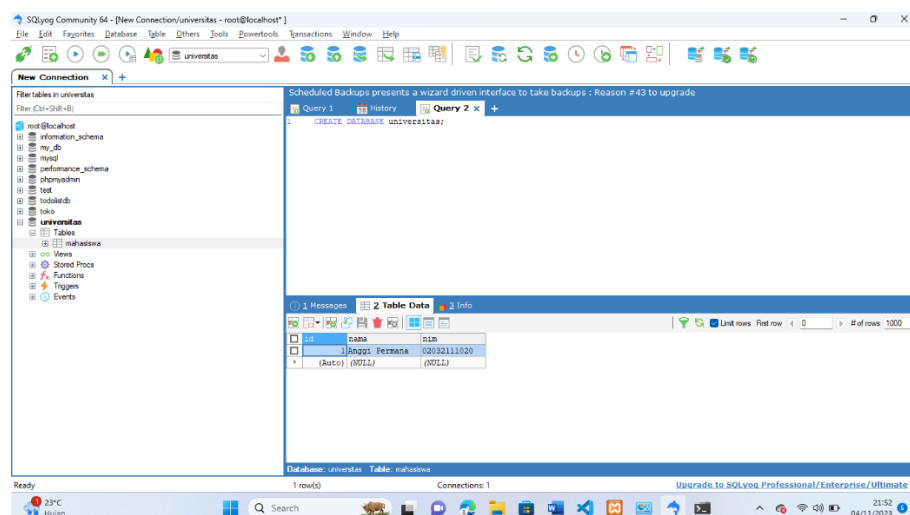
Namun table mahasiswa masih kosong atau belum memiliki data, kita coba tambah data melalui web browser, caranya akses <http://localhost:8080/add-mahasiswa> , maka tampilannya adalah sebagai berikut :



Lalu kita coba inputkan data nim dan namanya :



Setelah tekan button Save Mahasiswa maka inputan akan kosong Kembali seperti tampilan awal, lalu kita coba kembali ke SqlYog dan refresh table mahasiswa tersebut, maka data akan masuk ke table mahasiswa :



Kita berhasil mengimplementasikan konsep CREATE dalam CRUD pada database menggunakan JPA SpringBoot.

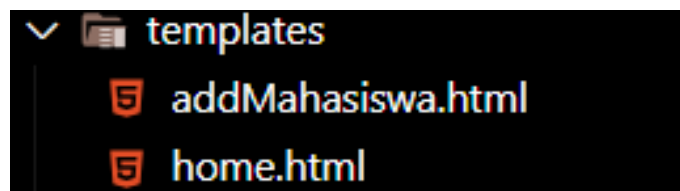
2. Bagian Membaca Data (READ)

Setelah kita berhasil membuat data dan menambahkannya ke dalam table, kini saatnya kita mengambil data tersebut dan menampilkannya melalui web browser.

Tambahkan kode berikut di *MahasiswaController.java* :

```
@GetMapping("/")
public String allMahasiswa(Model model) {
    model.addAttribute("allMahasiswa",
                       mahasiswaRepository.findAll());
    return "home";
}
```

Kemudian tambah tampilan **home.html** untuk menampilkan semua data mahasiswa dari database :



Isi **home.html** dengan kode berikut :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
    <head>
        <title>Data Mahasiswa</title>
    </head>

    <body>
        <h1>Data Mahasiswa</h1>
        <table border="1">
            <thead>
                <tr>
                    <th>Id</th>
                    <th>Nim</th>
                    <th>Nama</th>
                </tr>
            </thead>
            <tbody>
                <tr th:each="mahasiswa : ${allMahasiswa}">
                    <td th:text="${mahasiswa.id}"></td>
                    <td th:text="${mahasiswa.nim}"></td>
                    <td th:text="${mahasiswa.nama}"></td>
                </tr>
            </tbody>
        </table>
    </body>
</html>
```

```

        </table>
        <a href="/add-mahasiswa">
            <button>Tambah Mahasiswa</button>
        </a>
    </body>
</html>

```

Terakhir, agar setelah menambahkan data mahasiswa baru maka langsung Kembali ke home, maka ubah kode di controller **PostMapping** sebelumnya bagian redirect: kode sebelumnya :

```

@PostMapping("/save-mahasiswa")
public String saveMahasiswa (@ModelAttribute("mahasiswa")
                               Mahasiswa mahasiswa) {
    mahasiswaRepository.save(mahasiswa);
    return "redirect:/add-mahasiswa";
}

```

Kode baru :

```

@PostMapping("/save-mahasiswa")
public String saveMahasiswa(@ModelAttribute("mahasiswa")
                               Mahasiswa mahasiswa) {
    mahasiswaRepository.save(mahasiswa);
    return "redirect:/";
}

```

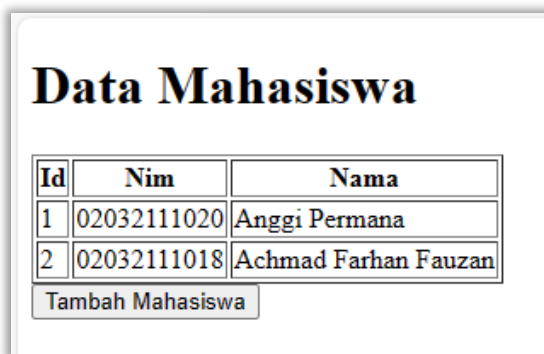
Kemudian jalankan/run project SpringBoot, lalu buka <http://localhost:8080/> maka akan menampilkan tampilan berikut :



Jika kita tekan tombol *Tambah Mahasiswa* maka akan berpindah ke halaman **addMahasiswa.html**, lalu kita coba tambah data baru :



Setelah itu klik tombol Save Mahasiswa maka akan Kembali ke halaman home lagi dan data sudah otomatis bertambah :



Id	Nim	Nama
1	02032111020	Anggi Permana
2	02032111018	Achmad Farhan Fauzan

Tambah Mahasiswa

Dengan demikian kita telah berhasil Menambah (CREATE) dan Menampilkan (READ) data dari table menggunakan JPA.

3. Bagian Mengedit Data (UPDATE)

Selanjutnya kita akan mencoba mengedit data yang sudah ada pada table Mahasiswa di database kita. Pertama, tambahkan kode berikut di **MahasiswaController.java** :

```
@GetMapping("/update-mahasiswa/{id}")
public String updateMahasiswa(@PathVariable(value = "id")
                               Integer id, Model model) {
    Mahasiswa mahasiswa = mahasiswaRepository.getReferenceById(id);
    model.addAttribute("mahasiswa", mahasiswa);
    return "updateMahasiswa";
}
```

Keterangan :

- `@GetMapping("/update-mahasiswa/{id}")` adalah anotasi yang mengindikasikan bahwa metode ini akan menangani permintaan HTTP GET ke URL yang diakhiri dengan `/update-mahasiswa/{id}`. `{id}` adalah variabel path yang digunakan untuk menentukan ID mahasiswa yang akan diperbarui.
- `public String updateMahasiswa(@PathVariable(value = "id") Integer id, Model model)` Metode ini mengambil ID mahasiswa dari variabel path menggunakan anotasi `@PathVariable`. Selain itu, metode menerima objek `Model` sebagai parameter, yang digunakan untuk mengirim data ke tampilan.
- `Mahasiswa mahasiswa = mahasiswaRepository.getReferenceById(id)` Di dalam metode, kita menggunakan repository `mahasiswaRepository` untuk mengambil referensi ke data mahasiswa berdasarkan ID yang diberikan. Fungsi `getReferenceById(id)` digunakan untuk mengambil referensi data mahasiswa. Ini mungkin berbeda dari `findById(id)` karena itu hanya mengambil data yang sudah ada di cache atau di tingkat perusahaan jika digunakan. Hasilnya disimpan dalam objek `mahasiswa`.
- `model.addAttribute("mahasiswa", mahasiswa)` Kemudian, objek `mahasiswa` tersebut ditambahkan ke model dengan nama mahasiswa. Ini memungkinkan data mahasiswa yang akan diperbarui ditampilkan pada halaman yang sesuai.
- `return "updateMahasiswa"` Metode ini mengembalikan nama tampilan `updateMahasiswa`. Halaman ini akan menampilkan data mahasiswa yang akan diperbarui, dan pengguna dapat mengedit informasi mahasiswa pada halaman ini.

Selanjutnya tambahkan file html baru yaitu `updateMahasiswa.html` :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Edit Mahasiswa</title>
  </head>
  <body>
    <h1>Update Mahasiswa</h1>
    <form action="/save-mahasiswa" method="post">
      <input type="hidden" th:field="${mahasiswa.id}">
      <input type="text" th:field="${mahasiswa.nim}">
      <input type="text" th:field="${mahasiswa.nama}">
      <button type="submit">Save Mahasiswa</button>
    </form>
  </body>
```

```
</html>
```

Hampir sama seperti tambah mahasiswa, namun ada tambahan yaitu : `<input type="hidden" th:field="${mahasiswa.id}">` Ini digunakan agar Ketika menyimpan data tidak membuat id baru, dan masih tetap menggunakan id yang lama, sehingga di table mahasiswa tidak bertambah data, melainkan data dengan id yang sama tersebut telah berubah/terupdate.

Terakhir, tambahkan tombol *update* dan *delete* pada *home.html* sehingga menjadi berikut :

```
<table border="1">
  <thead>
    <tr>
      <th>Id</th>
      <th>Nim</th>
      <th>Nama</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="mahasiswa : ${allMahasiswa}">
      <td th:text="${mahasiswa.id}"></td>
      <td th:text="${mahasiswa.nim}"></td>
      <td th:text="${mahasiswa.nama}"></td>
      <td>
        <a
          th:href="@{/update-mahasiswa/{id}
            (id=${mahasiswa.id})}"
        > <button>Update</button>
        </a>
        <a th:href="@{/delete-mahasiswa/{id}
          (id=${mahasiswa.id})}"
        > <button>Delete</button>
        </a>
      </td>
    </tr>
  </tbody>
</table>
```

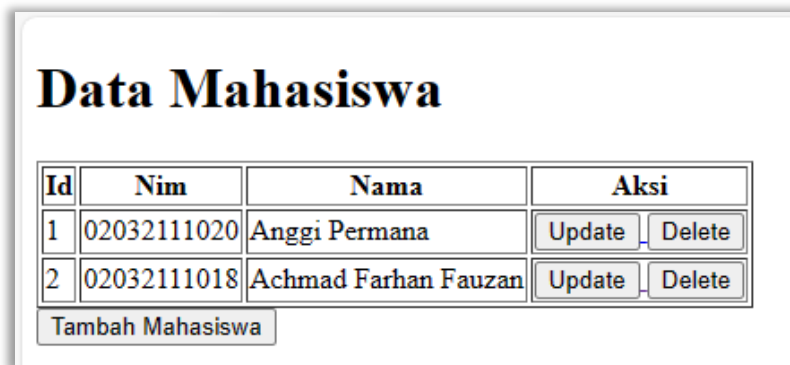
Keterangan :

- `<a th:href="@{/update-mahasiswa/{id}(id=${mahasiswa.id})}">` Ini adalah elemen HTML `<a>` (anchor) yang digunakan untuk membuat tautan (link). Atribut `th:href` adalah atribut Thymeleaf yang digunakan untuk menghasilkan URL tautan. Dalam kasus

ini, itu menghasilkan URL dengan path `/update-mahasiswa/{id}` dimana `{id}` akan digantikan oleh nilai dari properti `id` dari objek `mahasiswa`.

- `(id=${mahasiswa.id})` Ini adalah sintaks Thymeleaf yang digunakan untuk menyisipkan nilai dari properti `id` dari objek `mahasiswa` ke dalam URL. Ini membuat tautan dinamis sesuai dengan ID mahasiswa yang sedang diproses.

Selanjutnya jalankan project dan buka <http://localhost:8080/> maka akan tampil sebagai berikut :



Data Mahasiswa

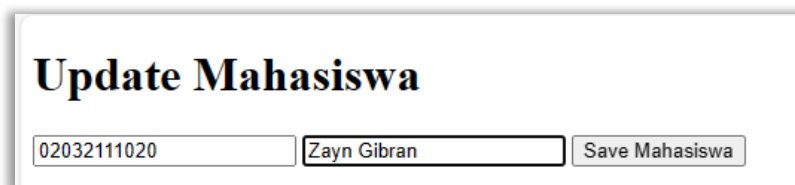
Id	Nim	Nama	Aksi
1	02032111020	Anggi Permana	Update Delete
2	02032111018	Achmad Farhan Fauzan	Update Delete

Jika kita klik tombol **update** yang paling atas pada data pertama, maka akan redirect ke halaman update dan akan menampilkan data yang kita pilih.



Update Mahasiswa

Jika kita mengganti nama nya seperti ini :



Update Mahasiswa

Lalu tekan tombol Save Mahasiswa maka akan redirect ke halaman home dan data akan terupdate.

Data Mahasiswa

Id	Nim	Nama	Aksi	
1	02032111020	Zayn Gibran	Update	Delete
2	02032111018	Achmad Farhan Fauzan	Update	Delete
Tambah Mahasiswa				

4. Bagian Hapus Data (DELETE)

Terakhir dari konsep CRUD adalah menghapus data atau DELETE, kita cukup menambahkan satu controller untuk menghapus data pada **MahasiswaController.java** :

```
@GetMapping("/delete-mahasiswa/{id}")
public String deleteMahasiswa(@PathVariable(value = "id") Integer id)
{
    mahasiswaRepository.deleteById(id);
    return "redirect:/";
}
```

Lalu jalankan project dan buka Kembali url <http://localhost:8080/>.

Data Mahasiswa

Id	Nim	Nama	Aksi	
1	02032111020	Zayn Gibran	Update	Delete
2	02032111018	Achmad Farhan Fauzan	Update	Delete
Tambah Mahasiswa				

Jika kita klik tombol **delete** mahasiswa Achmad Farhan Fauzan, maka akan langsung terhapus.

Data Mahasiswa

Id	Nim	Nama	Aksi	
1	02032111020	Zayn Gibran	Update	Delete
Tambah Mahasiswa				

BAB XI

RELASI ANTAR TABEL (ASSOCIATION MAPPING)

11.1 Association Mapping

Pada pertemuan sebelumnya kita sudah melakukan koneksi ke database dan membuat sebuah entitas. Anggap lah kalian sudah memiliki lebih dari 1 entitas, maka tugas selanjutnya adalah bagaimana cara menghubungkan beberapa entitas tersebut?

Jawabannya adalah dengan menggunakan konsep Association Mapping (Asosiasi Mapping). Asosiasi Mapping ini adalah proses menghubungkan 2 entitas di JPA dengan tujuan untuk menyimpan dan mengambil data dari database secara efisien.

Spring Data JPA Association Mapping



@OneToOne
@OneToMany
@ManyToOne
@ManyToMany

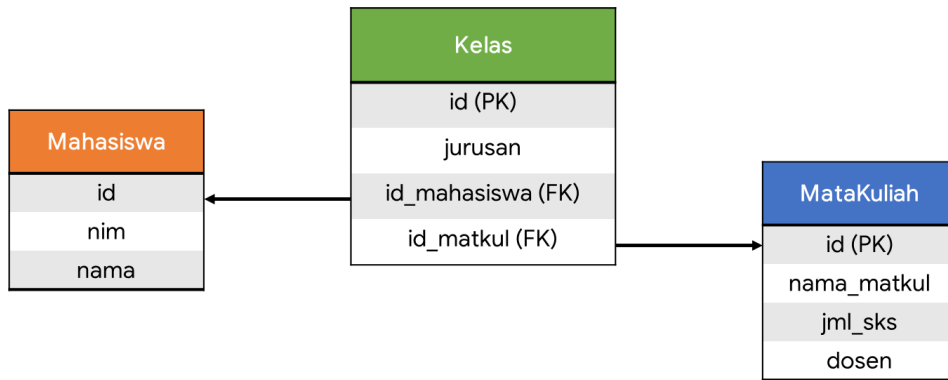
Terdapat 4 macam anotasi yang bisa dilakukan di JPA, yaitu:

- **@OneToOne**, atau satu ke satu
- **@OneToMany**, atau satu ke banyak
- **@ManyToOne**, atau banyak ke satu
- **@ManyToMany**, atau banyak ke banyak

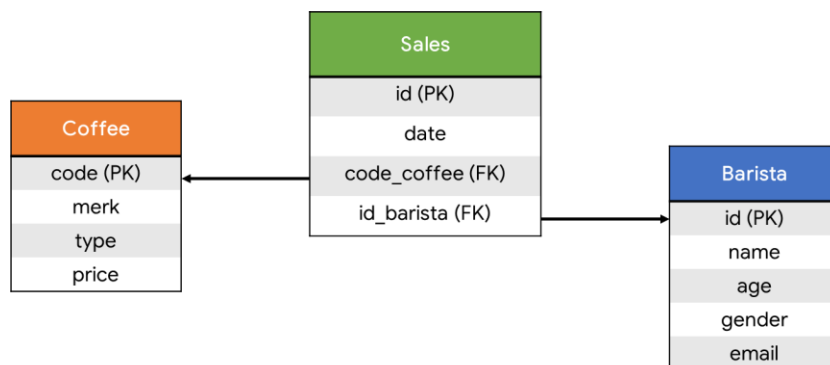
Di pertemuan ini, kita hanya akan membahas anotasi **@ManyToOne** saja, karena fokus kita sekarang adalah menghubungkan beberapa entitas dengan `foreign_key`. Untuk jenis-jenis relasinya akan dibahas pertemuan selanjutnya atau kalian bisa belajar mandiri.

11.2 Implementasi Kode

Misalnya kita memiliki 2 entitas bernama **Mahasiswa** dan **MataKuliah**. Kemudian kita ingin membuat entitas baru yaitu **Kelas** dengan skema database seperti gambar berikut.



Atau misalkan kita memiliki 2 entitas bernama **Coffee** dan **Barista**. Kemudian kita ingin membuat entitas baru yaitu **Sales** dengan skema database seperti gambar berikut.



Seperti biasa, hal yang harus kita lakukan adalah membuat entitasnya terlebih dahulu, kemudian buat repository, lalu buat end-pointnya untuk ditampilkan di view.

11.2.1 Membuat Entity Sales

Cara membuat Sales cukup sederhana, hanya ditambah beberapa kode berikut ini.

```

@Data
@Entity
public class Sales {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String date;

    @ManyToOne
    @JoinColumn(name = "code_coffee", referencedColumnName = "code")
    private Coffee codeCoffee;

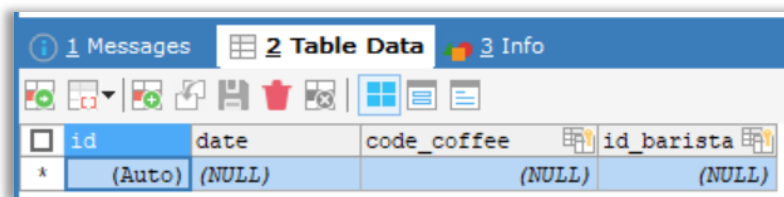
    @ManyToOne
    @JoinColumn(name = "id_barista", referencedColumnName = "id")
    private Barista idBarista;
  
```

```
}
```

Kode di atas tidak beda jauh dengan kode yang biasa kita buat pada pelatihan sebelumnya, hanya ada beberapa baris kode baru dengan keterangan sebagai berikut:

- **@ManyToOne** artinya 1 pesanan bisa memuat banyak kopi, namun 1 kopi hanya bisa dipesan dengan satu pesanan/struk. Begitu juga dengan barista, 1 pesanan bisa dibuat oleh banyak barista, namun 1 barista hanya bisa membuat 1 pesanan.
- **@JoinColumn** artinya kita membuat sebuah foreign_key dengan nama alias/inisial **code_coffee**, kemudian referensi kolomnya merujuk ke kolom **code** yang ada di tabel **coffee**. Begitu juga dengan objek barista.
- **private Coffee codeCoffee** dan **private Barista idBarista** artinya kita membuat sebuah objek dengan tipe data **Coffee** (injeksi dependensi) dengan nama **codeCoffee**. Begitu juga dengan barista. Objek ini bertujuan untuk mengolah dan menyimpan data yang diambil atau disimpan di database.

Jika kode di atas di run, maka akan membuat sebuah tabel di database seperti gambar di bawah ini:



id	date	code_coffee	id_barista
*	(Auto)	(NULL)	(NULL)

11.2.2 Akses Database dengan SalesRepository

Kemudian kita akan membuat sebuah class interface untuk mengakses database menggunakan **JpaRepository**

```
public interface SalesRepository extends JpaRepository<Sales, Integer>{  
}
```

11.2.3 Membuat End-Point dengan SalesController

Setelah kita membuat entitas dan repositori, sekarang kita harus membuat end-point agar data sales bisa kita tampilkan.

```
@Controller  
public class SalesContraller {
```

```

@Autowired
private SalesRepository salesRepository;

@GetMapping("sales")
public String showSales(Model model){
    model.addAttribute("sales", salesRepository.findAll());
    return "show-sales";
}
}

```

11.2.4 Menampilkan Data dengan Thymelaf

Dari kode sebelumnya, method `showSales` mengembalikan nilai string `"show-sales"`. Untuk itu sekarang kita buat sebuah file HTML dengan nama tersebut. Untuk singkatnya buatlah kode berikut:

```

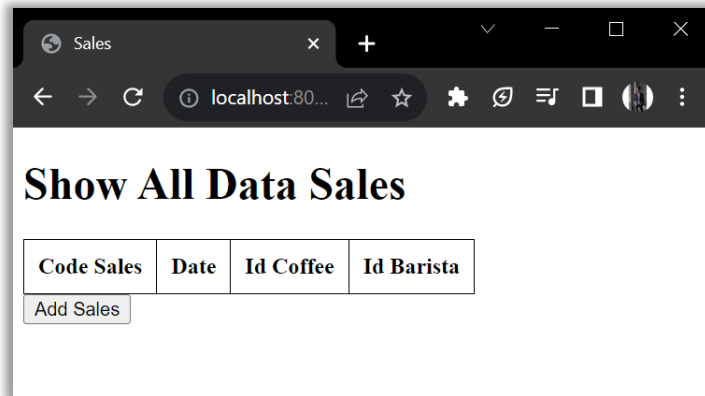
<body>
  <h1>Show All Data Sales</h1>
  <table>
    <tr>
      <th>Code Sales</th>
      <th>Date</th>
      <th>Id Coffee</th>
      <th>Id Barista</th>
    </tr>
    <tr th:each="s : ${sales}">
      <td th:text="${s.id}"></td>
      <td th:text="${s.date}"></td>
      <td th:text="${s.codeCoffee.code}"></td>
      <td th:text="${s.idBarista.id}"></td>
    </tr>
  </table>
  <a th:href="@{add-sales}">
    <button>Add Sales</button>
  </a>
</body>

```

Dari kode di atas, terdapat 2 buah kode yang baru kita lihat dengan keterangan sebagai berikut:

- `<td th:text="${s.codeCoffee.code}" ></td>` memiliki arti bahwa kita mengakses kolom `code_coffee` yang ada di database melalui objek `codeCoffee.code`
- Begitu juga dengan `<td th:text="${s.idBarista.id}" ></td>` yang memiliki arti bahwa kita mengakses kolom `id_barista` yang ada di database melalui objek `idBarista.id`

Jika sudah membuat kode dari beberapa tahapan di atas maka akan menghasilkan output sebagai berikut:



Kenapa tidak ada data yang muncul? Tentu saja karena kita belum mengisi data apapun di tabel tersebut 😊.

11.3 Menambahkan Data

Cara menambahkan datanya pun hampir sama seperti pertemuan kemarin. Untuk lebih lengkapnya perhatikan beberapa langkah berikut ini:

11.3.1 Membuat End-Point Untuk Menerima Inputan

Sebelumnya kita sudah membuat sebuah button di file `show-sales.html` yang mana jika kita klik tombol `"Add Sales"` maka akan menuju pada end-point `"add-sales"`. Sekarang tugas kita adalah membuat sebuah controller untuk menerima inputan dengan kode sebagai berikut

```
@Autowired
private CoffeeRepository coffeeRepository;

@Autowired
private BaristaRepository baristaRepository;

@GetMapping("add-sales")
public String addSales(Model model){
    Sales sales = new Sales();
    model.addAttribute("sales", sales);
    model.addAttribute("kopi", coffeeRepository.findAll());
    model.addAttribute("baris", baristaRepository.findAll());
    return "save-sales";
}
```

Sama seperti kemarin kita menyiapkan sebuah objek **Sales** yang digunakan sebagai tampungan dari inputan yang kita kirim.

Lalu kenapa kita memanggil tabel **coffee** dan **barista** dari database? Ini opsional saja, tidak memakai pun tidak apa-apa. Namun pada program ini kita akan menggunakannya di inputan **<select>**.

11.3.2 Membuat Tampilan Inputan di Halaman Web

Dari kode sebelumnya, method **addSales** mengembalikan nilai string berupa **"save-sales"**. Jadi sekarang kita buat sebuah file HTML dengan nama **save-sales.html** kemudian isi dengan kode berikut ini.

```
<body>
<h1>Add Sales</h1>
<form action="#" th:action="@{save-sale}" method="post">
  <label>Masukan tanggal</label>
  <input type="date" placeholder="Tanggal" th:field="${sales.date}" />
  <br />
  <label>Masukan code_coffee</label>
  <select th:field="${sales.codeCoffee.code}">
    <option
      th:each="s : ${kopi}"
      th:value="${s.code}"
      th:text="${s.code}"
    ></option>
  </select>
  <br />
  <label>Masukan id_barista</label>
  <select th:field="${sales.idBarista.id}">
    <option
      th:each="s : ${baris}"
      th:value="${s.id}"
      th:text="${s.id}"
    ></option>
  </select>
  <br />
  <input type="submit" value="Kirim" />
</form>
</body>
```

Seperti yang disampaikan sebelumnya, pengambilan data dari tabel **coffee** dan **barista** digunakan untuk inputan drop down di tag **<select>** dimana data data tersebut akan ditampilkan di tag **<option>**.

Tujuannya adalah agar kita tidak salah menginputkan `code_coffee` ataupun `id_barista`. Hal ini tidak diwajibkan karena di pertemuan selanjutnya kita akan membahas validasi di back-end (menggunakan package service).

Kode di atas akan menghasilkan output berikut:



Jika di dalam drop down `code_coffee` dan `id_barista` tidak memunculkan apa-apa, maka coba isi dulu data yang ada di kedua tabel tersebut.

Misalnya kita mengirimkan inputan seperti gambar berikut.



Dari form di atas, ketika kita menginputkan data maka data akan di kirimkan oleh atribut `th:action="@{save-sale}"`. Maka dari itu, sekarang kita akan membuat end-point untuk menyimpan data tersebut.

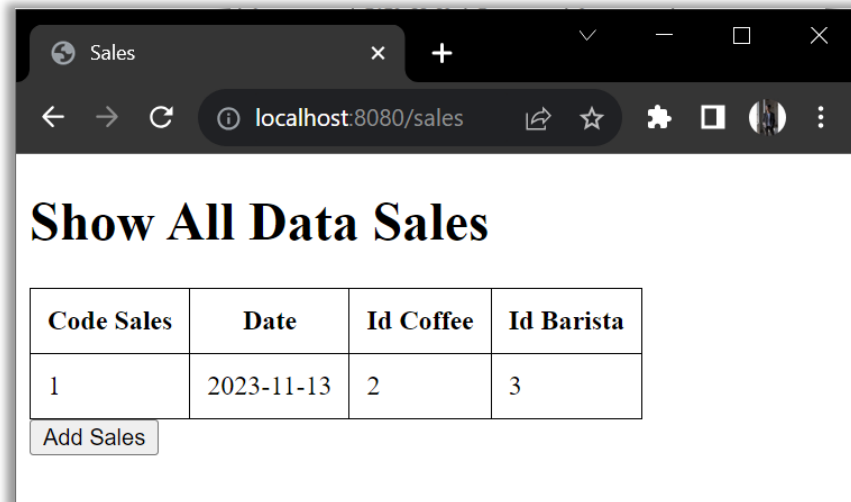
11.3.3 Menyimpan Data ke Database

Sekarang kita harus membuat end-point POST untuk menambahkan data dari client ke database kita dengan menambahkan kode berikut ini.

```
@PostMapping("save-sale")
public String saveSales(Sales sales){
    salesRepository.save(sales);
}
```

```
return "redirect:/sales";  
}
```

Method tersebut akan mengembalikan nilai string **"redirect:/sales"** sehingga jika data yang kita inputkan tadi di kirim, maka akan menghasilkan output berikut:



BAB XII

SEARCH & SORTING

12.1 Search

Fitur search sudah disediakan oleh JPA, jadi kita tidak perlu repot-repot untuk membuat query mencari sebuah data. Untuk menggunakan fitur ini kita harus menggunakan sebuah method abstrak yang ada di interface Repository.

Fitur search ini terbagi menjadi dua, yaitu:

- **findAllBy**, mencari semua data berdasarkan perintah
- **findBy**, mencari sebuah data berdasarkan perintah

Misalnya untuk mencari id sebuah entitas, maka kita harus menggunakan method **findById**. Atau jika ingin mencari semua data berdasarkan nama, maka kita harus menggunakan method **findAllByName**.

Sebagai contoh, mari gunakan project **coffe_ah** sebelumnya. Kemudian tambahkan kode berikut ini di interface **CoffeeRepository.java**

```
public interface CoffeeRepository extends JpaRepository<Coffee, Integer> {  
    List<Coffee> findByMerkContainingIgnoreCase(String merk);  
}
```

Kode di atas berarti kita ingin mencari sebuah data berdasarkan **merk** yang kita inputkan. Kemudian arti dari **ContainingIgnoreCase** artinya data akan ditampilkan walaupun persamaan katanya minimum. Untuk lebih jelasnya nanti kalian coba sendiri hasilnya.

Kemudian di file html kita tambahkan elemen form untuk menampung inputan. Contohnya adalah sebagai berikut:

```
<div class="search">  
  <form action="#" th:action="@{/search}" method="get">  
    <input  
      type="text"  
      placeholder="Cari Berdasarkan Nama Merk..."  
      name="search"  
    />  
    <input type="submit" value="Cari" />  
  </form>  
</div>
```

```
</form>
</div>
```

Kode di atas akan menampung inputan dari kita, kemudian data yang ditampung diteruskan ke endpoint **/sarch** yang isinya adalah sebagai berikut:

```
@GetMapping("/search")
public String searchByName(@RequestParam(name = "search")
                           String merk, Model model){
    List<Coffee> coffees = coffeeRepository.findByMerkContainingIgnoreCase(merk);
    model.addAttribute("coffees", coffees);
    return "show-coffee";
}
```

Kalian juga bisa membuat fitur search lain, misalnya search berdasarkan nama merk, harga, kuantitas, dll.

12.2 Sorting

Proses sorting di JPA terbilang cukup gampang. Karena kita hanya perlu menggunakan sebuah method yang sudah disiapkan otomatis oleh JPA. Berikut adalah cara penggunaannya.

Pertama kita tambahkan kode berikut di interface `CoffeeRepository.java`

```
public interface CoffeeRepository extends JpaRepository<Coffee, Integer> {
    List<Coffee> findAllOrderByMerkAsc();
    List<Coffee> findAllOrderByMerkDesc();
}
```

Jika kita ingat pada pelatihan basis data lalu, kita menggunakan query **ORDER BY ... ASC** untuk mengurutkan entitas dari yang terkecil hingga ke yang terbesar. Di JPA juga sama, hanya menambahkan keyword **findAllBy** atau **findBy** saja.

Kemudian di file HTML kita perlu membuat sebuah tombol untuk melakukan penyortiran. Berikut adalah contoh kodenya:

```
<a th:href="@{/sort-by-merk-asc}">
    <button>Sort By Merk Asc</button>
</a>
```

Kode di atas merujuk pada end-point **sort-by-merk-asc** yang berisi:

```
@GetMapping("/sort-by-merk-asc")
public String sortByMerkAsc(Model model){
    List <Coffee> coffees = coffeeRepository.findAllOrderByMerkAsc();
    model.addAttribute("coffees", coffees);
    return "show-coffee";
}
```